laSalle

UNIVERSITAT RAMON LLULL

Escola Tècnica Superior d'Enginyeria La Salle

Treball Final de Grau

Grau en Enginyeria Informàtica

MISSME: Un Simulador Visual de Jerarquies de Memòria

Alumne

Adrià Martínez Gallifa

Professor Ponent

Guillem Guerrero Massana

ACTA DE L'EXAMEN
DEL TREBALL FI DE CARRERA

Reunit el Tribunal qualificador en el dia de la data, l'alumne:
Adrià Martínez Gallifa

Va exposar el seu Treball de Fi de Carrera, el qual va tractar sobre el tema següent:

MISSME: Un Simulador Visual de Jerarquies de Memòria

Acabada l'exposició i contestades per part de l'alumne les objeccions formulades pels Srs. membres del tribunal, aquest valorà l'esmentat Treball amb la qualificació de

Barcelona,

VOCAL DEL TRIBUNAL VOCAL DEL TRIBUNAL

PRESIDENT DEL TRIBUNAL

Resum

missme és una eina d'escriptori que simula visualment jerarquies de memòria per a estudiants d'arquitectura de computadors. Permet configurar gràficament un nombre no acotat de nivells de cau, cadascun Von Neumann o Harvard, i el camí complet de traducció d'adreces amb TLB, taula de pàgines multinivell, marcs i intercanvi; l'usuari escriu un programa en RISC-V, l'executa per acció atòmica, per instrucció o de manera contínua, i inspecciona el contingut real de totes les estructures mentre s'executa.

El motor, en Rust, i la interfície, en Vue, es comuniquen a través de Tauri i queden separats de manera estricta, cosa que ha permès validar-lo amb 474 proves automàtiques i set exercicis contrastats manualment amb el marc teòric. A diferència de les eines docents existents, *missme* modela latències i permet contrastar l'AMAT mesurat amb el calculat a mà, i demostra els quatre objectius pedagògics fixats: relacionar, predir, distingir i seguir el comportament d'una jerarquia real.

Paraules clau: simulador de jerarquia de memòria, memòria cau, memòria virtual, RISC-V, arquitectura de computadors, eina docent, Rust, Vue.

Resumen

missme es una herramienta de escritorio que simula visualmente jerarquías de memoria para estudiantes de arquitectura de computadores. Permite configurar gráficamente un número no acotado de niveles de caché, cada uno Von Neumann o Harvard, y el camino completo de traducción de direcciones con TLB, tabla de páginas multinivel, marcos e intercambio; el usuario escribe un programa en RISC-V, lo ejecuta por acción atómica, por instrucción o de forma continua, e inspecciona el contenido real de todas las estructuras mientras se ejecuta.

El motor, en Rust, y la interfaz, en Vue, se comunican a través de Tauri y quedan separados de forma estricta, lo que ha permitido validarlo con 474 pruebas automáticas y siete ejercicios contrastados manualmente con el marco teórico. A diferencia de las herramientas docentes existentes, *missme* modela latencias y permite contrastar el AMAT medido con el calculado a mano, y demuestra los cuatro objetivos pedagógicos fijados: relacionar, predecir, distinguir y seguir el comportamiento de una jerarquía real.

Palabras clave: simulador de jerarquía de memoria, memoria caché, memoria virtual, RISC-V, arquitectura de computadores, herramienta docente, Rust, Vue.

Abstract

missme is a desktop tool that visually simulates memory hierarchies for computer architecture students. It lets users graphically configure an unbounded number of cache levels, each Von Neumann or Harvard, and the full address translation path with a TLB, a multilevel page table, frames and swap; the user writes a RISC-V program, runs it by atomic action, by instruction, or continuously, and inspects the real content of every structure while it executes.

The engine, in Rust, and the interface, in Vue, communicate through Tauri and are kept strictly separate, which made it possible to validate it with 474 automated tests and seven exercises checked manually against the theoretical framework. Unlike existing teaching tools, *missme* models latencies and allows contrasting the measured AMAT against one calculated by hand, demonstrating the four pedagogical objectives set out for the tool: relating, predicting, distinguishing, and following the behavior of a real hierarchy.

Keywords: memory hierarchy simulator, cache memory, virtual memory, RISC-V, computer architecture, educational tool, Rust, Vue.

Agraïments

Vull agrair, en primer lloc, al meu tutor, Guillem Guerrero, l'acompanyament durant tot el desenvolupament d'aquest treball. La seva disponibilitat per resoldre dubtes, els seus comentaris sobre les decisions de disseny i la confiança que hi ha dipositat des del principi han estat fonamentals per tirar-lo endavant.

També vull donar les gràcies a la meva família, que m'ha acompanyat durant tots aquests anys de carrera i ha fet costat en els moments més difícils del projecte. A la meva parella, per la paciència i el suport constant durant els mesos en què aquest treball ha ocupat bona part del meu temps. Als meus amics, per animar-me i preguntar com anava sempre que en tenien l'ocasió. I finalment, al professorat que, al llarg del grau, m'ha donat les eines i els coneixements que han fet possible arribar fins aquí.

Índex

1 Introducció **1**
1.1 Descripció del projecte 1
1.2 Estructura de la memòria 2

2 Fonaments teòrics **3**
2.1 Fonaments de la jerarquia de memòria 3
2.2 La jerarquia completa: de la memòria cau al disc 5
2.3 Organització interna de la memòria cau 8
2.4 Memòria virtual 11
2.5 Mètriques de rendiment 15
2.6 Simuladors i eines existents 18

3 Abast i requisits **23**
3.1 Públic objectiu i escenaris d'ús 23
3.2 Requisits funcionals 24
3.3 Paràmetres configurables i valors fixats 27
3.4 Simplificacions assumides 32
3.5 Requisits no funcionals 35
3.6 Síntesi de l'abast 38

4 Tecnologies i eines **39**
4.1 Criteris de selecció 39
4.2 Marc d'aplicació 39
4.3 Llenguatge del motor 41
4.4 Tecnologia de la interfície 41
4.5 Eines de suport 42

5 Disseny de l'arquitectura **43**
5.1 Visió general del sistema 43
5.2 Elecció de l'arquitectura MVC 44
5.3 Model de dades de la jerarquia 45
5.4 El motor de simulació 48
5.5 Comunicació Tauri 52
5.6 Gestió de l'estat a la interfície 54
5.7 Patrons de disseny aplicats 54

6 Implementació **57**
6.1 De l'assemblador al codi executable 57
6.2 El simulador de CPU 60

6.3 El motor de memòria . . . 62
6.4 Execució contínua . . . 65
6.5 Estadístiques i mètriques . . . 66
6.6 Configuració de la jerarquia des de la interfície . . . 68
6.7 Visualització de l'estat en execució . . . 70
6.8 Cicle de vida i coordinació a Vue . . . 73

7 Validació 75
7.1 Estratègia de validació . . . 75
7.2 Proves automàtiques del motor . . . 76
7.3 Proves automàtiques de la interfície . . . 79
7.4 Verificació manual . . . 80
7.5 Verificació dels requisits no funcionals . . . 85
7.6 Casos d'ús demostratius . . . 86
7.7 Limitacions de la validació . . . 88

8 Ús d'eines d'Intel·ligència Artificial 89
8.1 Eines utilitzades . . . 89
8.2 Finalitat i abast de l'ús . . . 89
8.3 Verificació dels resultats obtinguts . . . 90

9 Conclusions 91

10 Línies de futur 93

11 Planificació temporal 95

12 Referències 97

Apèndixs 101

A Subconjunt de RISC-V admès 101
A.1 RV32I . . . 101
A.2 RV64I . . . 103
A.3 Extensió M . . . 104
A.4 Extensions de coma flotant . . . 104
A.5 Pseudoinstruccions . . . 107
A.6 Directives . . . 108
A.7 Noms de registre . . . 109
A.8 Sintaxi admesa . . . 110
A.9 Límits i comprovacions . . . 111

B Programes dels exercicis de verificació **113**
B.1 E1. Localitat espacial 113
B.2 E2. Fallades de conflicte 113
B.3 E3. Divergència entre polítiques de reemplaçament 114
B.4 E4. Polítiques d'escriptura 114
B.5 E5. Mètriques amb dos nivells 115
B.6 E6. Traducció d'adreces 115
B.7 E7. Thrashing per pressió de marcs 116

C Detall de les proves per fitxer **117**
C.1 Proves del motor 117
C.2 Proves de la interfície 118

Índex de figures

1 Increment de la velocitat de CPU i RAM des de 1980 fins al 2000 . . 3
2 Patró de localitat observat durant l'execució d'un programa 4
3 Esquema d'implementació d'un bit amb la tecnologia SRAM 6
4 Esquema d'implementació d'un bit amb la tecnologia DRAM 6
5 Jerarquia de memòria per a un ordinador de sobretaula o portàtil . . 8
6 Exemple de mapeig directe d'una memòria cau 9
7 Miss rate respecte la mida de bloc 9
8 Demostració de selecció de posició per a bloc i lectura del mateix . . 10
9 Traducció d'una adreça virtual a una física 12
10 La TLB actua com una cache per a la taula de pàgines per les entrades que apunten cap a la memòria física 14
11 Flux de treball d'una traducció a partir de la TLB 15
12 Diagrama de components 43
13 Diagrama de paquets 45
14 Diagrama de classes de configuració 46
15 Diagrama de classes de l'estat en execució 48
16 Estats del cursor 50
17 Seqüència d'un accés amb fallada de pàgina 51
18 Diagrama de seqüència d'una comanda i la seva resposta 53
19 Diagrama de seqüència d'un esdeveniment amb els seus subscriptors . 53
20 Diagrama de components del front-end 54
21 Planificació temporal en diagrama de Gantt 95
22 Distribució de l'esforç per tipus de tasca 96

Índex de taules

1 Estructura bàsica d'una jerarquia de memòria 7
2 Comparativa de simuladors: entrada i modelatge de la jerarquia . . . 20
3 Comparativa de simuladors: execució, inspecció i distribució 21
4 Paràmetres de la jerarquia de memòria cau 31
5 Paràmetres de memòria virtual 32
6 Famílies d'aplicació avaluades contra els criteris de selecció 39
7 Marcs d'aplicació d'escriptori amb interfície web 40
8 Marcs de desenvolupament d'interfícies web considerats. 42
9 Correspondència entre els requisits i les vies de validació 76
10 Repartiment dels casos de prova del motor 77
11 Cobertura de línies del motor per subsistema 79
12 Resum dels exercicis de verificació manual 81
13 Resultats dels exercicis amb jerarquia de memòria cau 82
14 Comportament de les polítiques de reemplaçament 83
15 Verificacions creuades de l'exercici E7 85
16 RV32I: operacions aritmètiques i lògiques entre registres 101
17 RV32I: operacions amb immediat i desplaçaments 101
18 RV32I: accés a memòria 102
19 RV32I: salts 102
20 RV32I: immediats llargs i instruccions de sistema 102
21 RV64I: instruccions afegides sobre RV32I 103
22 Extensió M: multiplicació i divisió enteres 104
23 Operacions disponibles a les extensions F, D i Q. Substituïu `.x` pel sufix de l'extensió 105
24 Conversions i moviments entre formats 106
25 Pseudoinstruccions i la seva expansió 108
26 Directives reconegudes 108
27 Àlies dels registres enters 109
28 Àlies dels registres de coma flotant 109
29 Formats d'immediat admesos 110
30 Regles de formació d'etiquetes 110
31 Formes de comentari admeses 110
32 Rangs de salt comprovats durant la generació de codi 111
33 Comprovacions de l'anàlisi semàntica 112
34 Repartiment dels casos de prova del motor per fitxer 118
35 Repartiment dels casos de prova de la interfície per fitxer 119

Acrònims

AMAT Temps Mitjà d'Accés a Memòria (Average Memory Access Time)

API Interfície de Programació d'Aplicacions (Application Programming Interface)

CD Comprovació Directa

CPI Cicles Per Instrucció (Cycles Per Instruction)

CPU Unitat Central de Processament (Central Processing Unit)

CSS Fulls d'Estil en Cascada (Cascading Style Sheets)

DOM Model d'Objectes del Document (Document Object Model)

FIFO Primer a Entrar, Primer a Sortir (First In, First Out)

ISA Arquitectura del Joc d'Instruccions (Instruction Set Architecture)

LFU Menys Freqüentment Usat (Least Frequently Used)

LRU Usat Menys Recentment (Least Recently Used)

MPKI Fallades per Miler d'Instruccions (Misses Per Kilo Instructions)

MV Memòria Virtual

MVC Model-Vista-Controlador (Model-View-Controller)

PI Proves de la Interfície

PM Proves del Motor

PPN Número de Pàgina Física (Physical Page Number)

RAM Memòria d'Accés Aleatori (Random Access Memory)

RISC Computador amb Joc d'Instruccions Reduït (Reduced Instruction Set Computer)

RV32I RISC-V d'Enters de 32 Bits (RISC-V 32-bit Integer)

RV64I RISC-V d'Enters de 64 Bits (RISC-V 64-bit Integer)

SVG Gràfics Vectorials Escalables (Scalable Vector Graphics)

TLB Memòria de Traduccions (Translation Lookaside Buffer)

VM Verificació Manual

VPN Número de Pàgina Virtual (Virtual Page Number)

1 Introducció

1.1 Descripció del projecte

La jerarquia de memòria s'explica a classe amb diagrames estàtics i exercicis de paper, mentre que el comportament que importa és **dinàmic i acumulatiu**. Una taxa de fallada s'entén veient com un *patró d'accés* concret omple i buida uns conjunts, i aquesta part costa de transmetre amb una figura fixa o una taula de resultats. D'aquest **desajust** entre com s'explica la matèria i com funciona de veritat surt la motivació del projecte.

Aquest treball desenvolupa *missme*, un **simulador visual** de jerarquies de memòria pensat per a l'aprenentatge. L'usuari construeix gràficament la jerarquia que vol estudiar, amb tants nivells de memòria cau com necessiti, cadascun configurable de manera independent en geometria, polítiques de reemplaçament i escriptura, i amb la possibilitat de separar instruccions i dades a qualsevol nivell. També pot activar la **memòria virtual**, amb TLB, taula de pàgines multinivell, marcs de memòria física i espai d'intercanvi a disc.

Sobre aquesta jerarquia s'executa un programa escrit en assemblador RISC-V dins de la mateixa aplicació. L'execució avança per *acció atòmica*, per *instrucció* o de manera *contínua* a la velocitat que l'usuari triï, i s'hi poden col·locar punts de ruptura. Mentre el programa avança, el contingut real de totes les estructures és consultable, i la interfície indica quin element s'està accedint i si l'accés ha estat un **encert** o una **fallada**. Les mètriques de rendiment, entre elles l'AMAT, el CPI i l'MPKI, es deriven dels cicles acumulats durant l'execució i s'actualitzen amb ella.

L'usuari final és un **estudiant de grau** que cursa o ha cursat una assignatura d'arquitectura de computadors, i que ja sap què és una memòria cau i sap programar en assemblador. Els públics secundaris són el professorat que ha de demostrar un mecanisme a classe i qui vol provar una configuració per curiositat. Queden fora del públic objectiu la recerca i el *profiling*, per als quals ja existeixen eines millors.

L'eina **simplifica el model de manera deliberada**. No modela coherència de memòria cau, multiprocessador, prefetch ni execució fora d'ordre, entre altres mecanismes. Cada omissió respon al mateix criteri: eliminar les variables que en un sistema real se superposen és el que permet que l'efecte d'una decisió de configuració sigui *atribuïble* a aquella decisió i no a la suma de moltes.

Tècnicament, l'aplicació és d'**escriptori** i s'executa a la màquina de l'usuari sense dependre de cap servei extern. El motor de simulació està escrit en **Rust** i la interfície en **Vue**, comunicats a través de **Tauri**.

1.2 Estructura de la memòria

- **Fonaments teòrics**. Recull els fonaments teòrics de la jerarquia de memòria que l'eina ha de reproduir, i revisa els simuladors i les eines docents que ja existeixen.
- **Abast i requisits**. Defineix el públic objectiu i els escenaris d'ús, i fixa els requisits funcionals i no funcionals, els paràmetres configurables i el que queda fora del model.
- **Eines i tecnologies**. Justifica l'elecció de les tecnologies emprades i els costos que comporta cadascuna.
- **Disseny de l'arquitectura**. Descriu com es reparteix la responsabilitat entre les parts del sistema, com es representen les dades i com es resol el problema d'aturar un accés a memòria enmig del seu recorregut.
- **Implementació**. Explica com s'ha construït cada part, des del compilador d'assemblador fins a la visualització, i quins problemes concrets van aparèixer en fer-ho.
- **Validació**. Presenta les proves automàtiques, la verificació manual contra exercicis resolts i els casos d'ús que demostren el comportament de l'eina.
- **Conclusions**. Comprova l'assoliment dels objectius fixats a l'abast i valora el resultat del projecte.
- **Línies de futur**. Declara els enfocaments amb què es podria continuar el treball.
- **Apèndixs**. Recullen el subconjunt de RISC-V admès per l'eina, els programes dels exercicis de verificació i el detall de les proves per fitxer.

2 Fonaments teòrics

2.1 Fonaments de la jerarquia de memòria

2.1.1 CPU i Memòria

El processador pot consumir dades a un ritme molt més alt del que la memòria pot lliurar. La causa és que les dues tecnologies s'han optimitzat per objectius diferents: en un processador el que interessa és la **velocitat**, i en la DRAM la **capacitat**. La velocitat de la CPU ha crescut al voltant d'un **60%** anual mentre que la de la memòria només ho ha fet un **10%** [1], de manera que la distància entre totes dues creix **exponencialment** any rere any [10] i és previsible que ho continuï fent.

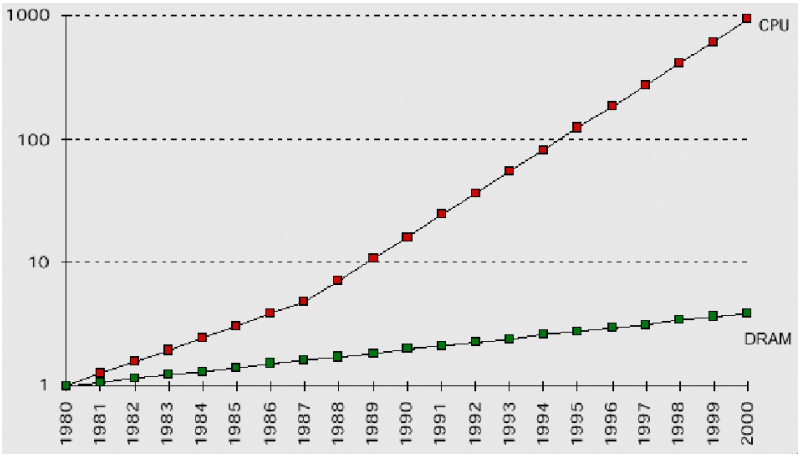


Figura 1: Increment de la velocitat de CPU i RAM des de 1980 fins al 2000. [1]

Amb aquesta diferència, un únic bloc de memòria pot convertir-se en el principal **coll d'ampolla** de gran part dels ordinadors moderns [1]. D'aquí ve la necessitat de dissenyar un sistema que accelerés el lliurament de dades i donés al programador la *il·lusió* d'una memòria **il·limitada i ràpida**. El disseny d'aquesta memòria *ideal* es recolza en dos principis: la **localitat espacial** i la **localitat temporal**. [2]

2.1.2 Principis de localitat

Els programes **no accedeixen uniformement** a tot el codi ni a totes les dades. El **principi de localitat** estableix que, en qualsevol moment, un programa treballa sobre porcions petites del seu espai. [2] N'hi ha de dos tipus:

- **Localitat Espacial**: si un objecte s'ha referenciat, és probable que els objectes en *adreces properes* també ho siguin. Les cadenes de caràcters en són un exemple, ja que treballar-hi normalment implica llegir posicions consecutives.
- **Localitat Temporal**: si un objecte s'ha referenciat, és probable que torni a ser-ho en un *curt termini*. Un bucle, que repeteix el mateix bloc de codi, n'és el cas més clar.

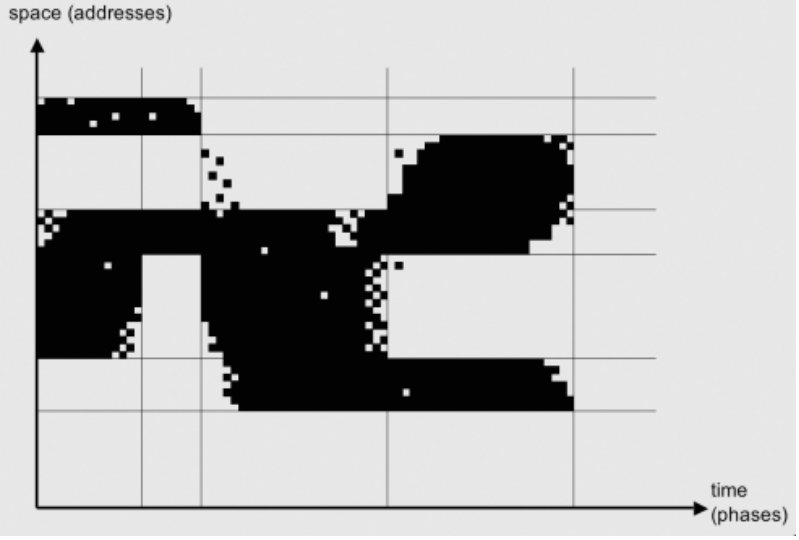


Figura 2: Patró de localitat observat durant l'execució d'un programa. [2]

Aprofitant aquests principis, avui la memòria d'un ordinador s'implementa com una **jerarquia**. La idea és combinar *diversos nivells* amb velocitats i capacitats diferents, de manera que el conjunt ofereixi la **major capacitat al menor preu possible** amb el **temps d'accés de la memòria més ràpida**. [15]

2.1.3 Arquitectures Von Neumann i Harvard

Les arquitectures **Von Neumann** i **Harvard** són els dos models bàsics d'organització de memòria. [22] A la pràctica se'n troben casos de totes dues i també de la combinació que se'n deriva, l'**arquitectura híbrida**.

L'**arquitectura Von Neumann** guarda instruccions i dades en la *mateixa memòria*. Els avantatges venen de necessitar una sola unitat que centralitza les dades i els accessos: el sistema és més *simple*, més *flexible* i més *barat*. El desavantatge és el **coll d'ampolla** que provoca compartir bus, ja que no es poden rebre dades i instruccions alhora.

L'**arquitectura Harvard** separa instruccions i dades en dues memòries amb busos diferents. Aquesta separació li dona una **velocitat major** de processament, perquè pot passar-les en paral·lel, a canvi d'un sistema **més complex, més car i menys flexible**. Els punts forts i febles de les dues arquitectures són, doncs, complementaris.

L'**arquitectura híbrida** busca els avantatges de totes dues i és la que adopten la majoria de processadors de propòsit general. [6] Habitualment els *nivells més propers a la CPU* són Harvard, de manera que en la memòria més ràpida s'aprofita la **velocitat de Harvard** mentre que la resta de nivells conserven la **flexibilitat de Von Neumann**.

2.2 La jerarquia completa: de la memòria cau al disc

2.2.1 Tecnologies

Les jerarquies de memòria actuals es componen de **tres** tecnologies principals : **SRAM** (*Static Random Access Memory*) per a la *memòria cau*, **DRAM** (*Dynamic Random Access Memory*) per a la *memòria principal* i el **disc magnètic** per a l'emmagatzematge. [15] Per damunt d'aquestes tres encara hi ha els **registres** del processador, el nivell més ràpid de tots, gestionat pel compilador en lloc del *hardware*.

La **memòria cau** ocupa els nivells més propers al processador i és la *més ràpida* i alhora la *més cara*. El cost per damunt de la seva quasi homònima DRAM ve de la **superfície per bit**: en el cas més habitual calen **6 transistors MOSFET** [14], com es pot observar en la figura 3. Quatre transistors ($M_{1\text{-}4}$) guarden el bit formant un *biestable* i els altres dos ($M_{5,6}$) controlen l'accés al biestable durant les lectures i escriptures. La velocitat ve de necessitar només *canvis de voltatge* per fer una operació, cosa que al mateix temps fa la memòria **volàtil**, ja que el contingut es perd en desconnectar l'alimentació. El resultat és una memòria **molt ràpida però molt costosa econòmicament i en espai**.

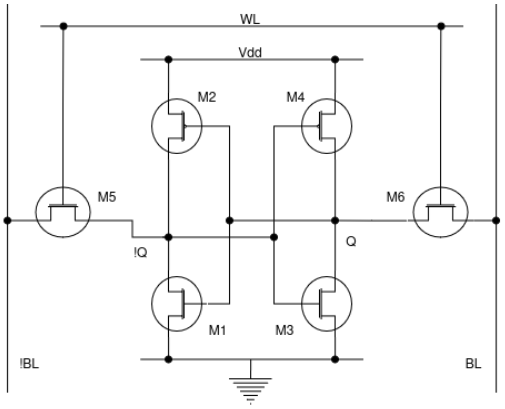


Figura 3: Esquema d'implementació d'un bit amb la tecnologia SRAM. Elaboració pròpia a partir de [14]

La **memòria principal** ocupa els nivells entre la memòria cau i el disc, i el seu avantatge és combinar una *capacitat decent* amb un *temps d'accés ràpid*. Com es pot veure en la figura 4, un bit de DRAM s'implementa amb **un transistor MOSFET i un condensador** que guarda el bit. [14] La superfície per bit és molt més reduïda que en la SRAM pel nombre de components, i avui es construeix el condensador dins del mateix transistor, cosa que redueix encara més el cost espacial. El condensador, però, és el que marca la velocitat de la tecnologia: mantenir una dada exigeix un *refresc* periòdic, i cada lectura la descarrega, de manera que després cal tornar-la a carregar. La memòria també resulta **volàtil**. En conjunt, doncs, és una memòria **ràpida amb un cost decent**.

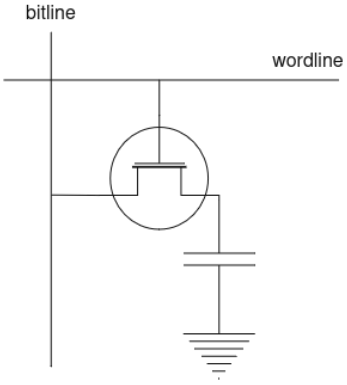


Figura 4: Esquema d'implementació d'un bit amb la tecnologia DRAM. Elaboració pròpia a partir de [14]

El **disc magnètic** permet guardar *grans quantitats de dades* amb una *latència altíssima*. [6] La característica que més el diferencia de la resta és que **no és volàtil**: les dades persisteixen encara que es desconnecti l'alimentació. Aquesta combinació de **gran capacitat i permanència** el fa la tecnologia adequada per a l'últim nivell de la jerarquia.

Velocitat		**Mida**	**Cost ($/bit)**	**Tecnologia**
	CPU			
Major	*Mem'oria*	Menor	Major	SRAM
	Mem'oria			DRAM
Menor	*Mem'oria*	Major	Menor	Disc

Taula 1: Estructura bàsica d'una jerarquia de memòria. Elaboració pròpia a partir de [15]

2.2.2 El patró dominant

La configuració de la jerarquia depèn del problema que es vulgui resoldre. El cas més habitual és el d'un **processador de propòsit general**, on el condicionant principal és el *desconeixement de la càrrega de treball*.

Per damunt de la memòria cau hi ha els **registres** del nucli, l'únic nivell *visible a la ISA* (*Instruction Set Architecture*) i gestionat pel **compilador** en comptes del *hardware*. [15]

La **L1** és la cau *més petita* perquè és la més propera al processador i la seva **latència ha de ser mínima**. Si fos més gran, l'encert trigaria més cicles, cosa que s'aprofundeix en l'apartat 2.3.1. Com la resta de memòries cau, la controla el *hardware*.

La **L2** existeix per **absorbir les fallades de L1**. [6] Com que només li arriben els accessos que ja han fallat, es pot permetre ser més lenta a canvi de ser més gran, i és el primer nivell on la *latència deixa de ser crítica*.

La **L3** queda fora de l'abast del simulador d'aquest projecte perquè s'utilitza de manera *compartida* en processadors amb múltiples nuclis. Habitualment L1 i L2 són *locals* de cada nucli i L3 és *compartida* entre tots. Ofereix una **capacitat molt elevada** a costa d'una latència més alta i de la **competència entre nuclis**.

La **RAM** és el primer *salt de tecnologia* a DRAM, amb un increment considerable de la latència, i el primer nivell **fora del xip**, amb el cost de comunicació que això comporta. A canvi guanya *flexibilitat* respecte de la memòria cau, ja que la capacitat es pot ampliar. També és el primer actor de la **memòria virtual**, que s'aprofundeix

a l'apartat 2.4, i per tant el primer que no controla el *hardware* sinó el **OS** (*Operative System*). La relació cau-RAM es reprodueix aquí com a **RAM-Disc**.

L'últim nivell és el **Disc**, caracteritzat per la *no volatilitat* i la *capacitat*.

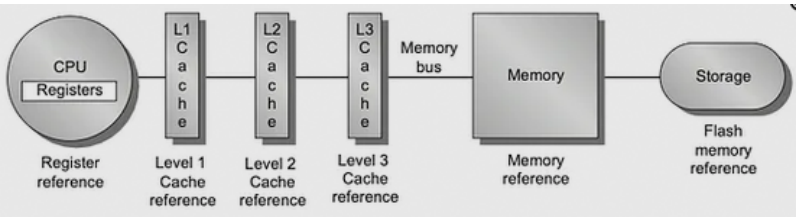


Figura 5: Jerarquia de memòria per a un ordinador de sobretaula o portàtil. Font: [15]

2.3 Organització interna de la memòria cau

Aquesta secció descriu com la memòria cau decideix *on col·locar* un bloc, *com trobar-lo*, *quin treure* quan és plena i *què fer* amb les escriptures.

2.3.1 Funció de mapeig i associativitat

Com que la memòria principal és molt més gran que la memòria cau, **moltes adreces han de compartir espai**. El **mapeig** és la regla que determina quines posicions pot ocupar un bloc, i l'**associativitat** amplia el nombre de posicions possibles a canvi d'augmentar el *temps per recórrer-les*. D'aquí surten tres implementacions: [15]

- **Mapeig directe**: cada bloc té una *única via possible*, determinada per la seva adreça. L'avantatge és que no cal cap comprovació addicional. El problema apareix quan dos blocs apunten a la mateixa posició i **s'expulsen mútuament** encara que la resta de la cau estigui buida.
- **Totalment associativa**: *qualsevol bloc pot anar a qualsevol posició*. Elimina els conflictes a canvi d'haver de comparar totes les posicions, i per això es reserva a estructures petites com la **TLB**, que es veu a l'apartat 2.4.
- **Associativa per conjunts**: el punt mitjà. L'adreça determina un *conjunt* i dins del conjunt el bloc pot ocupar qualsevol *via*. Els altres dos són **casos particulars** d'aquest, amb una via per adreça i amb un sol conjunt respectivament.

El nombre de conjunts es **deriva** de la mida total, la mida de bloc i l'associativitat, de manera que fixant-ne dos el tercer queda determinat. Cal remarcar també el

cost de l'associativitat: com més gran és el conjunt, més comparacions calen, i els nivells amb associativitat més alta *augmenten la latència significativament*. [7]

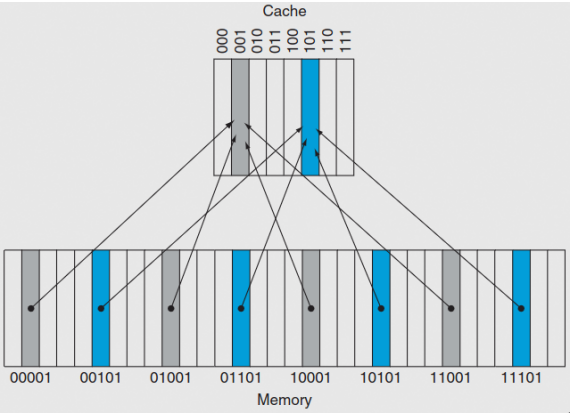


Figura 6: Exemple de mapeig directe d'una memòria cau. Font: [15]

2.3.2 Blocs i descomposició de l'adreça

La cau no transfereix bytes individuals sinó **blocs** de mida fixa. Hi ha dues raons: el principi de *localitat espacial* de l'apartat 2.1.2, i que el cost d'una transferència de memòria és majoritàriament *fix*, de manera que portar més dades per accés surt gairebé gratis. La mida del bloc també importa, perquè amb blocs massa petits no s'aprofita la localitat i amb blocs *massa grans* entren dades que no es faran servir. La mida òptima depèn de la **càrrega**. [15]

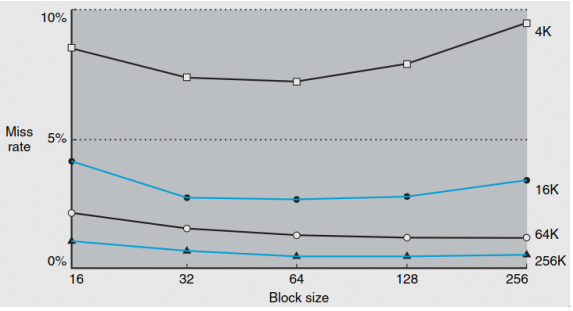


Figura 7: Miss rate respecte la mida de bloc. Font: [15]

Una adreça dins de la cau té tres camps: el **desplaçament** tria el byte dins del bloc, l'**índex** tria el conjunt i l'**etiqueta** identifica quin bloc ocupa la posició. Els bits van *de menys a més* pes, com es veu a la figura 8, i els últims 10 bits de les adreces de la

memòria inferior fan d'índex. Si es fessin servir els bits de major pes per a l'índex, adreces consecutives **col·lisionarien constantment** i es perdria l'aprofitament del principi de localitat de l'apartat 2.1.2. [15]

En relació amb l'apartat 2.3.1, si **dupliquem l'associativitat** el nombre de conjunts es redueix a la meitat, cosa que **treu un bit de l'índex i l'afegeix a l'etiqueta**.

Queden els **bits d'estat**. El **bit de validesa** indica si l'entrada conté dades reals, i el **bit de brut** indica que s'ha modificat una dada que encara no s'ha propagat. Aquest segon només té sentit quan la memòria és *write-back*, com es veu a l'apartat 2.3.4.

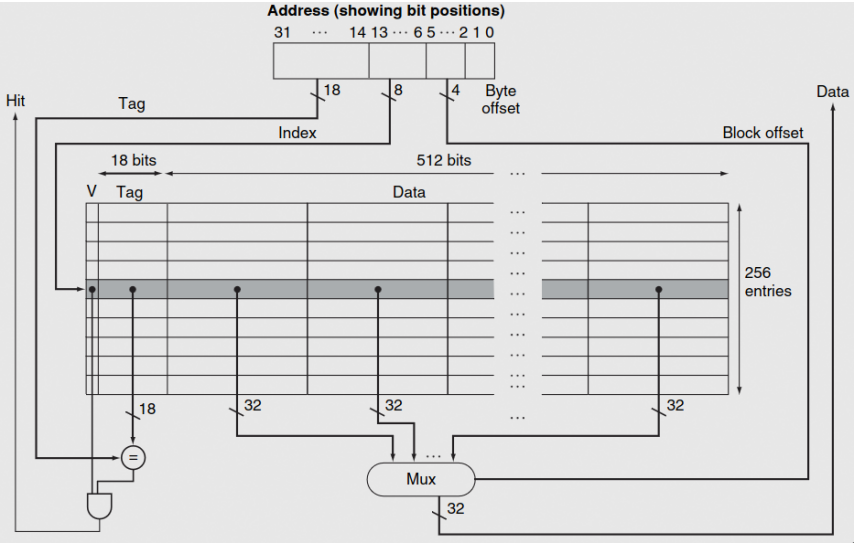


Figura 8: Demostració de selecció de posició per a bloc i lectura del mateix. Font: [15]

2.3.3 Polítiques de reemplaçament

Les polítiques de reemplaçament **només s'apliquen quan no hi ha mapeig directe**, segons l'apartat 2.3.1: amb una sola via possible només hi ha un candidat a ser reemplaçat. [15]

- **LRU (*Least Recently Used*)**: expulsa el *menys recentment usat*. És l'aproximació directa de la localitat temporal i sol donar bons resultats, a canvi de **mantenir un ordre** entre les vies que cal actualitzar a cada accés.
- **FIFO (*First In, First Out*)**: una *cua*. Expulsa el que fa més temps que ha entrat. És **molt barata**, només cal un comptador per conjunt, però **ig-**

nora la reutilització, de manera que pot eliminar un bloc que s'està usant constantment.

- **Random**: completament aleatòria, amb *hardware* mínim. Funciona sorprenentment bé amb *associativitat alta*. [7] L'inconvenient és que **no es pot predir**.
- **LFU (*Least Frequently Used*)**: expulsa el bloc amb *menys usos*. Un bloc molt utilitzat en una fase inicial pot **quedar-se a la cau quan ja no es necessita**.

2.3.4 Polítiques d'escriptura

Les *lectures* **no alteren res** i per tant no plantegen cap decisió. Amb les *escriptures* cal decidir què fer amb la nova dada, i d'aquí surten les polítiques d'escriptura. [15]

- **Write-Through**: quan una escriptura arriba al nivell, *s'actualitza automàticament el nivell inferior*. Les dades ja són més avall de la jerarquia i l'expulsió no requereix cap acció. El cost és de **trànsit**: cada una de les escriptures sobre un mateix bloc s'ha d'escriure també un nivell a sota.
- **Write-Back**: la modificació *no baixa* immediatament. Es modifica el bloc i s'activa el **bit de brut** de l'apartat 2.3.2. Les escriptures repetides sobre el mateix bloc no afegeixen trànsit, però les **expulsions són més costoses** perquè cal actualitzar el nivell inferior.

Hi ha una segona decisió, independent de l'anterior: què fer quan es produeix una **fallada** durant una escriptura. També hi ha dues opcions:

- **Write-Allocate**: *es porta el bloc a la memòria cau* i després s'hi escriu. Té sentit quan s'espera tornar a modificar el bloc en poc temps.
- **Sense Write-Allocate**: s'*escriu directament* al nivell on és la dada, sense pujar-la. Té sentit quan es fa una única escriptura sobre un bloc gran de dades, on portar-lo seria feina inútil.

2.4 Memòria virtual

Fins ara s'han tractat mecanismes gestionats pel *hardware* i **invisibles al programa que s'executa** (no pas al sistema operatiu). La memòria virtual afegeix una capa diferent, una tècnica que utilitza la **memòria principal com una memòria cau per al Disc**. [15] Parteix del fet que la memòria requerida pels programes pot ser molt més gran que la memòria principal mentre que, en un moment donat, *només*

se'n fa servir una petita porció. Aquest fet té una sèrie d'implicacions que es veuen en aquesta secció.

2.4.1 Traducció d'adreces

El programa treballa amb **adreces virtuals** que *no designen posicions físiques.* Entre l'adreça que el programa emet i la que arriba a la memòria s'introdueix una **capa de traducció**, cosa que crea la distinció entre **espai físic** i **espai virtual**.

La traducció opera sobre *blocs de mida fixa* i no sobre adreces individuals. A l'espai virtual s'anomenen **pàgines** i al físic **marcs**. La descomposició és la mateixa que a la relació cau-RAM de l'apartat 2.3.2: la part alta és el número de pàgina i la baixa el desplaçament. La traducció **només afecta el número de pàgina**, mai el desplaçament. [15]

La mida de pàgina també implica un **compromís**. Pàgines grans redueixen el nombre d'entrades a gestionar i milloren la *cobertura* de la TLB, però malbaraten memòria per *fragmentació interna* i encareixen cada fallada. [6] És el mateix compromís que la mida de bloc de l'apartat 2.3.2.

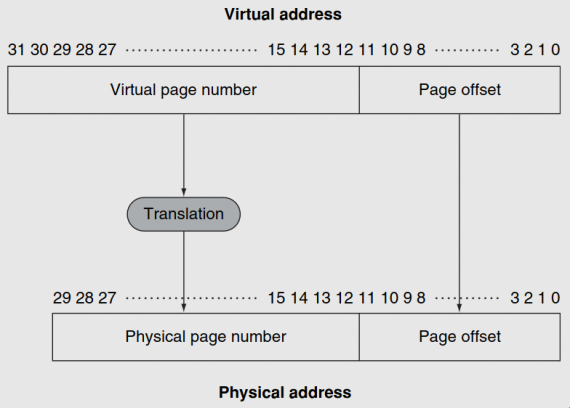


Figura 9: Traducció d'una adreça virtual a una física. Font: [15]

2.4.2 La Taula de Pàgines

La taula de pàgines guarda la traducció *pàgina-marc*, n'hi ha **una per procés** i viu a la memòria principal. [15]

Guardar-la amb una estructura plana és molt **ineficient**. Amb un espai virtual de 32 bits i pàgines de 4 KB, el desplaçament ocupa:

$$\log_2(4\text{ KB}) = 12\text{ bits} \tag{1}$$

de manera que el número de pàgina ocupa els 20 bits restants. Amb entrades de 4 bytes, la mida de la taula resultant és:

$$2^{20} \times 4 \text{ bytes} = 2^{22} \text{ bytes} = \textbf{4 MB per procés} \tag{2}$$

Respecte de l'emmagatzematge total la xifra no sembla preocupant:

$$\frac{4 \text{ MB}}{4 \text{ GB}} = 0.1\% \tag{3}$$

El problema és que aquesta mida **és per procés**: amb 100 processos en execució el cost es multiplica en la mateixa proporció,

$$100 \times 4 \text{ MB} = \textbf{400 MB} \tag{4}$$

reservats només a taules de pàgines. La solució passa per una estructura d'**arbre**.

La idea és *partir el número de pàgina en camps*, de manera que cada camp indexa una taula d'un nivell i l'entrada seleccionada apunta a l'entrada del nivell següent fins arribar al número de marc. Es canvia un *vector indexat* per una *estructura d'arbre*. [6] El punt central és que en arrencar **no es carrega la taula sencera** sinó només l'arrel. Cada cop que el procés toca una regió nova de l'espai virtual, el sistema operatiu crea les taules intermèdies que calen per arribar al marc. **L'arbre es construeix a mesura que es necessita** i el que no es toca mai no arriba a existir.

Repetim l'exemple anterior amb una taula de dos nivells, assumint en tots dos casos que les adreces són contigües. Es reparteixen 10 bits d'adreçament per nivell dels 20 disponibles, de manera que cada taula té (2^{10} =) 1024 entrades. El número **no és arbitrari**: amb 4 bytes per entrada, cada taula ocupa (1024 * 4 =) 4 KB, *exactament la mida d'una pàgina*. Un procés de 16 MB necessita (16 MB / 4 KB =) 4096 pàgines d'un total de (2^{20} =) 1.048.576 disponibles, és a dir, un **0.4%** de l'espai. Amb el disseny de dos nivells només calen la taula arrel i 4 taules més. El disseny té un **cost**, ja que cada nivell afegit és un *accés més a memòria* per traducció i les taules intermèdies ocupen espai, però la mida de l'estructura passa a dependre de **l'espai realment utilitzat pel programa** en comptes de l'espai adreçable.

2.4.3 La TLB

Amb l'estructura anterior, cada accés del programa costaria *tants accessos com nivells tingui l'arbre* més el de la dada. En un sistema amb DRAM, segons l'apartat

2.1.1, cada traducció **multiplicaria el cost de tot**. La solució torna a passar per la SRAM, en forma d'una memòria cau anomenada TLB (*Translation Lookaside Buffer*). [15] El funcionament és el mateix que el de l'apartat 2.3, però el que s'hi guarda són **traduccions d'adreces** en comptes de dades.

El motiu és el mateix *principi de localitat* de l'apartat 2.1.2: els accessos contigus cauen dins de poques pàgines, exactament com passa amb la memòria cau. L'associativitat de la TLB sol ser *gran o total* amb *poques entrades*, de manera que el temps de comparació és assumible i s'eliminen els conflictes, que aquí surten cars perquè cada fallada provoca un **page walk**.

Cal distingir entre una **fallada de TLB**, és a dir, que *no tenim la traducció a mà* i cal resoldre-la amb un *page walk*, i una **fallada de pàgina**, és a dir, que la pàgina *no és a la memòria principal* i s'ha de pujar del Disc; són conceptes diferents i no tenen per què donar-se alhora.

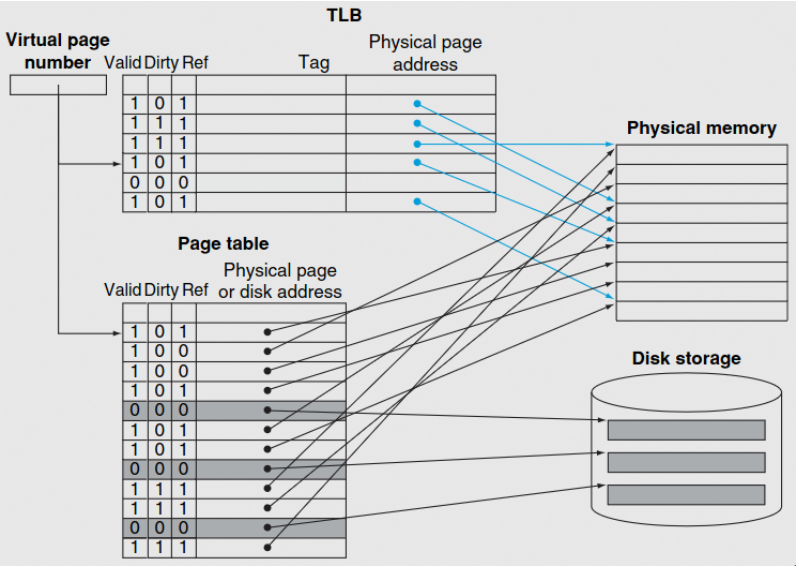


Figura 10: La TLB actua com una *cache* per a la taula de pàgines per les entrades que apunten cap a la memòria física. Font: [15]

2.4.4 Fallades de pàgina i gestió de marcs

Queda per resoldre què passa si la traducció existeix **però la pàgina no és present**. El SO localitza la pàgina a Disc, obté el marc corresponent, la carrega, actualitza la taula i reprèn la instrucció. [15] Com s'ha vist a l'apartat 2.1.1, la diferència de latència entre Disc i RAM és de **milions de cicles**, de manera que val la pena evitar-la sempre que sigui possible.

Per aquest motiu la memòria principal és **totalment associativa**, segons l'apartat 2.3.1. A les memòries cau el temps dels comparadors era limitant; aquí el cost de comparació és *negligible* davant del d'una fallada de pàgina.

Mentre hi ha marcs lliures, la decisió és **quin ocupar**; quan no en queden, és **quin expulsar**. Expulsar una pàgina *neta* és molt més barat que expulsar-ne una de *bruta*, i el **bit de modificació** fa aquí la mateixa funció que el bit de brut de les memòries cau. La RAM, com s'ha dit, passa a comportar-se com una memòria cau per al Disc. Queden fora de l'abast del projecte els bits de *permís* i de *protecció*, que controlen que un programa no pugui escriure en seccions que no li pertoquen i llancen una excepció quan ho intenta, perquè el simulador **només executa un procés**.

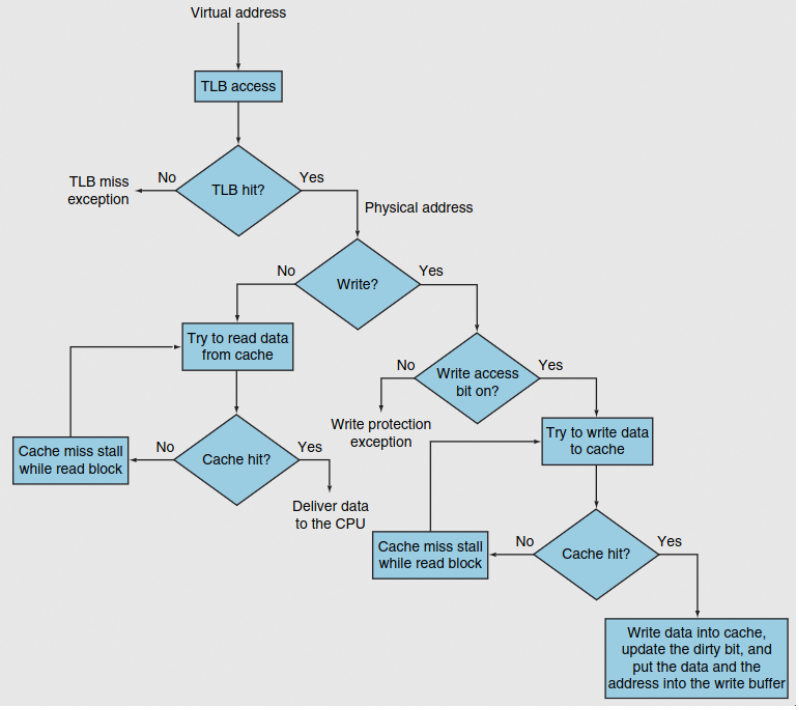


Figura 11: Flux de treball d'una traducció a partir de la TLB. Font: [15]

2.5 Mètriques de rendiment

Els mecanismes descrits introdueixen compromisos que **només tenen sentit si es poden quantificar**. Aquesta secció presenta les mètriques habituals per avaluar una jerarquia.

2.5.1 Taxes d'encert i fallada

La **taxa d'encert** (*Hit Ratio*) i la **taxa de fallades** (*Miss Ratio*) són les mètriques més bàsiques i indiquen la proporció d'encerts i de fallades sobre els accessos al nivell. [15] La de fallades és la que apareix als models de cost, on *multiplica la penalització*.

$$H_i = \frac{\text{encerts}_i}{\text{accessos}_i} \qquad m_i = \frac{\text{fallades}_i}{\text{accessos}_i} = 1 - H_i \tag{5}$$

La mitjana **pot enganyar**: una taxa de fallada del 5% que costi cent vegades més que un encert s'endú la meitat del temps. Per això cal tenir en compte sempre la **penalització**, que es veu a l'apartat 2.5.2.

Les taxes es poden calcular de manera **local** i **global**. La local correspon a la fórmula anterior, i la global es mesura sobre *tots els accessos que emet el processador*.

$$m_i^{\text{local}} = \frac{\text{fallades}_i}{\text{accessos}_i} \tag{6}$$

$$m_i^{\text{global}} = \frac{\text{fallades}_i}{\text{accessos}_1} = \prod_{j=1}^{i} m_j^{\text{local}} \tag{7}$$

Una altra mètrica habitual és **MPKI** (*Misses Per Kilo Instructions*) [6], fallades per cada miler d'instruccions. Com que *no depèn del nombre d'accessos*, permet comparar càrregues amb intensitats diferents.

$$\text{MPKI}_i = \frac{\text{fallades}_i}{\text{instruccions}} \times 1000 \tag{8}$$

2.5.2 Temps mitjà d'accés

L'objectiu aquí és un **sol número que combini freqüència i cost**, ja que una taxa de fallada per si sola ignora el cost. La mètrica és l'**AMAT** (*Average Memory Access Time*) [15], que indica el temps mitjà que triga un accés a memòria del processador.

Tot accés paga el temps de consultar el primer nivell, i només una fracció paga a més el cost d'anar a buscar la dada més avall. D'aquí surt la fórmula:

$$\text{AMAT} = t_{\text{encert}} + m \cdot P_{\text{fallada}} \tag{9}$$

Primer es comprova el nivell i, si la dada no hi és, es paga la penalització de la fallada (P_{fallada}). Convé aclarir que la penalització és el **cost addicional** de resoldre la fallada i no el cost total de l'accés fallat. Tant P_{fallada} com t_{encert}, i per tant l'AMAT, es poden expressar en *cicles*, com fa el simulador, o en *unitats de temps*.

Amb N nivells cal una **fórmula general**. Quan un accés falla al primer nivell, el que ve a continuació és un accés al segon nivell, i aquest té *exactament la mateixa estructura de cost* que el primer. **La penalització del nivell i és, doncs, el temps mitjà d'accés del nivell** $i+1$, cosa que dona una fórmula recursiva aplicable a qualsevol profunditat. [6]

$$\text{AMAT}_i = \begin{cases} t_i + m_i \cdot \text{AMAT}_{i+1} & \text{si } i < N \\ t_N & \text{si } i = N \end{cases} \tag{10}$$

Per a quatre nivells el resultat és:

$$\text{AMAT} = t_1 + m_1\big(t_2 + m_2(t_3 + m_3\, t_4)\big) \tag{11}$$

L'últim nivell no té penalització de fallada perquè la dada sempre hi és, de manera que només se'n compta el temps d'encert. La fórmula **només funciona amb taxes locals**: cada terme ja està condicionat per haver arribat al nivell, i fer servir taxes globals *dona resultats incorrectes*.

$$\text{AMAT} = t_1 + \sum_{i=2}^{N} \left(\prod_{j=1}^{i-1} m_j \right) t_i \tag{12}$$

Aquí es veu per què la **L1 s'optimitza per latència** i la resta de nivells **per capacitat**: el temps d'encert de L1 es paga a cada accés, mentre que el de l'últim nivell només afecta una fracció petita. Reduir una taxa val *proporcionalment a la penalització* que evita.

L'AMAT també es pot **mesurar** sobre una *execució instrumentada*, dividint el temps efectivament consumit en accessos a memòria pel nombre d'accessos realitzats. La fórmula anterior parteix de *paràmetres* i prediu un valor; aquesta mesura observa el *resultat*. Els dos valors coincideixen quan es compleixen les assumpcions del model, entre les quals que els accessos són **seqüencials i no es solapen**.

$$\widehat{\text{AMAT}} = \frac{\text{cicles consumits en accessos a memòria}}{\text{nombre d'accessos a memòria}} \tag{13}$$

En el cas del simulador, l'AMAT es calcula a cada accés a memòria, de manera que el valor sempre correspon al que ha passat realment durant l'execució.

Una altra mètrica bàsica és el **CPI** (*Cycles Per Instruction*) [15], la proporció de cicles consumits per instrucció. El que interessa en última instància és el **temps d'execució**, i la connexió passa pel CPI, on el cost dels accessos a memòria se suma com un *terme afegit al CPI base*. La descomposició assumeix que les fallades **aturen completament el processador**, cosa que no es compleix en processadors amb execució fora d'ordre. S'ha deixat així deliberadament, perquè resulta més interessant analitzar el comportament del sistema sense execució fora d'ordre.

$$\mathrm{CPI} = \mathrm{CPI}_{\mathrm{base}} + \frac{\text{accessos}}{\text{instrucció}} \cdot m \cdot P_{\mathrm{fallada}} \tag{14}$$

$$T_{\mathrm{exec}} = \mathrm{IC} \times \mathrm{CPI} \times T_{\mathrm{cicle}} \tag{15}$$

2.6 Simuladors i eines existents

2.6.1 Eines d'investigació

Gem5 és el simulador de *referència en recerca d'arquitectura*. Ofereix dos models de memòria: el *clàssic*, ràpid i senzill de configurar però amb protocol de coherència fix, i *Ruby*, que modela jerarquies inclusives i exclusives amb diverses polítiques de reemplaçament, protocols de coherència i xarxes d'interconnexió, i on **gairebé qualsevol aspecte** de la funcionalitat i la temporització es pot controlar. Es configura amb scripts de Python i pot arrencar un sistema operatiu complet. [8]

El seu propi equip documenta el problema de configurabilitat: modificar una configuració existent per *afegir un nivell de cau compartit* obliga a canviar de protocol o a implementar-ne un, cosa gens trivial perquè els protocols de Ruby són complexos i difícils de depurar. Això crea un **buit de flexibilitat** entre el model clàssic i Ruby. A més hi ha la corba d'entrada i el fet que **no hi ha visualització de l'estat intern**, ja que la sortida són fitxers d'estadístiques.

DineroIV és un simulador de caus dirigit per traces, d'Edler i Hill. Modela una jerarquia com un *arbre de caus* amb paràmetres independents per nivell, i suporta *write-through* i *write-back*, *write-allocate* i *no-write-allocate*, mides de bloc i subbloc, i diverses polítiques de reemplaçament. Té un límit de **cinc nivells** fixat al codi, modificable recompilant, i una interfície de línia d'ordres pura. [3]

La seva última versió és de **febrer de 1998**, gairebé tres dècades sense manteniment. És uniprocessador, *no modela temps ni latències*, només compta encerts i fallades, i **no té memòria virtual**.

Cachegrind és un cas especial perquè funciona com un *profiler* en comptes d'un simulador de disseny. Instrumenta un programa real i anota el codi font *línia a línia* amb les fallades atribuïdes a cada instrucció. Simula una màquina amb caus de primer nivell independents d'instruccions i dades, sostingudes per una cau unificada de segon nivell, i en màquines amb tres o quatre nivells simula **només el primer i l'últim**. La configuració es detecta de la màquina amfitriona. [27]

No té en compte l'activitat del nucli ni la d'altres processos, **ignora la correspondència entre adreces virtuals i físiques** i simula amb adreces virtuals encara que la majoria de caus reals s'indexin físicament, i no captura les fallades no visibles a nivell d'instrucció, com les derivades de fallades de TLB o d'execució especulativa. El model és de **dos nivells fix**, sense visualització i sense memòria virtual.

ChampSim és un simulador basat en traces mantingut a Texas A&M amb suport de la comunitat. Va néixer de les *competicions de prefetch i de polítiques de reemplaçament*, i aquest origen n'explica el disseny: està pensat perquè un investigador hi **encaixi el seu propi mòdul** de *prefetch* o de reemplaçament i el compari contra una línia base. Es configura amb un fitxer JSON i reporta *IPC* com a mètrica principal. [4]

L'entrada són traces comprimides de gran mida, generades amb un *instrumentador extern*, i cal especificar instruccions d'escalfament i de simulació. **No hi ha programa que l'usuari escrigui**, ni execució pas a pas, ni cap visualització. És una eina de comparació experimental per a especialistes i no pretén ser cap altra cosa.

2.6.2 Eines docents

RIPES és un simulador visual d'arquitectura per a RISC-V, de Morten Borup Petersen, amb article a WCAE 2021. Integra editor d'assemblador, assemblador propi, suport de compilació de C i simulador de cau al voltant d'una *visualització de la microarquitectura*. Ofereix diversos models de processador, de cicle únic i variants de cinc etapes, i permet **retrocedir la simulació** cicle a cicle. [17]

Des de la versió 2.1.0 simula caus L1D i L1I i permet configurar la disposició i el comportament de cada tipus, però **només un nivell**. La pròpia guia en documenta dues limitacions més: els models de processador *no accedeixen* al simulador de cau, sinó que aquest s'hi enganxa i analitza els accessos a memòria de cada cicle com si fossin una traça, i **RIPES no simula la latència d'accés a la cau**, de manera que no dona cap estimació del temps d'execució real i només informa de taxes d'encert, fallada i postescriptura.

Sense latències configurables **no hi ha AMAT possible**, i com que el simulador de cau *observa* el processador en comptes de *servir-lo*, la cau no forma part del

camí d'execució. Tampoc **no hi ha memòria virtual**. És una eina excel·lent per veure un pipeline i per introduir el concepte de cau, i no pretén cobrir la jerarquia completa.

MARS és un entorn de desenvolupament per a assemblador MIPS de Sanderson i Vollmar, escrit en Java, amb editor integrat, execució pas a pas endavant i endarrere, i un sistema d'*eines auxiliars* que s'enganxen a l'execució. Entre aquestes hi ha una eina de simulació de cau de dades, que s'hi connecta com a *observador* i actualitza en viu les estadístiques a mesura que el programa s'executa. [20]

Simula **únicament la cau de dades**, de manera que les cerques d'instruccions no hi apareixen. Té **un sol nivell**, sense latències i sense memòria virtual. La cau hi és una eina *accessòria* d'un simulador d'ISA.

Amnesia és un simulador de jerarquia de memòria desenvolupat des del 2007 a l'ICMC de la Universitat de São Paulo sota la direcció de Souza i Bruschi, concebut com a *objecte d'aprenentatge* i recurs educatiu obert. Simula registres basats en MIPS, caus, memòria principal i **memòria virtual paginada** dins d'una única vista jeràrquica, i permet comparar configuracions diferents. Hi ha estudis publicats que en mesuren l'efecte sobre l'aprenentatge. [26]

L'entrada és un fitxer *XML* que descriu l'arquitectura més un fitxer de text amb la *traça*, de manera que **l'usuari no escriu ni executa un programa** sinó que descriu una màquina i li dona una llista d'accessos. Els registres corresponen a un model MIPS fix.

2.6.3 Síntesi

Eina	**Entrada**	**Nivells de cau**	**Harvard**	**Mem. virtual**
gem5	Binari / SO	Configurable	L1	Completa
DineroIV	Traça	≤ 5	Sí	Cap
Cachegrind	Binari	2 (fix)	L1	Cap
ChampSim	Traça	Configurable	L1	TLB
RIPES	Codi font	1	L1	Cap
MARS	Codi font	1	Només dades	Cap
Amnesia	XML + traça	?	?	Completa
missme	Codi font	**Il·limitat**	**Per nivell**	Completa

Taula 2: Comparativa de simuladors: entrada i modelatge de la jerarquia. Font: elaboració pròpia basada en [8][3][27][4][17][20][26].

Eina	**Pas a pas**	**Inspecció**	**Retroc.**	**Export.**	**Distribució**
gem5	Cicle	Cap	×	✓	Compilació
DineroIV	No	Cap	×	✓	Compilació
Cachegrind	No	Cap	×	✓	Paquet
ChampSim	No	Cap	×	✓	Compilació
RIPES	Cicle	Cau	✓	×	Binaris
MARS	Instrucció	Cau	✓	×	JAR
Amnesia	Accés	Jerarquia	?	?	JAR
missme	**Acció**	**Completa**	×	×	Compilació

Taula 3: Comparativa de simuladors: execució, inspecció i distribució. Font: elaboració pròpia basada en [8][3][27][4][17][20][26].

Les eines de traces i binaris ofereixen *jerarquies profundes i models detallats*, però parteixen d'un **fitxer d'accessos ja produït**, no visualitzen l'estat intern i estan pensades per obtenir xifres agregades. Les docents parteixen d'un *programa que l'usuari escriu* i el visualitzen bé, però hi tracten la memòria com un **component accessori** amb jerarquia d'un sol nivell. Amnesia cobreix jerarquia i traducció alhora, tot i que l'entrada torna a ser una traça sobre una arquitectura descrita en XML. **Cap combina un programa executable escrit per l'usuari amb una jerarquia de profunditat lliure i el camí de traducció complet, tot inspeccionable a cada pas.**

Missme aporta **cinc punts diferents** respecte a la resta d'eines:

- El nombre de nivells de cau **no té límit**, ni al motor ni a la interfície. DineroIV en té cinc fixats al codi, Cachegrind dos, RIPES i MARS un.
- Cada nivell decideix independentment si és **Harvard o Von Neumann**, cosa que cap de les set fa. RIPES separa I i D però només al seu únic nivell.
- El camí de **memòria virtual és complet i configurable**, amb TLB amb conjunts reals, taula de pàgines multinivell construïda com a arbre amb assignació mandrosa de taules intermèdies, taula de marcs i disc d'intercanvi, i conviu amb la jerarquia de caus en la mateixa simulació.
- L'entrada és **assemblador RISC-V que l'usuari escriu i executa**, no una traça.
- L'execució avança per **acció atòmica**, amb cada estructura inspeccionable a cada pas.

RIPES executa un programa *però no té memòria virtual*, i Amnesia té memòria virtual *però no executa un programa.* La **intersecció d'aquestes dues capacitats**, sobre una jerarquia de profunditat lliure, és el que queda buit.

3 Abast i requisits

Tot simulador és una simplificació, i el valor d'una eina docent depèn tant del que modela com del que decideix deixar fora. Aquesta secció fixa aquestes decisions abans d'entrar en com s'han implementat.

3.1 Públic objectiu i escenaris d'ús

3.1.1 Objectiu pedagògic

El problema al qual intenta donar solució l'eina és la representació de la jerarquia durant el procés d'aprenentatge. La jerarquia de memòria s'explica amb *diagrames estàtics* i exercicis de paper, però el comportament que importa és **dinàmic i acumulatiu**. Una taxa de fallada no s'entén mirant una taula; s'entén veient com un patró d'accés concret omple i buida uns conjunts. Aquest **desajust entre com s'explica i com funciona** és el que motiva desenvolupar l'eina.

L'usuari després de l'ús de l'eina hauria de ser capaç de *relacionar* un patró d'accés amb el nombre de fallades que provoca, *predir* l'efecte de canviar un paràmetre i comprovar si encertava, *distingir* per què falla una cosa mirant l'estat actual de la jerarquia i acabar podent *seguir* una traducció d'adreça sencera sense perdre'l.

La idea del disseny és que l'estudiant tingui el poder de **manipular i visualitzar allò que està intentant entendre**. Per això, com veurem durant la secció, s'ha mantingut fix o s'ha omès tot allò que afegiria complexitat sense afegir comprensió.

3.1.2 El públic objectiu

L'objectiu principal de l'eina és un **estudiant de grau** que cursa o ha cursat una assignatura d'arquitectura de computadors. Es prenen certes assumpcions de coneixement de l'estudiant. Aquest sap què són les memòries cau i una adreça, sap programar en assemblador RISC-V i entén el concepte de latència. L'eina li ajudarà a entendre conceptes com la *relació entre paràmetres i comportament*, i la *intuïció sobre quan un mecanisme ajuda i quan no.*

Un usuari amb aquest perfil no vol configurar un simulador de recerca ni escriure fitxers de configuració, vol **provar coses**, jugar amb les diferents combinacions amb diferents programes. Això justifica que la configuració sigui gràfica, que no calgui preparar traces, i que l'eina hagi de funcionar sense passos previs de compilació de càrregues.

El públic secundari serien per una banda els **docents** que han de preparar material o demostrar un mecanisme a classe, que necessiten explicar pas a pas mentre expliquen.

Per altra banda tindríem l'**usuari curiós**, que ja domina la matèria i vol provar una configuració per veure si podria arribar a ser vàlida.

El perfil al qual **no** va dirigit en cap cas és a l'investigador que necessita xifres comparables amb *hardware* real, ni el desenvolupador que vol fer *profiling* d'un programa en concret. Per això ja existeixen les altres eines que són millors en aquest aspecte (apartat 2.6).

3.1.3 Escenaris d'ús

S'han plantejat **tres escenaris principals** d'ús de l'eina.

El primer de tots, on l'usuari escriu un programa i mira què provoca a la jerarquia. És el *cas d'entrada*, el que fa servir algú que encara no sap què buscar.

El segon seria quan l'usuari fixa el programa i varia la configuració, o al revés, per veure què canvia. És el cas que ensenya els **compromisos de disseny**, i probablement el més valuós dels tres.

I per últim, quan l'usuari vol entendre un *mecanisme concret* (una expulsió, un *page walk*, una fallada de pàgina...) i el segueix pas a pas.

L'usuari en tot moment ha de poder **veure l'estat intern a cada moment**, això és el que separa l'eina respecte de la resta (apartat 2.6).

3.2 Requisits funcionals

Definits els públics objectius, en aquesta secció declararem què ha de poder fer l'eina. S'agruparan en quatre blocs: *definir* una jerarquia, *executar-hi* un programa, *inspeccionar-ne* l'estat i *mesurar-ne* el comportament.

3.2.1 Definició de la jerarquia

El primer punt i un dels més importants és donar a l'usuari la llibertat de poder escollir el **nombre de nivells de cau que vulgui**. Incloent el cas de cap nivell. Les jerarquies varien molt en termes de nivells així que acotar el nombre de nivells no té molt de sentit quan volem fer una eina que ensenya tot el que passa dintre.

L'eina permet quatre tipus d'elements: TLB, cau, memòria principal i Disc. La **seqüència és fixa**, però el que permet variar l'eina és el nombre de caus i els seus paràmetres. Aquesta restricció és *deliberada* i correspon a l'organització habitual (apartat 2.2.2).

Cada nivell de cau ha de poder ser Von Neumann o Harvard de manera **independent a la resta** (apartat 2.1.3). La idea és que els usuaris puguin replicar tant patrons

reals com allunyar-se'n.

L'eina també ha de permetre **activar o desactivar la memòria virtual**. Principalment per la pròpia comparació entre els dos casos.

I finalment, la idea és no utilitzar fitxers externs per a la configuració. Tota la jerarquia s'hauria de poder definir des de la interfície. Al final l'objectiu és *poder fer proves*, no un experiment.

3.2.2 Execució de programes

Com a codi inicial per a l'eina s'ha escollit **RISC-V**. Concretament la ISA no privilegiada de RV32I i RV64I com a base. L'usuari podrà afegir extensions d'aquesta, en particular els mòduls de multiplicació (M), coma flotant (F), doble coma flotant (D) i quàdruple coma flotant (Q). Aquest no és un requisit obligatori ja que podríem haver agafat qualsevol altre llenguatge, la idea és fer un simulador de jerarquies de memòria *generalitzat* a qualsevol arquitectura o llenguatge. RISC-V proporcionava la ISA combinant **simplicitat i popularitat**.

L'eina ha de permetre a l'usuari **escriure codi i executar-lo en la mateixa eina**. Pensant novament en la simplicitat per als alumnes. També la ISA ha de poder proporcionar el suport suficient per poder generar patrons d'accés interessants. Conjunt de base d'enters de variants de 32 i 64 bits i les extensions aritmètiques i de coma flotant. S'ha de poder escriure bucles, recórrer estructures indexades, fer aritmètica d'adreces i moure dades entre registres i memòria. Sense això no es poden construir els patrons d'accés que fan interessant l'exercici.

En executar el codi, el programa ens permetrà fer **tres tipus d'accions**. Primer de tot podem avançar el programa a nivell d'**acció atòmica** (comparació en un nivell de cau per exemple) per al seguiment d'un mecanisme, és la part més interessant ja que en cap de la resta d'eines és possible. Avenç a nivell d'**instrucció** per a l'observació. I finalment, un **avenç continu** amb la velocitat regulable per arribar a un estat concret o per obtenir mètriques d'un programa sencer.

Finalment, el càlcul de les mètriques de temps es fa en **cicles**, ja que és la unitat que no pot variar en el temps. Al cap i a la fi, els usuaris podran convertir-la a temps fent una conversió molt fàcilment. L'execució ha d'acumular el cost temporal de cada accés segons les latències configurades, perquè sense això *no hi ha ni AMAT ni CPI*.

3.2.3 Inspecció de l'estat

L'usuari ha de poder veure el **contingut real de cada estructura** en qualsevol moment de l'execució, no un resum ni un agregat final. Ha de poder veure què

està passant en tota la seva jerarquia sencera. Cal poder inspeccionar els conjunts i les vies de cada nivell de cau amb els seus bits d'estat, l'etiqueta i el contingut; les entrades de la memòria de traduccions amb la correspondència que guarden; la taula de pàgines; els marcs físics; el contingut de la memòria principal i de l'espai d'intercanvi; i els registres.

Com que la taula de pàgines és una estructura d'arbre construïda de manera incremental, ha de ser **navegable per nivells** i no llistada plana. Igualment, l'usuari pot només posar un nivell en la taula de pàgines i fer la memòria principal amb una estructura plana si ell vol. En anar-se generant mentre avança el programa i també per la seva naturalesa d'arbre, la idea és que vagis saltant de node a node. Per exemple, pots saltar a nodes fill o al teu node pare.

Un altre punt important és que les vistes s'han d'**actualitzar després de cada pas**, sigui quina sigui la granularitat escollida, i sense que l'usuari hagi de demanar-ho. Poder aturar l'execució entre dues accions atòmiques no serveix de res si l'estat mostrat correspon a un moment anterior, o si cal fer una acció addicional per refrescar-lo. La combinació de tots dos requisits és el que permet l'escenari de seguiment, i és també el que imposa la restricció de rendiment.

Finalment, s'ha de mostrar quina és la idea en tot moment. Ja sigui un *fetch* de la instrucció, una lectura o escriptura. L'usuari ha de poder identificar **quin element concret s'està accedint** en cada moment i si l'accés ha resultat en encert o en fallada; caldrà indicar-ho tant per la memòria de dades com per la d'instruccions. Aquesta és la diferència entre una *vista de dades* i una *explicació*. Sense aquesta indicació, l'usuari veu una taula de valors i ha de deduir per si mateix quina fila importa, cosa que en una jerarquia amb diversos nivells i centenars d'entrades és impracticable. Amb ella, la seqüència de passos es llegeix com un recorregut.

3.2.4 Mètriques

S'hauran de comptar encerts i fallades per nivell i, quan el nivell és Harvard, per costat, i derivar-ne les taxes **locals i globals**. L'eina també haurà de calcular AMAT i CPI a partir dels cicles realment acumulats durant l'execució. Com hem mencionat abans en l'apartat 2.5.2, l'AMAT s'obté **mesurant**, no aplicant la fórmula analítica i, per tant, el valor que dona és *contrastable* amb el que l'usuari calcularia a mà amb la formulació recursiva a partir de les taxes i les latències. Cap de les eines docents de l'apartat 2.6 permet aquest exercici, perquè cap no modela latències.

Cal oferir **MPKI** com a mètrica independent del nombre d'accessos a memòria. Quan l'usuari compara dos programes diferents, o el mateix programa amb modificacions, la taxa de fallada no és directament comparable si la proporció d'instruccions d'accés a memòria ha canviat. Un programa amb pocs accessos pot tenir una

taxa de fallada alta i un impacte real petit, i sense una mètrica normalitzada per instruccions aquesta distinció queda amagada.

Les mètriques han d'estar **disponibles durant l'execució** i actualitzar-se amb ella, no calcular-se només en acabar. Veure com evoluciona una taxa mentre el programa avança aporta informació que la xifra final amaga. Per exemple, la diferència entre la *fase inicial d'ompliment* i el *règim estacionari*, que és la manifestació directa de les fallades obligatòries. A més, han d'estar organitzades de manera que es puguin consultar tant per nivell com de forma agregada, perquè les dues lectures responen a preguntes diferents.

3.3 Paràmetres configurables i valors fixats

Aquesta secció fixa el grau de llibertat concret de cada paràmetre: quins pot definir l'usuari, quins es deriven d'altres, i quins queden fixats amb un valor únic.

3.3.1 Criteri de decisió

El criteri és que serà **configurable** tot allò que materialitza un compromís de disseny que l'usuari ha d'entendre, es **deriva** allò que és conseqüència d'altres paràmetres i es **fixa** allò que afegiria complexitat sense afegir comprensió. El criteri prové directament de l'objectiu pedagògic (apartat 3.1.1). Si un paràmetre no canvia cap de les capacitats que allà s'estableixen com a objectiu, exposar-lo només afegeix soroll a la interfície i decisions que l'usuari no sabrà prendre.

També, en parlar d'un paràmetre derivat no significa que sigui menys configurable, l'usuari el controla, però *indirectament*, a través dels que el determinen. El cas més clar és el nombre de conjunts d'una cau, que queda determinat per la mida total, la mida de bloc i l'associativitat. Modificar-lo com a quart paràmetre independent permetria configuracions inconsistents i, sobretot, **amagaria la relació entre els tres**. Fer-lo derivat obliga l'usuari a entendre la relació, que és precisament el que interessa.

Els paràmetres fixats són de **dos tipus**. Els que venen determinats per una decisió anterior com per exemple l'amplada de l'adreça que ve determinada per la ISA escollida. I els que, sent decisions reals en un sistema autèntic, no aporten res. Tots els casos del segon tipus es justifiquen individualment a l'apartat 3.3.4.

3.3.2 Paràmetres de la jerarquia de cau

Primer de tot, cada nivell de cau permet establir la **mida total**, la **mida de bloc** i l'**associativitat**, en resum, la geometria de la cau. D'aquests tres es deriva el nombre de conjunts. Els tres paràmetres cobreixen tres compromisos diferents que

l'apartat 2.3 desenvolupa per separat. Cadascun es pot variar deixant els altres fixos. La mida total s'expressa com a *potència de dos*, cosa que garanteix que la descomposició d'adreça (apartat 2.3.2) tingui un nombre enter de bits d'índex.

Després decidim independentment en cada nivell si separa instruccions i dades, i quan ho fa, cada costat té la seva pròpia geometria i les seves pròpies polítiques. Aquesta independència és el que permet reproduir **configuracions asimètriques**, on la cau d'instruccions i la de dades tenen mides o associativitats diferents, que és el cas habitual en processadors reals i que cap eina docent revisada permet explorar.

Cada nivell configura la seva política de reemplaçament entre les quatre que descriu l'apartat 2.3.3, la política d'escriptura entre postescriptura i escriptura immediata, i l'assignació en fallada d'escriptura. Que siguin **per nivell i no globals** és una decisió deliberada, permet reproduir configuracions reals on els nivells superiors i inferiors adopten polítiques diferents, i sobretot permet *aïllar-ne l'efecte* canviant-ne una sola i deixant la resta igual. Els dos eixos de la política d'escriptura són independents, tal com estableix l'apartat 2.3.4, i es configuren per separat perquè és així com s'entenen.

Seguint, podem configurar el **temps d'encert en cicles** per a cada nivell. És el paràmetre que fa possible calcular l'AMAT, i el motiu pel qual s'edita en comptes de derivar-lo de la geometria és intencional, derivar-lo exigiria un model de cost circuital que quedaria com una *caixa negra*, mentre que modificar-lo deixa l'usuari observar directament l'efecte de la relació entre latència i taxa de fallada que la fórmula de l'AMAT descriu. Convé notar que això vol dir que l'usuari pot configurar *combinacions irrealistes*, encara que també és la idea de l'eina.

I finalment, l'eina permet establir quina **proporció de línies apareixen inicialment marcades com a modificades**. No és un paràmetre de manual, així que necessita justificació explícita, serveix per estudiar el cost de les expulsions sota postescriptura.

3.3.3 Paràmetres de memòria virtual

Primer de tot, la **mida de pàgina** és el paràmetre global de la traducció i el que més relacions té amb la resta, cosa que el fa especialment valuós pedagògicament. Determina la fragmentació interna, la cobertura de la memòria de traduccions, quants bits queden per al número de pàgina i, indirectament, quants nivells de taula tenen sentit. L'apartat 2.4.1 el presenta com el mateix tipus de compromís que la mida de bloc, i exposar-lo permet comprovar-ho directament, augmentar-la redueix les fallades de traducció i alhora encareix cada fallada de pàgina.

En un altre element, la taula de pàgines, l'usuari configura el **nombre de nivells**,

però no pot excedir els bits de número de pàgina realment disponibles, que depenen de l'amplada d'adreça i de la mida de pàgina. Com es calculen aquests ve a ser una simplificació, repartint tots els bits en parts iguals, és a dir, calculant b_{total}/N sent b_{total} el nombre de bits totals i N el nombre de nivells decidit per l'usuari. Si sobren bits, aquests es repartiran d'un a un en els nivells més baixos de l'arbre. És a dir, si tenim 22 bits de número de pàgina i volem 4 nivells de taula de pàgines acabarem amb una configuració de $5 + 5 + 6 + 6$, en els últims dos nivells s'han repartit els 2 bits restants.

Sobre la TLB permet configurar el nombre d'entrades, associativitat, política de reemplaçament i latència. Que siguin **els mateixos paràmetres que una cau** és deliberat i reforça l'argument de l'apartat 2.4.3 on menciona que el mecanisme és el mateix però a un contingut diferent.

En la memòria física l'usuari configura el **nombre de marcs** disponibles, que és el paràmetre que controla la pressió de memòria i, per tant, la freqüència de les fallades de pàgina. Configurar-lo com a nombre de marcs i no com a mida en bytes és coherent, el marc és la *unitat de gestió*, i pensar en marcs és el que fa entendre el mecanisme. També permet modificar la política d'assignació de marc lliure, però recordem que **assignació i expulsió són decisions diferents**, i només la primera és configurable, per raons que es discutiran en l'apartat 3.3.4.

En la memòria principal i l'emmagatzematge secundari es configurarà la **latència d'accés**. La de l'emmagatzematge és especialment important ja que permet observar com una sola fallada de pàgina domina el temps total d'execució, i transmet la idea millor que qualsevol xifra en un manual.

3.3.4 Valors fixats i la seva justificació

Els primers valors que podem trobar com a usuaris són els **determinats per la ISA**. L'amplada d'adreça i la mida de les entrades de la taula de pàgines no són decisions lliures sinó conseqüències de la variant de RISC-V escollida. En un sistema real tampoc no són configurables, i fer-los variables modelaria una arquitectura que no existeix. L'usuari els controla *indirectament* triant entre les variants de 32 i 64 bits, que és exactament el grau de llibertat que té un dissenyador real.

Com hem anat mencionant, la **política que decideix quin marc s'allibera** quan tots estan ocupats és fixa. En memòria virtual, la política de reemplaçament pràcticament sempre s'aproxima a LRU, és a dir, decideix segons quant fa que s'ha fet servir cada marc, perquè el cost de la fallada justifica invertir-hi i les alternatives no hi competeixen. Oferir-ne d'altres modelaria variants que no es fan servir. Per tant no aporta valor al simulador.

El **cost d'execució d'una instrucció**, al marge dels accessos a memòria, és constant. Aquesta decisió deriva del model d'execució que l'apartat 3.4 desenvoluparà. Sense *pipeline* ni riscos, no hi ha res que faci variar aquest cost, i introduir-hi variabilitat exigiria un model de processador que queda fora d'abast. El CPI que reporta l'eina **aïlla l'efecte de la jerarquia**, cosa que és una limitació si es busca realisme però un avantatge si es busca atribuir un efecte a una causa concreta, que és el propòsit mencionat en l'apartat 3.1.1.

Un altre aspecte que s'ha decidit fixar són els **bits d'estat de la taula de pàgines**. Les entrades mantenen només la informació necessària per a la traducció i la gestió de presència. Els bits de permisos i protecció queden fora, i la justificació és que, com ja s'ha mencionat prèviament, el model executa un *únic procés*. Sense múltiples espais d'adreces competint, la protecció no té res de què protegir, i els mecanismes que la implementen quedarien com a estructures inertes.

3.3.5 Taula resum

Paràmetre	**Àmbit**	**Llibertat**	**Motiu**
Nombre de nivells	Global	Configurable	Les jerarquies reals varien en profunditat
Variant de la ISA	Global	Configurable	Determina l'amplada d'adreça
Memòria virtual activa	Global	Configurable	Permet comparar els dos règims
Mida total	Per cau	Configurable	Compromís capacitat–cost
Mida de bloc	Per cau	Configurable	Compromís localitat–contaminació
Associativitat	Per cau	Configurable	Compromís conflictes–comparació
Nombre de conjunts	Per cau	*Derivat*	Determinat pels tres anteriors
Bits d'índex i etiqueta	Per cau	*Derivat*	Determinats per la geometria
Organització Harvard	Per nivell	Configurable	Mou la frontera de la unificació
Política de reempl.	Per cau	Configurable	Objecte d'estudi directe
Política d'escriptura	Per cau	Configurable	Eix independent de l'assignació
Assignació en fallada	Per cau	Configurable	Eix independent de l'escriptura
Latència d'encert	Per cau	Configurable	Necessària per a l'AMAT
Línies brutes inicials	Per cau	Configurable	Aïlla el cost de les expulsions
Cicle base d'instrucció	Global	**Fix**	Sense *pipeline* no hi ha variabilitat
Granularitat de dades	Global	**Fix**	Decisió de presentació

Taula 4: Paràmetres de la jerarquia de memòria cau. Font: elaboració pròpia.

Paràmetre	**Àmbit**	**Llibertat**	**Motiu**
Mida de pàgina	Global	Configurable	Compromís fragmentació–cobertura
Nivells de taula	Global	Configurable	Compromís espai–accessos
Límit de nivells	Global	*Derivat*	Limitat pels bits de pàgina disponibles
Bits d'índex per nivell	Global	*Derivat*	Determinats per pàgina i mida d'entrada
Entrades de la TLB	Global	Configurable	Determina la cobertura
Associativitat de TLB	Global	Configurable	Mateix compromís que a la cau
Política de reempl. TLB	Global	Configurable	Mateixes opcions que la cau
Latència de TLB	Global	Configurable	Cost de la traducció
Nombre de marcs	Global	Configurable	Controla la pressió de memòria
Assignació de marc lliure	Global	Configurable	Decisió sobre quin marc ocupar
Expulsió de marc	Global	**Fix**	Es tria segons quant fa que s'ha fet servir
Latència de RAM	Global	Configurable	Primer salt de tecnologia
Latència d'emmagatz.	Global	Configurable	Fa palpable la diferència d'ordres
Mida d'entrada de taula	Global	**Fix**	Determinada per la ISA
Amplada d'adreça	Global	**Fix**	Determinada per la ISA
Bits de permisos		**Fix**	Un únic procés: res a protegir

Taula 5: Paràmetres de memòria virtual. Font: elaboració pròpia.

3.4 Simplificacions assumides

Tot simulador modela un *subconjunt* de la realitat, i la tria d'aquest subconjunt és una decisió de disseny tan important com les anteriors. En una eina d'exploració, el criteri no és la **fidelitat** sinó l'**atribució**, cal que l'usuari pugui relacionar un efecte observat amb la causa que l'ha provocat, i cada mecanisme addicional que s'incorpora és una variable més que interfereix en aquesta relació. Les simplificacions que segueixen s'han escollit sota aquest criteri, i cadascuna s'acompanya del que implica.

3.4.1 Model d'execució

El model és **funcional amb comptabilitat de cicles**, cada instrucció s'executa completament abans de començar la següent, i el temps s'acumula sumant el cost fix d'execució i el dels accessos a memòria. No hi ha, per tant, encavalcament entre instruccions ni cap de les estructures que el fan possible.

En el cas de la **segmentació**, introdueix una font de variabilitat en el temps d'execució que és independent de la jerarquia de memòria, i que dificultaria atribuir un canvi de rendiment a un canvi de configuració de la jerarquia. Un usuari que modifiqui l'associativitat d'un nivell i observi una variació en el CPI ha de poder concloure que la variació ve d'aquí, i això només és cert si res més no hi contribueix. Per això, no hi ha etapes solapades ni, en conseqüència, riscos de dades ni de control ni bombolles.

Una altra simplificació ha sigut l'absència d'**execució fora d'ordre i especulativa**. Un processador real amaga part de la latència de memòria executant altres instruccions mentre espera, cosa que fa que una fallada no aturi necessàriament el processador. El problema principal és que les mètriques clàssiques assumeixen *accessos serialitzats* i que aquesta assumpció no es compleix en *hardware* modern. El model d'aquesta eina compleix l'assumpció per construcció, que acaba derivant en que les fórmules de l'apartat 2.5 s'hi apliquen **exactament**, i l'usuari pot comprovar-les sense que hi intervinguin efectes que la fórmula no captura.

També s'ha decidit no implementar cap tipus de ***prefetch***. Cap nivell anticipa accessos futurs. Aquest mecanisme redueix el temps d'execució sense reduir necessàriament el nombre de fallades (de fet les pot augmentar) i introduir-la trencaria la relació directa entre patró d'accés i comportament observat. En una eina on l'usuari escriu el programa precisament per veure què provoca, un mecanisme que actua per compte propi entre el programa i la jerarquia oscureix exactament el que es vol mostrar.

Una altra simplificació és que el model executa un sol programa amb una **funcionalitat mínima de gestió de memòria** implementada dins del motor, però sense un sistema operatiu com a entitat executable. Sense múltiples processos no hi ha protecció entre espais d'adreces, sense múltiples nuclis no hi ha coherència, i sense sistema operatiu no hi ha activitat de nucli que contamini les mesures. És la simplificació **més estructural** de totes i la que més coses arrossega.

3.4.2 Aspectes fora d'abast

El primer aspecte, i el més evident, és que només es modela un **únic nucli**, de manera que no hi ha protocols de coherència ni caus compartides entre nuclis. La coherència

és un tema propi, amb la seva pròpia teoria i els seus propis mecanismes, i incorporar-la desplaçaria el focus del comportament d'una jerarquia al de la *interacció entre jerarquies*. En l'apartat 2.2.2 ja s'acota la L3 compartida com a fora d'abast pel mateix motiu. Això sí, és una de les extensions més naturals si el treball es continua.

No existeix cap estructura que agrupi o difereixi les escriptures cap al nivell inferior, és a dir, el **buffer d'escriptura**. Cada escriptura s'aplica de manera síncrona. El buffer és el que fa viable l'escriptura immediata a la pràctica, i per tant el model **sobreestima el cost** d'aquesta política respecte del que s'observaria en un sistema real. La comparació relativa entre les dues polítiques d'escriptura segueix sent vàlida i il·lustrativa, però la *magnitud* de la diferència no ho és.

La memòria principal es modela com un espai adreçable amb una **latència uniforme**, sense l'estructura interna de bancs, files i columnes que determina que en un sistema real el cost d'un accés depengui del que s'hagi accedit abans. Aquest detall pertany al disseny del controlador de memòria, no al de la jerarquia, i modelar-lo introduiria una variabilitat en les latències que faria molt més difícil relacionar una mesura amb una decisió de configuració.

Les entrades de la taula de pàgines no mantenen **bits de permisos**, i no hi ha, per tant, fallades de protecció. La justificació és clara, el model només permet un únic procés on no hi ha de qui protegir-se.

També queda fora d'abast la **ubicació física de les taules de pàgines**. Les taules intermèdies no ocupen marcs de memòria física reals ni competeixen, per tant, amb les dades del programa per l'espai disponible. En un sistema real ho fan. L'exclusió simplifica la lectura del que passa als marcs, l'usuari veu només les pàgines del seu programa, cosa que fa el mecanisme de l'apartat 2.4.4 molt més llegible a costa de no reflectir aquest cost addicional.

3.4.3 Mètriques no incloses

L'eina **no classifica les fallades** en obligatòries, de capacitat i de conflicte. La raó no és de dificultat conceptual sinó de model d'execució. La classificació s'obté per comparació amb *configuracions de referència*. Una cau de capacitat infinita per aïllar les obligatòries, i una de totalment associativa de la mateixa mida per separar capacitat de conflicte. Obtenir-la exigiria simular en paral·lel dues jerarquies addicionals per cada nivell, cosa que multiplicaria el cost de cada pas i entraria en tensió amb el requisit d'interactivitat.

Les 3C responen a la pregunta de *per què* falla una cau, i sense elles l'usuari té la xifra però no la causa. La inspecció de l'estat permet arribar-hi per una altra via, més lenta però **més formativa**, veure dos blocs expulsant-se mútuament d'un

conjunt mentre la resta de la cau està buida és observar una fallada de conflicte, i la conclusió que l'usuari n'extreu és seva en comptes de ser una etiqueta que li dona l'eina.

3.4.4 Efecte sobre la validesa dels resultats

Les xifres que dona l'eina són vàlides per a la **comparació relativa** entre configuracions sota el mateix programa, o entre programes sota la mateixa configuració. Si una configuració dona menys fallades que una altra, la conclusió es manté en un sistema real, si un patró d'accés és millor que un altre, també. Les simplificacions no comprometen l'objectiu del treball.

S'ha de tindre clar que les xifres **no són predictives** en termes absoluts: el temps mitjà d'accés o el CPI que reporta l'eina no estimen el que s'obtindria en cap màquina concreta, perquè no hi ha correspondència entre les latències configurades i les d'un maquinari real. Qui necessiti xifres comparables amb *hardware* real ha de fer servir les eines de l'apartat 2.6, que estan dissenyades per a això.

Les simplificacions podrien induir a un error. Per exemple, l'absència de *buffer* d'escriptura fa que l'escriptura immediata sembli *pitjor* del que és. L'absència de segmentació i d'execució fora d'ordre fa que qualsevol fallada sembli aturar completament el processador, cosa que *sobreestima* l'impacte de les fallades en general. I l'absència de *prefetch* fa que patrons d'accés regulars semblin *més costosos* del que serien en maquinari que els sabria anticipar.

Aquests possibles errors són acceptables ja que com hem dit en reiterades ocasions, és una **eina d'aprenentatge**, no competeix amb un simulador de recerca en fidelitat, i no ho intenta, el que ha d'oferir és que la relació entre una decisió i el seu efecte sigui **neta i atribuïble**, i això s'aconsegueix precisament eliminant les variables que a la realitat s'hi superposen. Un model que ho modelés tot seria més fidel i alhora molt pitjor perquè cap efecte observat no seria atribuïble a una sola causa.

3.5 Requisits no funcionals

Els requisits anteriors estableixen *què* ha de fer l'eina; aquests estableixen *com* ho ha de fer. En una eina d'aprenentatge la distinció importa més del que sembla, perquè bona part del valor pedagògic no depèn de la funcionalitat sinó de la manera com s'ofereix. Un simulador que calculi exactament el mateix però trigui uns segons a respondre o exigeixi preparar fitxers deixa de complir l'objectiu encara que compleixi tots els requisits funcionals.

3.5.1 Rendiment i interactivitat

La resposta ha de ser **immediata**. Cada pas de simulació, sigui quina sigui la granularitat, ha de completar-se i reflectir-se a la interfície en un temps que l'usuari percebi com a instantani. La justificació no és de comoditat sinó pedagògica. El procés de comparació depèn que provar una configuració nova sigui *barat*, perquè la intuïció no apareix a la segona prova sinó a la vintena. Qualsevol fricció perceptible entre l'acció i el resultat redueix el nombre de proves que l'usuari farà, i per tant redueix directament el que aprendrà.

Les estructures que l'usuari pot inspeccionar poden arribar a ser molt grans, una memòria principal amb molts marcs, o una cau de mida considerable, i transferir-les senceres a la interfície després de cada pas seria incompatible amb el requisit anterior. Per tant, l'eina ha de transferir **només la porció que l'usuari està mirant**, i el volum per consulta ha d'estar acotat.

L'eina està dimensionada per a **programes curts**, de l'ordre de desenes o centenars d'instruccions amb bucles, no per a càrregues de treball reals. És una conseqüència directa del propòsit: els patrons que vol fer visibles s'observen en pocs accessos, i un programa llarg produiria un volum d'esdeveniments que faria impossible el seguiment pas a pas.

Canviar un paràmetre i tornar a executar ha de ser una operació immediata que *no* exigeixi reiniciar l'aplicació ni recarregar el programa.

3.5.2 Usabilitat

Tots els paràmetres de l'apartat 3.3 s'han de poder definir des de la **interfície gràfica**, amb els valors i les restriccions visibles al mateix lloc on es configuren. L'usuari no ha d'haver de consultar documentació externa per saber quins valors són vàlids ni haver d'entendre un format de fitxer.

Aquest és el requisit d'usabilitat més específic d'aquesta eina i el que més la diferencia d'una aplicació qualsevol. Diverses relacions entre paràmetres són **en si mateixes contingut pedagògic**: que el nombre de conjunts es derivi de tres paràmetres, que els nivells de taula estiguin acotats pels bits disponibles, que certes mides hagin de ser potències de dos. Quan l'eina impedeix o ajusta una configuració, ha de fer visible **per què**, perquè aquesta explicació és exactament el que l'apartat 3.1.1 estableix com a objectiu.

L'usuari no ha d'haver de fer cap acció per veure què està passant: l'estat de la jerarquia i les mètriques han de ser accessibles **en tot moment**, sense obrir diàlegs ni executar comandes. És el complement d'usabilitat del requisit funcional d'inspecció, i el que permet que l'atenció de l'usuari es mantingui en el que està observant.

Quan un programa no compila o una configuració no és vàlida, el missatge ha d'identificar el problema **en termes que l'usuari entengui** i, si és possible, on. L'usuari final de l'apartat 3.1.2 és algú que pot estar escrivint assemblador per primera vegada, i un error críptic el bloqueja fora del bucle d'aprenentatge abans d'haver-hi entrat.

3.5.3 Portabilitat i distribució

L'eina ha de funcionar als tres sistemes d'escriptori habituals (*Windows*, *macOS* i *Linux*), perquè l'usuari final és un col·lectiu d'estudiants on cadascú té un sistema operatiu diferent a l'altre.

L'aplicació s'executa a la **màquina de l'usuari** sense dependre de cap servei extern ni connexió. La justificació són tres punts: primer de tot, no és necessari cap tipus de connexió per simular una jerarquia. Segon, garanteix que funcioni en qualsevol context d'ús, per exemple, una aula sense xarxa fiable. I finalment, evita tota la infraestructura que un desplegament al servidor implicaria.

L'usuari ha de poder començar a fer servir l'eina amb el **mínim d'esforç possible**. Per exemple, Amnesia i MARS es distribueixen com a arxius executables i RIPES com a binaris precompilats, mentre que les eines de recerca exigeixen compilació. La distribució de binaris es manté com la millor opció per a aquest projecte.

3.5.4 Mantenibilitat i extensibilitat

El nucli de simulació i la capa de presentació han de ser **independents**, de manera que el comportament simulat no depengui de com es mostra i que la interfície pugui evolucionar sense tocar el model. Fa possible provar el motor de manera automatitzada sense interfície.

El disseny ha de permetre incorporar una **política de reemplaçament o d'escriptura nova** tocant una superfície reduïda del codi i sense modificar la lògica de recorregut de la jerarquia. És el tipus d'extensió més probable i és també la que un docent podria voler afegir per a un exercici concret.

El processament del codi font està organitzat com un **compilador**. L'entrada passa per les fases d'anàlisi lèxica, sintàctica i semàntica fins a generar una *representació intermèdia*, i és aquesta representació el que el motor de simulació executa. La conseqüència és que el motor **no coneix cap llenguatge concret**, de manera que incorporar-ne un de nou consisteix a afegir un *front-end* que generi la mateixa representació intermèdia, sense modificar ni el motor ni la capa de presentació.

3.6 Síntesi de l'abast

L'eina que es descriu en aquest treball és una **aplicació d'escriptori** que executa programes escrits per l'usuari en assemblador RISC-V sobre una jerarquia de memòria definida per ell mateix. La jerarquia admet un **nombre no acotat** de nivells de memòria cau, cadascun amb la seva pròpia geometria, organització i polítiques, i incorpora opcionalment el camí complet de traducció d'adreces, amb memòria de traduccions, taula de pàgines multinivell, gestió de marcs i espai d'intercanvi. Durant l'execució, que pot avançar per acció atòmica, per instrucció o de manera contínua, l'usuari pot inspeccionar el contingut real de totes les estructures i consultar les mètriques de rendiment, que s'actualitzen a cada pas. El model d'execució és funcional amb comptabilitat de cicles, sense segmentació ni els mecanismes que amaguen latència en un processador real, de manera que els resultats són vàlids per **comparar** configuracions entre si però no per **predir** el comportament de cap màquina concreta.

D'aquest abast se'n deriven **quatre criteris** que condicionen les decisions tecnològiques de l'apartat següent. En primer lloc, el requisit de disponibilitat als tres sistemes d'escriptori habituals exigeix una via de desenvolupament i distribució *multiplataforma* que no impliqui mantenir versions separades. En segon lloc, la combinació del recorregut pas a pas amb la inspecció completa de l'estat exigeix un nucli de simulació prou *eficient* perquè cada pas es resolgui de manera imperceptible, i un mecanisme de comunicació entre motor i presentació que no penalitzi l'actualització constant de les vistes. En tercer lloc, la quantitat i la varietat d'estructures que cal representar demana una capa de presentació capaç de gestionar *interfícies denses* que canvien contínuament. I finalment, la separació entre motor i presentació ha de ser prou estricta perquè el primer sigui *verificable de manera independent*, condició de la qual depèn tot el procés de validació.

4 Tecnologies i eines

Els criteris derivats de l'abast anterior condicionen les decisions tecnològiques del projecte. Aquest capítol justifica les quatre eleccions principals mesurant-les contra aquests criteris, sense entrar a descriure unes tecnologies que ja s'han documentat àmpliament.

4.1 Criteris de selecció

Com ja hem esmentat en l'apartat 3.6, els criteris importants seran la **multiplataforma** sense mantenir versions separades, l'**eficiència** del nucli de simulació i de la comunicació amb la presentació, la **capacitat de la capa visual** per a interfícies denses i canviants, i la **separació estricta** entre motor i presentació per poder verificar el primer aïlladament.

En l'elecció de tecnologies s'han aplicat aquests quatre criteris. De forma menys significativa, també es considerarà la quantitat de *documentació* de la tecnologia, i l'*experiència prèvia* personal en ella.

4.2 Marc d'aplicació

Abans d'entrar en les tecnologies podem mirar quina família d'aplicació s'ajusta millor a les nostres necessitats. Aplicació *nativa per plataforma*, aplicació *web* servida des d'un servidor o aplicació d'*escriptori* amb interfície construïda amb tecnologies web.

L'alternativa d'usar la nativa pura no és del tot adient. Compleix el criteri d'eficiència però **falla el de multiplataforma**: exigiria o bé mantenir versions separades o bé un marc gràfic multiplataforma amb un cost de desenvolupament d'interfície molt superior al d'una capa web.

Una web tampoc és una opció viable. Compleix multiplataforma i capacitat visual, però exigiria **infraestructura de servidor i connexió**, i el requisit era execució local sense dependències externes.

Família	Multiplat.	Eficiència	Capa visual	Sense servidor	Cost desenv.
Nativa per plataforma	×	Alta	Mitjana	✓	Molt alt
Nativa multiplataforma	✓	Alta	Mitjana	✓	Alt
Web (servidor)	✓	Mitjana	Alta	×	Mitjà
Escriptori amb webview	✓	Alta	Alta	✓	Mitjà

Taula 6: Famílies d'aplicació avaluades contra els criteris de selecció. Elaboració propia.

Dins d'aquesta família, l'alternativa consolidada és **Electron** [13], i la principal alternativa recent és **Tauri** [25], existeixen també propostes com Wails [31] o Neutralino [11], que segueixen el mateix model amb altres llenguatges però amb comunitats i ecosistemes considerablement menors i no aporten cap avantatge addicional en aquest cas.

Un dels punts importants a veure és el **llenguatge utilitzat al *back-end***. En el cas d'Electron seria JavaScript, encara que sempre es pot implementar un servidor i connectar-los amb una API. Encara que això ve amb les seves limitacions. Per una banda tenim que hauríem de distribuir *dos binaris per plataforma* i per altra banda la gestió de dos processos diferents: gestionar el cicle de vida del procés fill, arrencar-lo, detectar si ha mort, tancar-lo quan l'aplicació es tanca, i decidir què passa si el port ja està ocupat. S'ha de definir el protocol un mateix, amb la serialització i el maneig d'errors que impliqui. I si és per *socket*, s'ha obert un port a la màquina de l'usuari, amb el que això comporta. Tauri fa exactament aquesta arquitectura però **el pont ja hi és**, amb el sistema de comandes i esdeveniments, l'empaquetat conjunt i la gestió del cicle de vida resolts. La separació entre motor i interfície és exactament el que necessitem, i una de les dues opcions la dona feta mentre l'altra la deixa construir.

Es va escollir, per tant, **Tauri** com a marc d'aplicació [25], decisió que arrossega Rust com a llenguatge del motor i que es desenvolupa a l'apartat següent. Cal reconèixer, però, **dues renúncies**. La primera és de *maduresa*: Electron acumula molts més anys, una documentació més extensa i una comunitat més gran, cosa que en un projecte desenvolupat per una sola persona i amb temps limitat és un avantatge. La segona és *estructural* i deriva precisament del que fa lleugera l'opció escollida: en fer servir el motor de renderització de cada sistema operatiu, la interfície es renderitza amb tecnologies diferents segons la plataforma, de manera que **no hi ha garantia d'un comportament idèntic** del *front-end* a tot arreu. Empaquetar un motor propi, com fa l'alternativa, elimina aquest risc a canvi del cost que s'ha descrit.

Marc	**Llenguatge**	**Motor web**	**Distribuïble**	**Documentació**
Electron	JavaScript	Empaquetat	Gran	Molt alta
Tauri	Rust	Del sistema	Reduït	Alta
Wails	Go	Del sistema	Reduït	Mitjana
Neutralino	C++	Del sistema	Reduït	Baixa

Taula 7: Marcs d'aplicació d'escriptori amb interfície web. Elaboració propia basada en [13][25][31][11]

4.3 Llenguatge del motor

El motor executa instruccions, recorre estructures i actualitza estat a cada pas, i el requisit d'interactivitat exigeix que això sigui **imperceptible**. A més ha de ser verificable aïlladament i, en aquest cas, ve parcialment determinat pel marc escollit a l'apartat 4.2.

Recalcant possibles alternatives, per una banda tenim l'opció d'utilitzar el llenguatge de la pròpia interfície, cosa que simplificaria el projecte en no haver d'implementar l'intercanvi d'informació entre el *back-end* i el *front-end*. Aquesta opció seria la indicada per a programes curts com els mencionats a l'apartat 3.5.1, això no és suficient ja que el criteri decisiu no és el rendiment sinó la separació verificable. Per altra banda, podríem, com hem dit abans, utilitzar un programa amb un llenguatge compilat, fent ús d'una API.

El factor decisiu és la **separació estricta entre motor i presentació**. El motor no pot accedir a la interfície encara que algú ho intenti, i és verificable amb proves automatitzades sense res de la capa visual. A això s'hi suma el *sistema de tipus*, que fa que la representació de la jerarquia sigui explícita i que **estats invàlids no es puguin construir**.

Tenint tot això en compte és com s'ha arribat a la conclusió que **Rust** com a llenguatge de *back-end* és l'opció que millor s'ajusta a aquests criteris. [19] Primer de tot ja està implementat en l'arquitectura de Tauri, així que afegeix simplicitat i ens manté en el mateix ecosistema per buscar documentació i suport per part dels usuaris. Això no ens evita renunciar a certs aspectes. Primer de tot, la **corba d'aprenentatge** de Rust és notable per a qualsevol programador i intimida possibles contribuïdors a l'eina en termes discutits a l'apartat 3.5.4. Després també tindrem en compte la **duplicació de tipus** entre el motor i la interfície, que obliga a mantenir sincronitzades dues definicions de les mateixes estructures.

4.4 Tecnologia de la interfície

El criteri d'elecció es basarà en la necessitat de tindre *moltes vistes amb molts elements* que canvien després de cada pas. El problema no és la complexitat de cada vista sinó la **freqüència i el volum de l'actualització**, i això apunta a un marc amb gestió *reactiva* de l'estat en comptes de manipulació manual del document.

Els marcs reactius habituals compleixen tots el requisit funcional, el que els separa en un projecte d'una sola persona és la **corba d'entrada** i la quantitat de decisions d'arquitectura que imposen.

Marc	**Reactivitat**	**Corba d'entrada**	**Ecosistema**	**Mida del resultat**
React	Virtual DOM	Mitjana	Molt ampli	Mitjana
Vue	Virtual DOM	Baixa	Ampli	Mitjana
Angular	Detecció de canvis	Alta	Ampli	Gran
Svelte	Compilada	Baixa	Reduït	Petita
SolidJS	Granular	Mitjana	Reduït	Petita

Taula 8: Marcs de desenvolupament d'interfícies web considerats. Elaboració propia basada en [9][30][5][24][23]

Totes les eines són molt semblants però **Vue** destaca per la corba de dificultat, que és molt baixa respecte a la resta de marcs. [30] Un altre punt a favor és l'*experiència prèvia* en l'entorn de treball, la qual ha facilitat la feina. Dit això, amb aquesta elecció ens quedaríem amb un **ecosistema més petit** que el de l'alternativa dominant, React, cosa que es nota quan es busca una biblioteca específica.

4.5 Eines de suport

Més enllà de les tecnologies principals, el desenvolupament s'ha recolzat en un conjunt d'eines auxiliars. La major part no responen a una decisió independent sinó que venen determinades pels ecosistemes escollits als apartats anteriors, de manera que es descriuen breument.

Al *back-end* s'empra **Cargo**, el gestor de paquets i sistema de construcció estàndard de Rust [18], que s'encarrega tant de les dependències com de la compilació i de l'execució de les proves. A la interfície s'utilitza **npm** per a la gestió de dependències [12] i **Vite** com a eina de construcció i servidor de desenvolupament [28], que és el que integra el marc d'aplicació per defecte.

Les proves del motor s'escriuen amb el **sistema de proves integrat de Rust** [18], que permet executar-les sense cap dependència addicional ni cap component de la interfície, i que és el que fa efectiva la separació establerta com a requisit a l'apartat 3.5.4. Per a la capa de presentació s'ha emprat **Vitest** [29], que permet comprovar el comportament dels components de manera aïllada.

I finalment, el codi es manté en un repositori **Git** allotjat a GitHub, amb una branca per cada funcionalitat desenvolupada. Com a editor s'ha emprat **Neovim** amb els *plugins* corresponents als dos llenguatges del projecte, i s'han utilitzat les eines de format i anàlisi estàtica que cada ecosistema proporciona per defecte.

5 Disseny de l'arquitectura

En aquest apartat es descriuen les **decisions estructurals** del sistema: com es reparteix la responsabilitat entre les parts, com es representen les dades, i com es resol el problema central del projecte, que és poder **aturar un accés a memòria enmig del seu recorregut**. La seva materialització concreta, amb el codi que li correspon, es tracta al capítol d'implementació.

5.1 Visió general del sistema

El sistema es divideix en **tres blocs** clarament diferenciats: la interfície, escrita en *Vue* i executada dins del *webview*, el motor, escrit en *Rust* i executat com a procés natiu, i el **pont de Tauri**, que és l'**única via de comunicació** entre tots dos. Les vistes no tenen accés directe al motor, ni el motor coneix res de com es presenta la informació. Aquesta separació és la que fa possible la resta de decisions que es descriuen en aquest capítol.

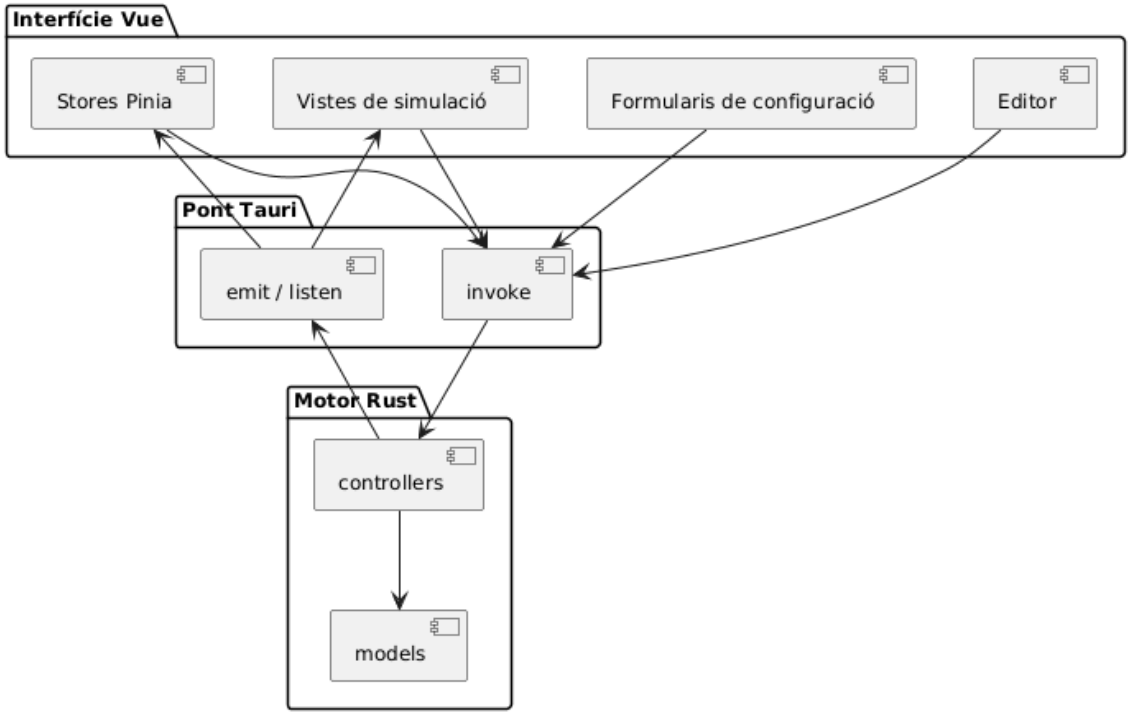


Figura 12: Diagrama de components. Font: elaboració pròpia [16].

Aquesta divisió en tres blocs, i no una aplicació monolítica, respon a dos motius. El primer és que el motor ha de ser **verificable sense cap component d'interfície**,

tal com estableix el requisit de mantenibilitat de l'apartat 3.5.4. El segon és de **rendiment**: el càlcul de la simulació no pot competir amb el renderitzat de la interfície pel mateix fil d'execució.

El diagrama distingeix **dos sentits de comunicació** amb naturalesa diferent: les *comandes*, que viatgen cap avall de manera **síncrona**, i els *esdeveniments*, que viatgen cap amunt de manera **asíncrona**. Cadascun d'aquests dos mecanismes es detalla més endavant, a l'apartat 5.5 d'aquest mateix capítol.

5.2 Elecció de l'arquitectura MVC

El patró es projecta directament sobre l'estructura de directoris del projecte. Les **vistes** són els components de Vue, els **controladors** corresponen als quatre fitxers del directori `controllers/`, i el **model** ocupa el directori `models/`, dividit en `interpreter/` i `simulation/`. Cadascuna d'aquestes tres parts es descriu amb més detall a mesura que avança el capítol.

La divisió dels controladors és **per tipus d'operació i no per estructura de dades**: `editor_controller` s'ocupa de la compilació, `simulation_session_controller` del cicle de vida de la sessió, `memory_contents_controller` de la lectura de contingut, i `stats_controller` de les estadístiques. Totes les vistes de contingut fan la mateixa operació sobre estructures diferents, i agrupar-les per estructura hauria duplicat la validació de finestres a set llocs diferents.

El model és **accessible i verificable sense passar pels controladors**. Els 28 fitxers que hi ha sota `models/` inclouen proves marcades amb `#[cfg(test)]` que instancien directament `SetAssociativeCache`, `Tlb` i `PageTable`. És aquesta propietat la que fa efectiu el requisit de *mantenibilitat* descrit a l'apartat 3.5.4.

La dependència entre les dues capes és **unidireccional i no té cicles**: cap fitxer de `models/` importa res de `controllers/`. La figura 13 marca explícitament aquest sentit únic, que és el que permet compilar i provar el model de manera completament aïllada de la resta del sistema.

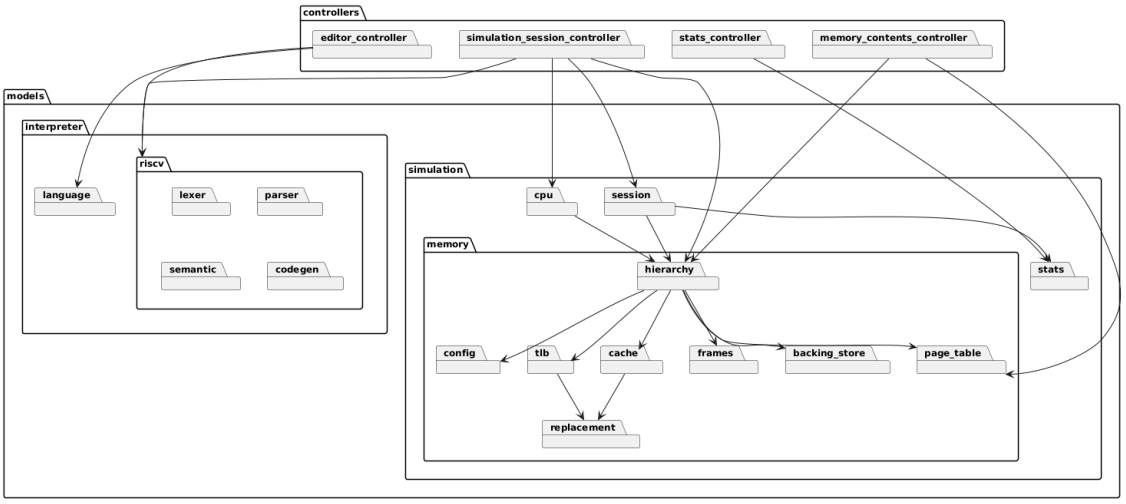


Figura 13: Diagrama de paquets. Font: elaboració pròpia [16].

El patró no s'aplica sense excepcions, i és preferible declarar-les **obertament**. Al *back-end*, els controladors no fan una delegació pura: constants com `MAX_COUNT = 512` o `DEFAULT_BLOCK_SIZE = 32` viuen al controlador i no al model.

Al *front-end* la separació és encara més **parcial**: `Editor.vue`, `CacheLevelView.vue`, `TlbView.vue`, `RamView.vue` i `StatsView.vue` criden `invoke()` directament, i només el control d'execució passa per un magatzem. Aquesta decisió es justifica i es retoma a l'apartat 5.6: cada vista demana una finestra diferent sobre una estructura diferent, i centralitzar-ho hauria significat un magatzem que replica **set formes de paginació diferents**.

5.3 Model de dades de la jerarquia

Tot el model de dades parteix d'una distinció entre **dues jerarquies de tipus completament separades**: la *configuració*, que recull què demana l'usuari i queda fixada en iniciar la sessió, i l'*estat en execució*, que recull què està passant en cada moment. Cap dels dos costats comparteix tipus amb l'altre, cosa que es desenvolupa en les subseccions següents.

5.3.1 Configuració

L'arrel de la configuració és `HierarchyConfig`, que recull la ISA, si la memòria virtual està activa, la mida de pàgina, la llista de nivells (`Vec<Level>`), la TLB, la RAM, el disc i la llavor aleatòria. Cada `Level` baixa fins a un màxim de tres

`CacheEntry` (unificada, d'instruccions i de dades), i cada entrada acaba en una `CacheConfig` amb la seva pròpia geometria.

Tota aquesta estructura es compon **per valor**, sense `Rc` ni `Arc`: una configuració és **autocontinguda** i no comparteix referències amb res més. La conseqüència pràctica és que és *clonable* i *serialitzable* sencera, cosa que permet enviar-la des del formulari amb una sola comanda, `start_simulation`.

La configuració defineix **cinc *enums* de política**: `ReplacementPolicy`, `TlbReplacementPolicy`, `WritePolicy`, `InclusionPolicy` i `FrameAllocationPolicy`. Cadascun d'aquests camps es correspon directament amb un dels paràmetres configurables que enumera l'apartat 3.3. La figura 14 mostra l'arbre de classes complet.

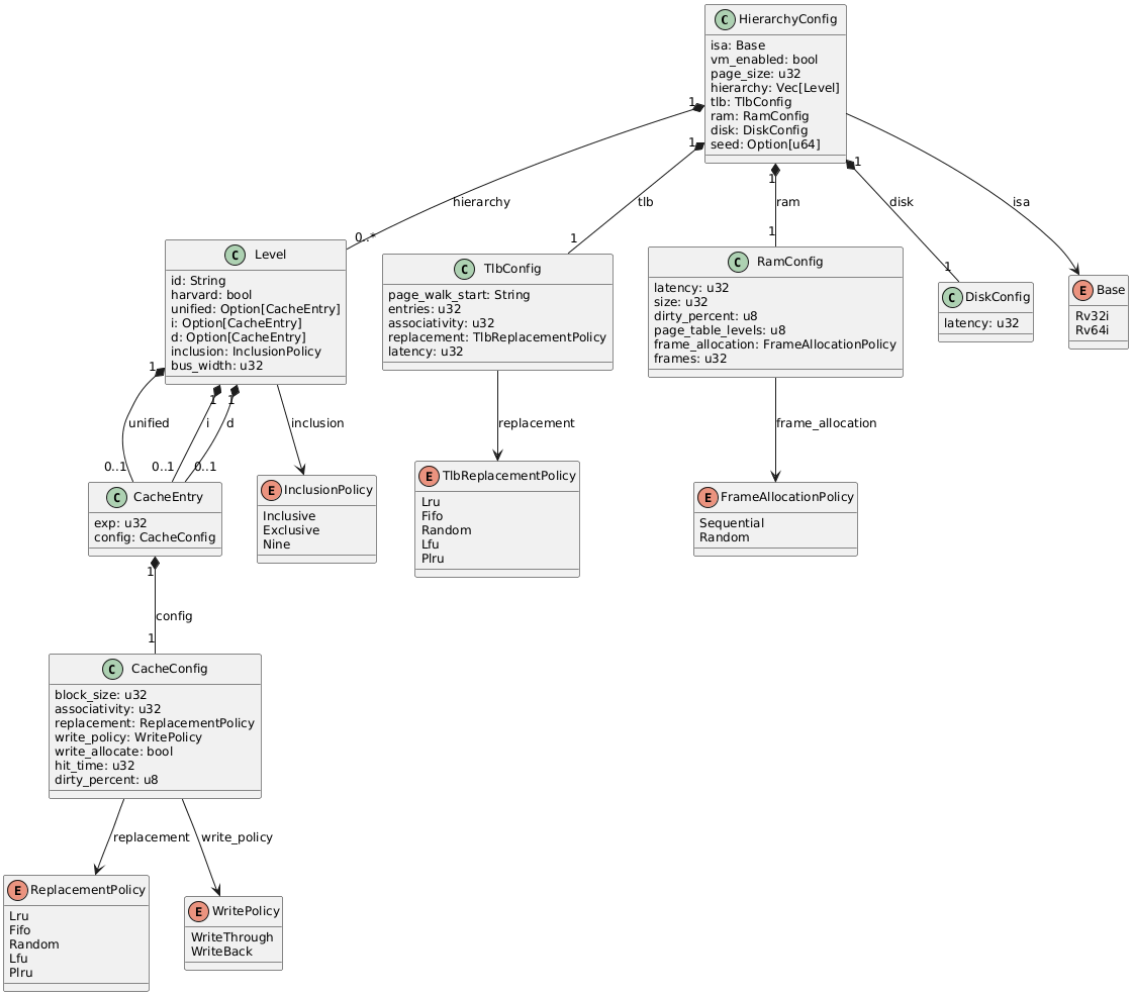


Figura 14: Diagrama de classes de configuració. Font: elaboració pròpia [16].

5.3.2 Estat en execució

L'orquestrador de l'estat en execució és `MemoryHierarchySimulator`, i els seus camps es divideixen en **dos grups**. El primer conté els components de la jerarquia: la TLB, la taula de pàgines, els marcs, els nivells de cau, la RAM i el disc. El segon conté l'**estat de l'accés en curs** (`in_progress`, `cursor`, `finished_result`, `translated_address`, `walk`, `pending_fault`), i és aquest segon grup el que fa possible tot el que es descriu a l'apartat 5.4.

`Tlb` és un **tipus propi**: no hereta de `SetAssociativeCache` ni el conté, però reprodueix el mateix mecanisme de conjunts, associativitat i política. El motiu és que el que guarda és una correspondència VPN→PPN, no una etiqueta i un bloc de bytes, unificar-los hauria obligat a parametritzar el contingut de la línia segons el tipus de dada que conté.

`DataStore` és, en canvi, una **mateixa abstracció** que es reutilitza de forma independent tres vegades: per a la memòria principal, per a l'espai d'intercanvi i pel contingut dins de `FrameTable`. Les tres instàncies no comparteixen res entre elles, cosa coherent amb el fet que representen espais d'adreçament diferents.

La taula de pàgines és un únic `HashMap<(nivell, clau), PageTableEntry>`, sense cap *struct* de taula ni nodes enllaçats: **l'arbre hi és implícit a les claus**. Es construeix incrementalment i mai s'omple del tot, de manera que uns nodes enllaçats reservarien memòria per a entrades que potser no arribaran a existir mai. La conseqüència és que la **navegació pare-fill** que exigeix el requisit d'inspecció es reconstrueix a cada consulta, i per això la comanda de lectura rep sempre `level` i `parent_key`. La figura 15 recull totes aquestes classes.

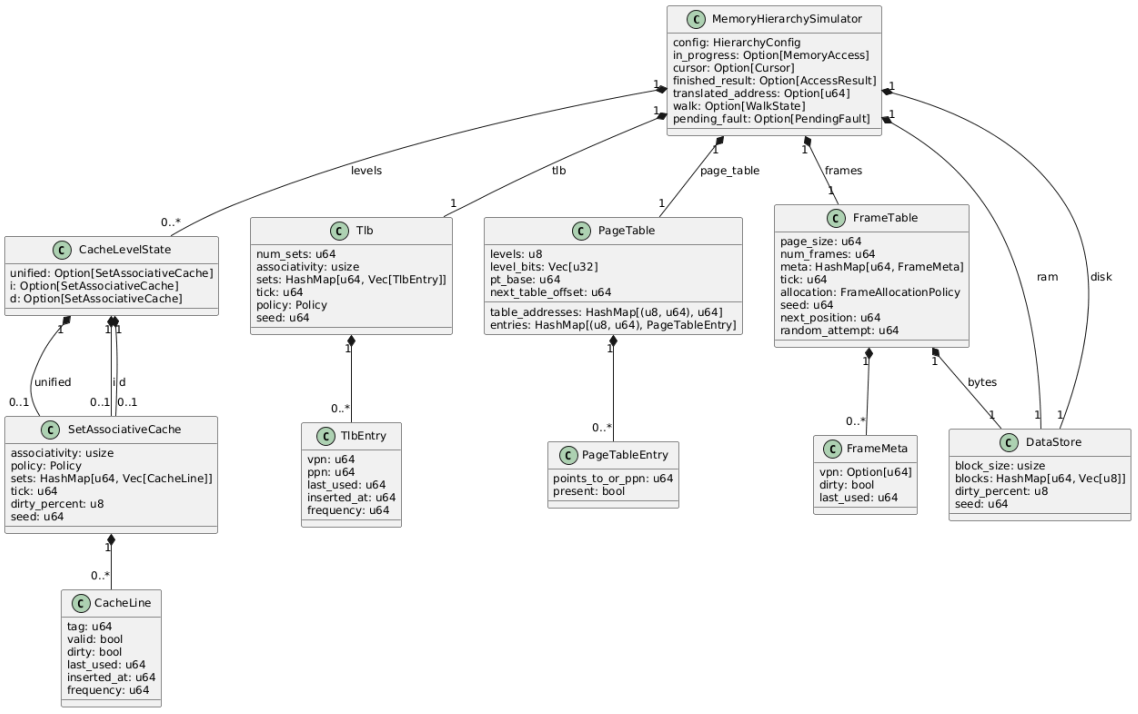


Figura 15: Diagrama de classes de l'estat en execució. Font: elaboració pròpia [16].

5.3.3 Relació entre configuració i estat

El simulador posseeix la seva pròpia còpia de `HierarchyConfig`: la rep **per valor** al constructor, la consumeix, i només l'exposa cap enfora com a **referència prestada** a través de `config()`. Cap altra part del sistema pot mutar-la directament.

Cada component extreu només els paràmetres que necessita: `Tlb`, `PageTable` i `FrameTable` guarden valors solts en comptes de la configuració sencera, mentre que cada cau sí que conserva la seva pròpia `CacheConfig` completa. La diferència reflecteix quants paràmetres necessita realment cada component durant l'execució.

Això té dues implicacions directes. Per a la **reconfiguració**, no hi ha canvi en calent: modificar un paràmetre sempre significa iniciar una sessió nova. Per a les **proves**, cada component es pot instanciar amb els seus paràmetres directament, sense haver de construir mai una `HierarchyConfig` completa.

5.4 El motor de simulació

Un accés a memòria **no és atòmic**: pot travessar traducció, recorregut de taula de pàgines, intercanvi i diversos nivells de cau. El requisit d'avanç per acció de

l'apartat 3.2.2 exigeix poder aturar-se enmig d'aquest recorregut, cosa que obliga a representar explícitament **en quin punt es troba** en cada moment.

5.4.1 El cursor d'accés

El punt on es troba un accés es representa amb l'*enum* `Cursor`, amb variants `Tlb`, `Walk`, `SwapOut`, `SwapIn`, `Cache(usize)` i `Ram`. És rellevant que **no hi hagi cap variant de disc per a un accés normal**: el disc només s'assoleix a través de l'intercanvi, mai directament.

Un pas del simulador és, per tant, una **transició del cursor**. La funció `advance()` despatxa per `match` a `step_tlb`, `step_walk`, `step_swap_out`, `step_swap_in`, `step_cache` i `step_ram`, i cadascuna d'aquestes funcions o bé fixa el cursor següent o bé acaba l'accés fixant `finished_result`.

L'alternativa d'una implementació recursiva es va descartar perquè no es pot aturar enmig sense fils ni corrutines, i cal retornar el control al bucle d'esdeveniments després de cada pas per poder refrescar les vistes. A més, un **cursor explícit** és inspeccionable per si mateix, i és precisament el que permet informar la interfície de quin element concret s'està accedint en cada moment.

5.4.2 Transicions

La taula completa de transicions recull, per a cada estat d'origen, el destí i la condició que el determina, incloent-hi les dues entrades inicials possibles segons si `vm_enabled` és cert o fals. No es reprodueix aquí sencera, però el seu contingut és el que implementen literalment les funcions `step_*` descrites a l'apartat anterior.

Hi ha **dos punts a destacar**. El primer és que `Cache(i)` és l'única variant que pot acabar l'accés o continuar-lo segons el resultat i la política d'escriptura: un encert d'escriptura amb escriptura immediata continua cap al nivell següent, mentre que un encert amb postescriptura acaba l'accés. El segon és que `Ram` és sempre **terminal**.

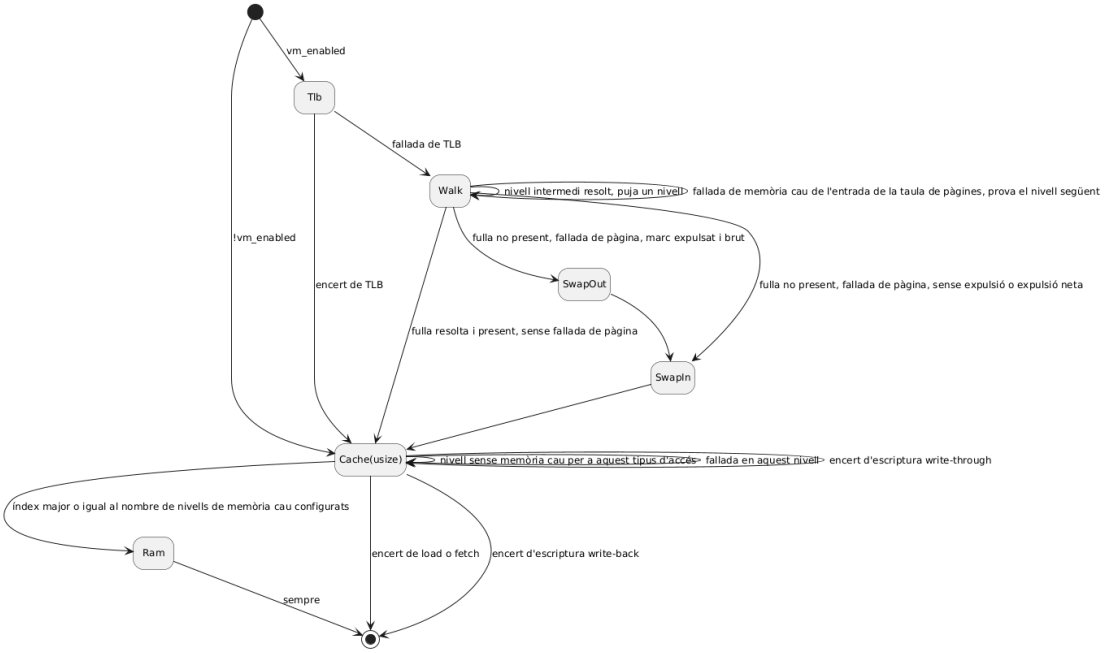


Figura 16: Estats del cursor. Font: elaboració pròpia [16].

La figura 16 és la **figura important** d'aquest capítol: correspon exactament al camí complet d'un accés que s'ha descrit al marc teòric del treball, ara traduït als estats concrets que implementa el motor.

5.4.3 Resolució d'una fallada de pàgina

Quan una traducció falla, **no hi ha cap estat de represa** ni es torna a llançar l'accés des de zero. El mateix `MemoryAccess` que ja estava guardat a `in_progress` travessa `Tlb` → `Walk` → `SwapOut`/`SwapIn` i continua exactament des d'on l'havia deixat.

El punt de reentrada és sempre `Cursor::Cache(0)`, amb `translated_address` ja fixat. A partir d'aquí, l'accés continua exactament igual que si no hi hagués hagut cap fallada, sense cap branca de codi especial per a aquest cas.

`RiscvCpuSimulator` **no distingeix en cap moment** si hi ha hagut una fallada de pàgina: només rep el senyal `instruction_complete`, igual que en qualsevol altre accés. La conseqüència de disseny és que la CPU **no necessita saber res de memòria virtual**. La figura 17 en mostra la seqüència completa.

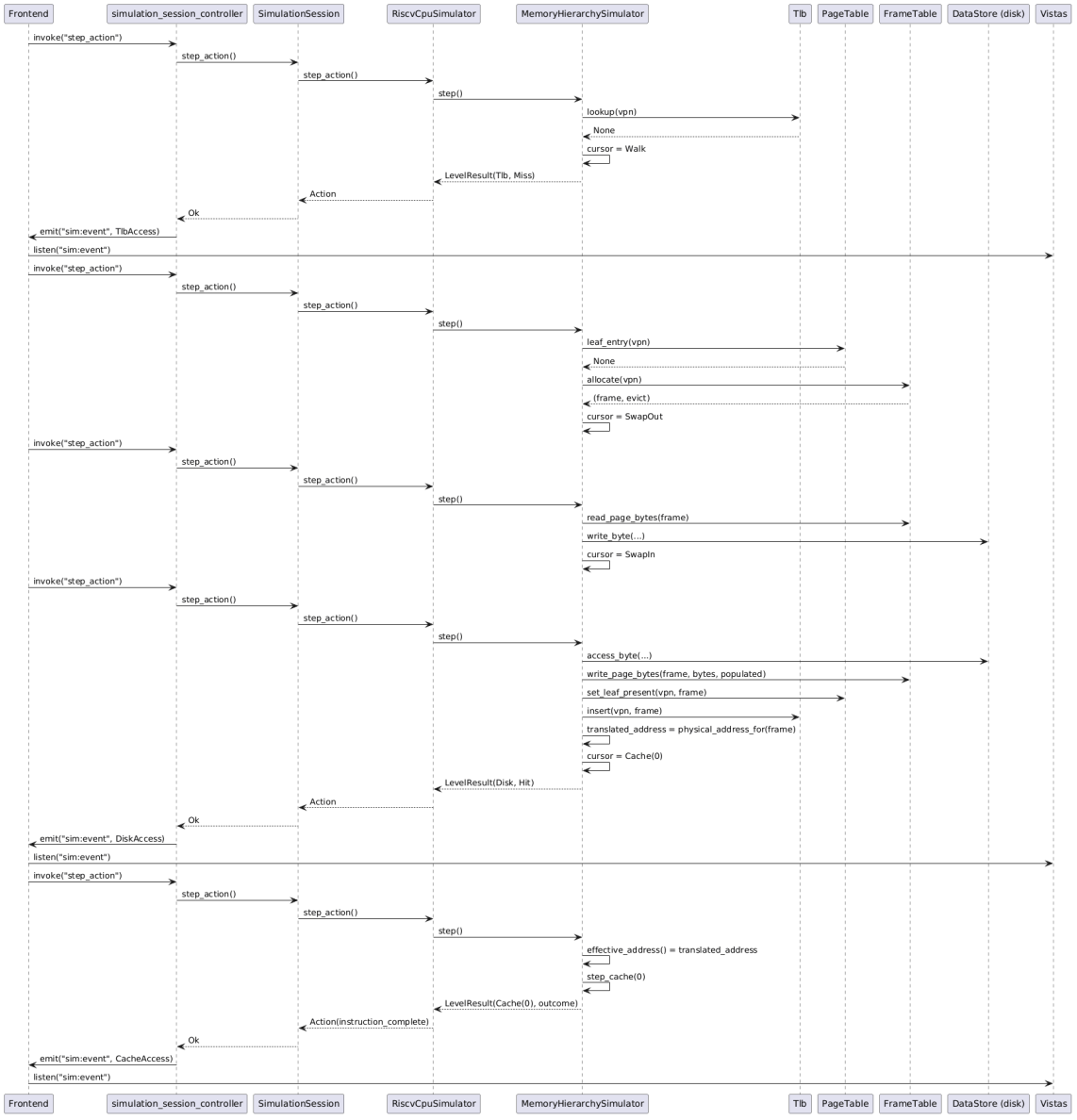


Figura 17: Seqüència d'un accés amb fallada de pàgina. Font: elaboració pròpia [16].

5.4.4 Comptabilitat de cicles

El comptador de cicles viu a `StatsAccumulator.total_cycles`, i s'incrementa exclusivament des de **dos llocs**: `record_access`, que hi suma el cost del pas de memòria, i `record_instruction_complete`, que hi suma el cicle fix per instrucció.

Aquest **cicle base fix** és la materialització directa de la decisió presa a l'apartat 3.4 de no modelar segmentació ni riscos: el temps d'execució de la instrucció en si no es modela, només el temps d'accés a memòria.

L'AMAT, el CPI i l'MPKI es deriven **d'aquest mateix comptador** en el moment de prendre la instantània, en comptes d'acumular-se per separat. Aquesta decisió elimina qualsevol possibilitat que les mètriques es desincronitzin entre elles.

5.5 Comunicació Tauri

5.5.1 Comandes

Les comandes són el **mecanisme síncron** de petició i resposta: el *front-end* crida `invoke()`, i al *back-end* respon una funció marcada amb `#[tauri::command]`. El seu `Result<T, String>` arriba a JavaScript com una promesa *resolta* o *rebutjada*, segons si l'operació ha reeixit.

Les comandes s'agrupen **per finalitat**: compilació, control de sessió, lectura de contingut i estadístiques, seguint exactament la mateixa divisió que ja s'ha fet servir per als controladors a l'apartat 5.2. No cal reproduir aquí la taula sencera, n'hi ha prou amb els grups i un exemple representatiu de cadascun.

Les peticions de contingut no demanen mai un volum sencer, sinó una **finestra**: reben una posició i una quantitat, i retornen només aquesta franja, amb un **límit dur de 512 entrades** validat al controlador. És la resposta directa al requisit de volum transferit de l'apartat 3.5.1. L'única excepció és `get_stats`, que retorna sempre la instantània completa perquè la seva mida és fixa i no proporcional a la mida de la memòria configurada.

5.5.2 Esdeveniments

Tots els esdeveniments viatgen per un **únic canal**, `"sim:event"`, amb variants etiquetades definides a l'*enum* `SimEvent`: `RegWrite`, `CacheAccess`, `TlbAccess`, `InstructionComplete`, entre d'altres, serialitzades amb un camp discriminant. Un canal únic permet que qui escolta filtri per etiqueta, i que afegir un tipus d'esdeveniment nou no obligui mai a registrar un canal addicional.

És un **mecanisme asíncron** de notificació: el motor emet l'esdeveniment i no sap ni li importa qui l'escolta. Actualment hi ha **nou subscriptors independents** repartits per la interfície. Les figures 18 i 19 mostren, respectivament, la seqüència d'una comanda i la d'un esdeveniment amb els seus subscriptors.

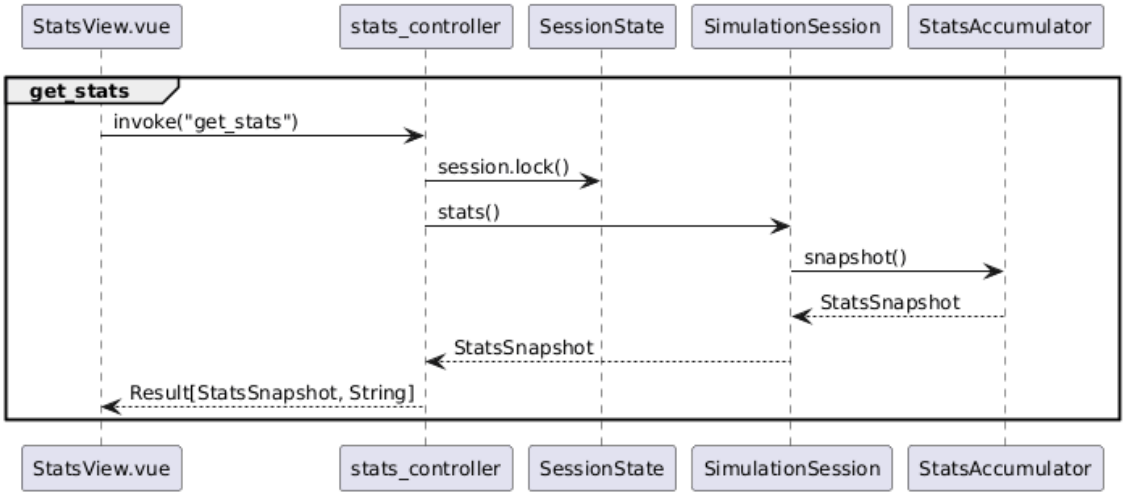


Figura 18: Diagrama de seqüència d'una comanda i la seva resposta. Font: elaboració pròpia [16].

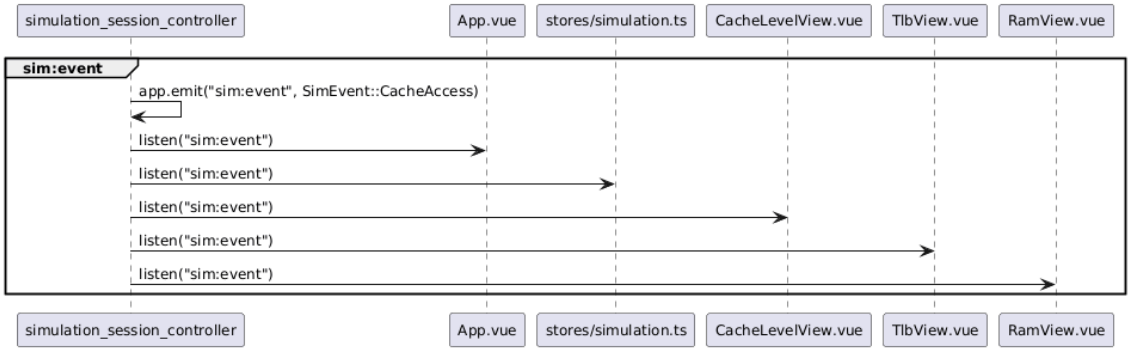


Figura 19: Diagrama de seqüència d'un esdeveniment amb els seus subscriptors. Font: elaboració pròpia [16].

5.5.3 Duplicació de tipus

Les estructures que travessen el pont existeixen definides **dues vegades**: un cop en Rust, com per exemple `CacheConfig` i `RamConfig`, i un altre cop com a interfícies equivalents en TypeScript. Aquesta duplicació **es manté manualment**, sense `ts-rs`, `specta` ni cap generació al `build.rs`. La conseqüència observable és que la convenció no és del tot uniforme, fins al punt que el retorn de `compile` ni tan sols té una interfície pròpia anomenada.

És el cost que ja s'havia anunciat a l'apartat 4.2 en triar Rust com a llenguatge del motor. Es van descartar **dues alternatives**: la generació automàtica de tipus,

perquè afegeix un pas de compilació i una dependència per a un conjunt d'estructures que canvia poc, i un esquema únic compartit entre totes dues capes.

5.6 Gestió de l'estat a la interfície

La interfície manté **tres magatzems**. `useCompileConfigStore` guarda els paràmetres de compilació i la mida del darrer binari generat, `useHierarchyConfigStore` guarda la configuració completa de la jerarquia, i `useSimulationStore` guarda `isActive`, `isRunning`, `speedMs`, `currentLine` i `activeView`.

La distinció principal és que aquests magatzems només guarden **estat compartit i de control**, el contingut de memòria no hi passa mai. De fet, els únics camps que són realment centralitzats i s'empenyen a totes les vistes són `currentLine` i `isRunning`.

Cada vista demana les seves **pròpies dades** en comptes de rebre-les del magatzem, perquè cadascuna necessita una finestra diferent sobre una estructura amb una forma de paginació diferent, i una vista que no és visible no ha de generar cap trànsit. El cost, dit obertament, és que la lògica de subscripció i de refresc es repeteix a cada component.

El **patró de refresc** és sempre el mateix: cada vista registra el seu `listen("sim:event")` en muntar-se, filtra per etiqueta i torna a demanar només la finestra que té visible en aquell moment. La figura 20 en recull tots els components i les seves subscripcions.

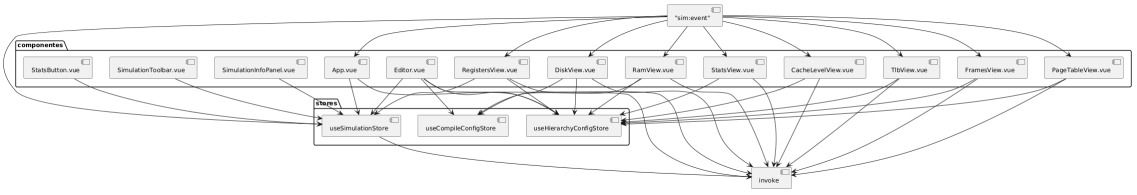


Figura 20: Diagrama de components del front-end. Font: elaboració pròpia [16].

5.7 Patrons de disseny aplicats

L'únic patró clàssic que encaixa completament és **Observer**: el motor emet `sim:event` sense conèixer els seus subscriptors. El problema concret que resol és que, amb sis vistes independents que poden estar muntades o no en cada moment, qualsevol alternativa hauria obligat el motor a mantenir una llista de qui vol rebre què.

Hi ha d'altres mecanismes que s'assemblen a patrons clàssics **sense ser-ho exactament**. La màquina d'estats del cursor resol el mateix que **State**, però amb despatx

per `match` sobre un *enum* en comptes d'un *trait* polimòrfic. `pick_victim` resol el mateix que **Strategy**, igualment amb `match` i sense una implementació separada per política. I la unicitat de la sessió la garanteix el marc d'aplicació mitjançant `.manage()`, no un `getInstance()` escrit a mà.

El **despatx per *enum*** és una decisió i no una mancança. El conjunt de variants és tancat i conegut per endavant, el compilador verifica de manera exhaustiva que cap `match` deixa un cas sense tractar, i s'evita una indirecció addicional a cada pas de la simulació.

Cal fer una darrera precisió: les comandes de Tauri són **invocació remota**, no el patró **Command**. No hi ha reïficació de l'ordre en un objecte, ni cua, ni desfer, precisament perquè el retrocés no existeix, tal com ja s'havia establert a l'apartat d'abast i requisits.

6 Implementació

El capítol anterior descrivia **quina estructura** té el sistema, aquest explica **com s'ha construït** i quins problemes concrets van aparèixer en fer-ho. L'ordre segueix el recorregut d'una sessió d'ús: del codi font a les instruccions, de les instruccions als accessos, dels accessos a les mètriques, i d'aquí a la interfície.

6.1 De l'assemblador al codi executable

Cal convertir el text que escriu l'usuari en **instruccions amb adreça assignada**. No es tracta d'un compilador d'ús general, sinó del *mínim necessari* per als programes curts que fixa l'abast (apartat 3.2.2), i aquesta restricció justifica bona part de les decisions que segueixen.

6.1.1 Les cinc etapes i la comunicació per fitxers

El procés de compilació es divideix en **cinc etapes** definides pel *trait* `Language`: `preprocess` neteja el text font, `tokenize` el converteix en testimonis, `parse` en construeix l'arbre sintàctic, `analyze` el valida, i `generate` produeix el codi final. `analyze` és l'única etapa que **no produeix una sortida pròpia**: només comprova que tot sigui correcte.

Cada etapa rep i escriu **camins de fitxer**, no valors en memòria: el fitxer `.s` preprocessat es converteix en `tokens.jsonl` (un testimoni per línia), aquest en `ast.json`, i finalment en `codegen.json`. Cap etapa no rep mai directament l'estructura de dades de l'anterior.

Aquesta separació es justifica per **tres motius**. Cada etapa és executable i verificable aïlladament donant-li només un fitxer, els formats intermedis són inspeccionables durant el desenvolupament sense haver d'instrumentar res, i la serialització ja era necessària per travessar el pont de Tauri [21], de manera que no s'hi afegeix cap dependència nova.

Cap d'aquestes cinc etapes no s'encadena des del model: **no existeix un** `compile()` **a** `models/`, sinó que és la mateixa comanda del controlador qui les crida una després de l'altra. És una altra manifestació de la delegació no pura que ja es va declarar a l'apartat 5.2.

6.1.2 Preprocessament

El preprocessament elimina els **comentaris de línia i de bloc**, aquests últims mantenen un estat entre línies i generen un error si no es tanquen abans d'acabar

el fitxer. Respecta el contingut que hi ha entre cometes, i normalitza els espais interiors tot preservant la indentació original.

La directiva `.include` es **rebutja explícitament**, amb un missatge que indica que només s'admet un únic fitxer font. La justificació és *pedagògica*: els programes són de desenes d'instruccions, i la inclusió hi afegiria una dimensió de gestió de fitxers aliena al que l'eina vol ensenyar. Tampoc no hi ha cap tipus de **macro**.

Aquesta etapa es manté **separada del tokenitzador** perquè la neteja és una transformació de *text a text*, mentre que la tokenització és de *text a estructura*. Separar-les manté el tokenitzador lliure d'estat multilínia, i fa que el fitxer intermedi resultant sigui llegible directament per una persona.

6.1.3 Representació del programa

Cada testimoni és un `Token { kind, lexeme, line, column }`. La **posició** s'hi adjunta des del primer moment i sobreviu fins a `IrEntry`, que conserva la línia i la columna d'origen. És això el que permet, més endavant, **il·luminar la línia** corresponent durant l'execució.

`InstructionKind` enumera cada mnemònic com una variant pròpia, agrupades per blocs: la base de 32 bits, l'extensió de 64, la de multiplicació i la de coma flotant [32] (apèndix A). El reconeixement es fa amb un `match` **exhaustiu**, sense cap expressió regular, pel mateix motiu que justifica el despatx per *enum* del cursor a l'apartat 5.4: el compilador garanteix que cap etapa posterior deixa un mnemònic sense tractar. No hi ha, en canvi, cap instrucció de les extensions A ni C.

El programa sencer és un `Program { nodes: Vec<Node> }`, amb un `Node` de **tres variants** i un `Operand` de **cinc**. Cada node porta també la seva pròpia posició, seguint el mateix criteri que el testimoni del qual prové.

6.1.4 Expansió de pseudoinstruccions i assignació d'adreces

L'eina admet **24 pseudoinstruccions** [32] (apèndix A). La justificació és pedagògica: l'usuari final escriu `li`, `mv` o `j` perquè és el que ha vist a classe, i obligar-lo a expandir-les a mà seria una fricció aliena al que l'eina vol ensenyar.

L'expansió té lloc **dins del mateix bucle** que assigna les adreces, a `layout::build()`. Cada instrucció real resultant rep la seva pròpia adreça, i el cursor avança quatre bytes per cadascuna.

Que l'expansió i l'assignació d'adreces vagin juntes importa perquè **una pseudoinstrucció pot expandir-se a més d'una instrucció real**, si l'expansió fos posterior, totes les adreces següents quedarien desplaçades. L'exemple més clar és `li` amb un

immediat que no cap en dotze bits amb signe: genera dues instruccions, amb la part alta i la baixa ajustades. `la` en genera sempre dues.

La línia i la columna del node original es copien **idèntiques** a totes les instruccions que en resulten, de manera que dues instruccions consecutives poden apuntar a la mateixa línia de codi font. És això el que fa que la **il·luminació de línia** durant l'execució es comporti tal com l'usuari espera.

Pel que fa a les directives, `.bss` es comporta exactament igual que `.data` i **no reserva zeros**, `.global` s'ignora completament, i `.space` i `.align` no existeixen. Es tracta d'un subconjunt *asimètric*, però suficient per als programes curts que l'eina espera rebre (apèndix A).

6.1.5 Resolució d'etiquetes

La resolució d'etiquetes es reparteix en **dos mecanismes** amb responsabilitats separades. La taula de símbols comprova, durant l'anàlisi, que tota referència resolgui a alguna etiqueta existent, l'assignació de les adreces reals, en canvi, no arriba fins a la generació de codi.

Per a les referències dins de dades cal **pedaçar-les en dues fases**, perquè quan es recorre l'arbre una referència pot apuntar a una etiqueta que encara no s'ha visitat.

Per a les referències dins d'instruccions es calcula el **desplaçament relatiu** al comptador de programa, amb una comprovació de rang explícita segons el format, i es genera un error si el salt no hi cap. Aquest és el motiu pel qual `IrEntry` ja no conté cap referència a etiqueta: totes han passat a ser *immediats*.

`CodegenOutput { layout, entries }` **travessa el pont de Tauri dues vegades**: surt cap a la interfície com a resultat de la compilació, i hi torna com a paràmetre en iniciar la sessió. La interfície no l'interpreta en cap moment, només el custodia, cosa que manté la compilació i la simulació com **dues operacions completament independents**.

6.1.6 Acumulació i presentació d'errors

Les cinc etapes s'executen **sempre**, sense aturar-se encara que una etapa anterior ja hagi reportat errors. La justificació és donar a l'usuari **tot el que està malament d'un sol cop**, i estalviar-li un cicle de correcció per cada error trobat.

Els errors dels quatre tipus possibles s'unifiquen en una **entrada comuna** i s'ordenen per línia i columna, de manera que la llista final segueix l'ordre del fitxer font i no l'ordre en què s'han detectat les etapes.

El format del missatge segueix l'estil de les *eines de línia d'ordres*. Connecta directament amb el requisit d'usabilitat de l'apartat 3.1.2: identifica el problema en termes que l'usuari entén i indica on es troba, que és exactament el que calia per a algú que potser escriu assemblador per primera vegada.

El que arriba a la interfície és només el **recompte i el text ja format**, no una estructura detallada. Estructurar-ho hauria significat un tipus més duplicat a banda i banda del pont, amb el cost de manteniment manual que ja s'ha descrit a l'apartat 5.5. Una llista estructurada seria, de fet, el pas previ necessari per oferir una navegació des de l'error fins a la línia corresponent, i queda apuntada com a *línia de treball futura*.

6.2 El simulador de CPU

El simulador de CPU és el component que **consumeix el resultat de la compilació** i el converteix en accessos concrets que alimenten el motor de memòria descrit a l'apartat 5.4. Cada instrucció que s'executa acaba, tard o d'hora, generant una petició que travessa el cursor definit en aquell mateix apartat, i és aquest punt de contacte el que lliga els dos motors del sistema. Aquesta secció descriu com es representa l'estat intern de la CPU, com es produeixen aquestes peticions, i com es gestiona el fet que una instrucció pugui quedar **a mig resoldre**.

6.2.1 Estat i descodificació

`RiscvCpuSimulator` guarda tot l'estat necessari per executar un programa: els registres enters, els registres de *coma flotant*, el comptador de programa, i el magatzem d'instruccions ja descodificades. També manté una referència a la jerarquia de memòria sobre la qual opera, que és cap a on acaben dirigint-se totes les seves peticions d'accés. S'hi afegeixen, finalment, **dos camps que representen una operació pendent**: una de dades i una de *fetch*. Aquests dos últims camps són els que fan possible aturar l'execució enmig d'una instrucció, i es descriuen en detall a l'apartat 6.2.3.

La descodificació és un únic `match` sobre el tipus d'instrucció, però la divisió que realment importa no és entre categories d'instrucció sinó entre **dues formes d'acabar la crida**. Unes instruccions s'executen i acaben dins de la mateixa crida, com és el cas de les aritmètiques o dels salts condicionals, que no necessiten cap resposta de la jerarquia de memòria. Unes altres, en canvi, **deixen una operació pendent** i retornen sense haver-se completat, a l'espera que un pas posterior del motor de memòria en resolgui l'accés. És aquesta segona categoria la que dona sentit als dos camps de pendència descrits més endavant.

El tractament de l'**amplada de paraula** és l'única concessió real que fa el simulador a la doble ISA de 32 i 64 bits que admet l'eina. Queda concentrat en tres funcions: una d'*emmascarament*, una d'*extensió de signe*, i una màscara de la quantitat de desplaçament, i totes tres despatxen segons la base activa en aquell moment. Aquesta concentració és deliberada: cap altra part del codi necessita saber quina variant de la ISA s'està executant.

Els registres de coma flotant **no són purament decoratius**, tot i que podrien haver-se implementat com un simple ornament per completar la ISA descrita a l'apartat 3.2.2. Les instruccions de càrrega i emmagatzematge dedicades hi escriuen directament, i les operacions aritmètiques tant de *precisió simple* com de *precisió doble* hi operen amb normalitat. Formen, per tant, un banc de registres tan actiu com el dels enters.

6.2.2 El magatzem d'instruccions i el *fetch* simulat

El magatzem d'instruccions guarda **totes les entrades ja descodificades**, indexades per la seva adreça. Quan arriba el moment d'executar una instrucció, el simulador la recupera directament d'aquest magatzem, i en cap moment no torna a llegir bytes de la jerarquia de memòria per tornar-la a descodificar. La descodificació, per tant, només passa **una vegada**, durant la fase de compilació.

La conseqüència d'aquesta decisió és que el *fetch* que travessa la jerarquia de memòria a cada instrucció **no serveix per obtenir-ne el contingut real**. Existeix únicament per generar les estadístiques d'accessos i per produir el moviment que les vistes de la interfície mostren a l'usuari. És una **simplificació deliberada**, i el codi la documenta explícitament allà on passa.

La justificació és doble. Per una banda, l'objectiu pedagògic de l'eina (apartat 3.1.1) és que l'usuari vegi el **cost d'accedir a les instruccions**, no que se li reprodueixi fidelment el procés de descodificació de bytes. Per l'altra, mantenir dues còpies independents del mateix codi introduiria un **problema de coherència** entre el codi carregat i el codi que realment s'executa, sense cap benefici a canvi.

6.2.3 Els dos camps de pendència

Els dos camps de pendència guarden **informació diferent** perquè resolen problemes diferents. L'operació de dades pendent porta el registre de destinació, la mida de l'accés i com s'han d'interpretar els bytes un cop arribin, ja sigui com a enter amb signe, sense signe, o com a coma flotant. El *fetch* pendent, en canvi, només porta la línia de codi font associada, ja que la instrucció que se n'obtindrà ja és coneguda d'entrada.

Que siguin dos camps i no un de sol respon al fet que la seva **finalització és diferent**. Una operació de dades acaba escrivint en un registre, mentre que un *fetch* acaba executant la instrucció que n'és el resultat. A cada pas del simulador, l'ordre de comprovació és sempre el mateix: primer es mira si hi ha una **operació de dades pendent**, després si hi ha un ***fetch* pendent**, i només si cap dels dos camps té res pendent es dona per completada la instrucció anterior i s'inicia un *fetch* nou.

6.3 El motor de memòria

El motor de memòria és qui rep les peticions que genera el simulador de CPU i les fa avançar pel cursor descrit a l'apartat 5.4. Aquesta secció baixa un nivell més i descriu com s'implementen realment les estructures que aquell cursor travessa: la **cau**, la **taula de pàgines**, els **marcs** i l'**emmagatzematge de dades**. Cada subapartat es correspon amb un dels components de la jerarquia.

6.3.1 Indexació i emmagatzematge dispers

El nombre de conjunts, l'etiqueta i l'índex de conjunt es calculen amb les **mateixes fórmules** per a totes les caus, independentment de si emmagatzemen dades de programa o entrades de taula de pàgines. Quan una entrada de taula de pàgines es busca a la cau que correspon, s'hi aplica exactament el mateix càlcul d'índex i etiqueta. Aquesta reutilització no és casual: és la mateixa idea que ja justificava, a l'apartat 2.4.3, que la TLB compartís els paràmetres d'una cau.

Els conjunts d'una cau **no es reserven tots per endavant**, sinó que viuen en un mapa amb *creació peresosa*. Una inserció crea el conjunt la primera vegada que s'hi escriu alguna cosa, mentre que una consulta de lectura mai no en crea cap: si un conjunt no s'ha tocat mai, simplement **no existeix** a l'estructura. Qualsevol consulta que hi vagi a parar falla directament.

La justificació d'aquesta creació peresosa és de **rendiment i de memòria**. Una cau configurable per l'usuari pot arribar a tenir una mida considerable, mentre que a la pràctica, amb els programes curts que fixa l'abast (apartat 3.2.2), només se n'accedeix a una petita porció. Reservar l'estructura sencera per endavant tindria un cost proporcional a una mida que **rarament s'omple del tot**.

6.3.2 Metadades temporals i polítiques

Cada línia de cau manté **quatre camps** de metadades temporals i de freqüència. Alguns només s'escriuen quan la línia s'insereix per primera vegada, d'altres també s'actualitzen en cada encert, ja que reflecteixen l'ús recent o la freqüència d'accés, i n'hi ha que una fallada, per definició, **no arriba mai a tocar**.

Cada política de reemplaçament consumeix un camp diferent d'aquests quatre. És per aquest motiu que s'han de mantenir **tots quatre alhora**, encara que una configuració concreta només en faci servir un: no se sap per endavant quina política triarà l'usuari a la interfície.

6.3.3 Determinisme: llavor i línies fantasma

El generador pseudoaleatori del motor és un **mesclador de bits** aplicat a la llavor de la sessió combinada amb una llista de coordenades pròpies de cada crida. El resultat depèn exclusivament d'aquestes dues coses: no hi ha cap estat global, ni cap dependència del rellotge del sistema. Donada la mateixa llavor i les mateixes coordenades, el resultat és **sempre idèntic**, per construcció.

Cada lloc de crida del generador utilitza un **primer element fix** que actua com a etiqueta pròpia d'aquell ús concret. Sense aquesta etiqueta, dos mecanismes diferents que fessin servir la mateixa llavor de sessió produirien exactament la mateixa seqüència de nombres, i acabarien correlacionats d'una manera que no respon a cap intenció de disseny.

Les **línies fantasma** són vies que el motor reporta com a ja ocupades i brutes, amb una etiqueta inventada pel generador, sense que cap inserció real les hagi tocat mai. **No s'emmagatzemen enlloc**: es recalculen a cada consulta a partir de les seves coordenades, de manera que un accés real i una consulta de visualització mai poden arribar a discrepar entre ells. En el moment que una inserció real ocupa finalment aquella via, la línia fantasma deixa d'existir com a tal **per sempre**.

Aquestes línies existeixen per **simular que la cau ja portava un temps en ús** abans que l'usuari comencés a observar-la. Sense elles, la primera expulsió de cada conjunt seria sempre la d'una línia neta, i l'usuari mai no arribaria a veure el cost real d'una escriptura diferida durant els primers accessos del programa.

La conseqüència que lliga aquest mecanisme amb l'abast del projecte és directa: la mateixa llavor dona sempre la **mateixa execució**. Sense aquesta garantia, la comparació entre configuracions que l'apartat 3.6 estableix com a condició de validesa dels resultats no seria justa, ni les proves automatitzades del motor serien reproduïbles.

6.3.4 El recorregut de la taula de pàgines

`WalkState` manté **tres camps** mentre dura el recorregut. La pàgina virtual que s'està traduint es manté fixa des del principi fins al final, el nivell actual de la taula decreix progressivament a mesura que el recorregut s'acosta a la fulla, i l'índex de nivell de cau indica quin nivell s'està provant en cada moment.

Que el nivell de cau sigui un camp de `WalkState` i **no una variable local** del recorregut respon al mateix motiu que justifica els camps de pendència del simulador de CPU (apartat 6.2.3). Cada consulta d'una entrada de taula és, ella mateixa, un accés a memòria que ha de poder-se aturar enmig, i en reprendre-la cal saber **exactament per quin nivell** s'havia quedat.

L'adreça física de cada taula intermèdia també s'assigna de manera **peresosa**: es reserva únicament la primera vegada que es necessita, seguint el mateix criteri de creació sota demanda que ja s'aplica als conjunts d'una cau. Aquesta coherència entre els dos mecanismes no és casual.

Entre taules consecutives s'insereix una **entrada de farciment** que trenca deliberadament l'alineació exacta. El problema que evita és subtil: sense aquest farciment, la mida de cada taula seria un múltiple exacte d'una potència de dos, i dues taules consecutives caurien sistemàticament al mateix conjunt de la cau. Els conflictes que en resultarien serien un **artefacte de la implementació**, no pas el comportament real que l'eina vol mostrar, i hi ha una prova específica que fixa aquest cas.

6.3.5 Marcs i intercanvi

La política d'assignació de marc lliure només tria el **punt de partida** de la cerca: seqüencial des de l'última posició assignada, o aleatori determinista fent servir el generador descrit a l'apartat 6.3.3. A partir d'aquell punt, totes dues alternatives avancen linealment fins a trobar-ne un que estigui lliure.

L'expulsió d'un marc, quan cap no està lliure, és sempre **LRU** i **no es pot configurar**. És una simplificació conscient, i cal declarar-ho obertament: la justificació és que l'usuari ja pot experimentar amb polítiques de reemplaçament a la cau, que és on el concepte s'entén millor.

`PendingFault` té un **cicle de vida de tres punts**. Es crea en el moment de resoldre la fulla de la taula de pàgines, es llegeix quan cal escriure la pàgina expulsada al disc, i es consumeix definitivament quan la pàgina sol·licitada s'acaba de llegir des del disc. Aquest cicle és el que permet, tal com ja s'establia a l'apartat 5.4, que el mateix accés original travessi tota la resolució sense que calgui rellançar-lo mai.

6.3.6 L'emmagatzematge de dades

L'emmagatzematge de dades és un **mapa dispers de blocs**, amb una mida de bloc configurable. Hi ha una distinció important entre dues formes d'accedir-hi: la *lectura passiva*, que retorna zeros sense crear res a l'estructura, i l'*accés real*, que pot arribar a materialitzar el bloc amb contingut pseudoaleatori determinista si la tirada del generador cau per sota del percentatge de brutícia configurat.

Aquesta distinció importa perquè una **consulta de visualització** no ha de modificar mai l'estat que s'està observant. És exactament el mateix principi que ja justificava el comportament de les línies fantasma descrites a l'apartat 6.3.3: observar mai no ha de tenir efectes secundaris sobre allò que s'observa.

La **mateixa estructura** serveix per a la memòria principal, per al disc, i pel contingut dels marcs físics, aquest últim amb blocs de la mida d'una paraula. Enllaça directament amb la decisió d'arquitectura, ja presentada a l'apartat 5.3, de reutilitzar la mateixa abstracció `DataStore` tres vegades sense compartir cap instància.

6.4 Execució contínua

L'execució contínua és la tercera de les tres granularitats que fixa l'abast (apartat 3.2.2), i és la que permet arribar a un estat concret o obtenir mètriques d'un programa sencer **sense fer clic a cada pas**. Aquesta secció descriu com s'implementa aquest mode sense bloquejar la interfície ni introduir inconsistències quan l'usuari l'atura i el torna a arrencar de pressa.

6.4.1 El fil i el comptador de generació

L'execució contínua s'implementa com un **fil del sistema operatiu**, amb un bucle bloquejant que espera entre iteracions successives. La mateixa funció serveix tant per arrencar l'execució des de zero com per reprendre-la després d'una pausa, l'única diferència entre els dos casos és **un únic paràmetre**.

El problema apareix quan l'usuari atura i torna a arrencar l'execució molt de pressa: un **fil antic pot seguir viu** durant un instant, competint pel mateix bloqueig sobre la sessió que el fil nou acaba de reclamar.

La solució és un **comptador de generació**. Cada bucle es queda amb el valor del comptador en arrencar, i a cada iteració comprova si encara és el seu, si no ho és, retorna i el fil *mor sol* sense necessitat de cap senyal extern. L'ordre importa: el comptador s'incrementa **abans** d'instal·lar la sessió nova, perquè si no un bucle antic podria arribar a agafar la sessió nova amb un comptador ja caducat.

Cal fer una nota honesta: el comptador **no es reinicia mai** mentre l'aplicació roman oberta. No suposa cap problema pràctic donat el rang del tipus que l'emmagatzema, però és una simplificació que val la pena declarar.

6.4.2 Velocitat en calent

La velocitat d'execució es llegeix de manera **fresca a cada iteració** del bucle, just abans de l'espera. El comandament que rep l'usuari des de la interfície es limita a

escriure aquest valor, no interactua amb el fil de cap altra manera.

La conseqüència és que un canvi de velocitat s'aplica ja a partir de la següent espera, **sense rellançar el fil** ni perdre cap part de l'estat de la simulació. És això el que fa que moure el control de velocitat mentre el programa s'executa sigui *immediat*.

6.4.3 Punts de ruptura

Els punts de ruptura es reben en iniciar la sessió i es guarden **dins de la sessió mateixa**, no a l'estat compartit de l'aplicació. Pertanyen al programa concret que s'està executant, i han de desaparèixer amb ell quan la sessió acaba.

A cada iteració, el bucle compara la línia font actual amb el conjunt de punts de ruptura. Si hi coincideix, marca l'execució com a **aturada**, emet l'esdeveniment de pausa indicant-ne el motiu, i el bucle acaba immediatament.

Hi ha un **cas subtil**: en reprendre l'execució des d'una línia que és, ella mateixa, un punt de ruptura, la comprovació ha d'estar *desactivada* mentre aquesta línia no canviï. Sense això, l'execució es tornaria a aturar immediatament sense avançar mai. Hi ha dues proves de regressió que fixen aquest comportament, i val la pena mencionar-ho perquè és exactament el tipus d'error que tendeix a reaparèixer.

6.4.4 Emissió d'esdeveniments en mode continu

El mode continu **no agrupa ni limita** la freqüència dels esdeveniments: el bucle emet exactament el mateix esdeveniment per cada pas atòmic que emetria el comandament manual equivalent. No hi ha, per tant, cap camí separat ni cap tractament especial per a aquest mode.

La justificació és mantenir una **única via de notificació** i garantir que el mode continu no amagui res que el mode manual mostraria. El preu és que, a velocitats molt baixes, el volum d'esdeveniments generat és proporcional al nombre de passos, cosa que l'abast ja acota en limitar la mida dels programes que l'eina espera rebre.

6.5 Estadístiques i mètriques

Les **mètriques** són el resultat visible de tot el que han descrit els apartats anteriors: cada accés que travessa el *cursor* deixa un rastre que acaba convertit en una xifra. Aquesta secció descriu com s'**acumulen** aquests rastres, com es **deriven** les mètriques finals, i com arriben a la interfície.

6.5.1 L'acumulador i la taxonomia de comptadors

L'**acumulador** guarda quatre coses: les *instruccions completades*, els *cicles totals*, un mapa indexat per compartiment, i els comptadors d'accessos al cap de la cadena, separats entre instruccions i dades.

La clau del mapa és un *enum* de **compartiment** (memòria de traduccions, cau amb índex de nivell i costat, memòria principal, disc), combinat amb un *enum* de costat de tres valors. Un nivell **Harvard** genera dues entrades separades que no es barregen mai, i hi ha una prova que ho fixa.

Aquesta taxonomia es prefereix a comptadors solts perquè el **nombre de nivells no es coneix** fins que l'usuari configura la jerarquia. El mapa amb *creació peresosa* és el que permet que el conjunt de comptadors s'adapti a qualsevol configuració sense necessitat de cap codi condicional.

Dins de cada compartiment, els **accessos** i els **cicles** s'anoten a cada pas, sense excepció. Els **encerts** i les **fallades**, en canvi, només s'anoten quan hi ha realment un resultat que classificar, ja que no tot pas de la jerarquia n'arriba a produir un.

6.5.2 Derivació de les mètriques

Les fórmules s'implementen tal com estan al codi. La que no és òbvia és la dels **cicles de memòria**: s'obtenen restant del total de cicles les instruccions completades, aprofitant que cada instrucció n'aporta exactament un de fix. Enllaça directament amb la decisió de *comptabilitat de cicles* presa a l'apartat 5.4.

El **temps mitjà d'accés** surt de dividir els cicles de memòria pels accessos al cap de la cadena, el **CPI**, dels cicles totals entre instruccions, i les **fallades per miler d'instruccions**, de la suma de fallades de tots els compartiments.

El càlcul es reparteix entre les dues capes. Aquestes tres mètriques es calculen al **motor**, però les **taxes d'encert i fallada** es calculen a la *interfície*, perquè el motor només exposa comptadors bruts. La justificació és que la taxa global necessita triar el denominador segons el costat del compartiment, cosa que és una decisió de presentació i no de simulació.

Cal remetre a les fórmules del marc teòric del treball i comprovar, una per una, que les que implementa el codi hi corresponen fidelment.

6.5.3 Presentació

La interfície té **quatre components de cel·la**, cadascun aportant una lectura diferent. El més rellevant és la **barra segmentada**, que permet excloure segments

amb caselles per veure el repartiment de cicles sense que un component dominant aixafi visualment la resta.

L'organització és per **pestanyes**, i el contingut de la de memòria principal canvia segons si la memòria virtual està activa. Amb ella, hi apareixen les *traduccions* i les *fallades de pàgina*, sense ella, només hi ha accessos i cicles.

Un detall revela una separació **incompleta**: la latència d'encert que es mostra no ve de les estadístiques, sinó directament del magatzem de configuració. El motiu és senzill: és un *paràmetre* que l'usuari ha fixat, no una mesura que calgui acumular.

El refresc envia una **instantània completa** a cada esdeveniment rellevant, mai de manera incremental. La justificació és que el conjunt és petit i de mida fixa, no proporcional a la memòria configurada, i per això és l'**excepció** al principi de finestres que ja es va establir a l'apartat 5.5.

6.6 Configuració de la jerarquia des de la interfície

Configurar la jerarquia és el primer que fa qualsevol usuari en obrir l'eina, i és també la part de la interfície amb **més decisions pròpies**, ja que no es dedueix directament de cap patró estàndard. Aquesta secció descriu com es representa la jerarquia a la pantalla, com es manipula, i com es valida allò que l'usuari hi construeix.

6.6.1 Les posicions com a model

Cada cau té associada una **posició**, i la seva mida és sempre dos elevat a aquesta posició. La jerarquia es modela, per tant, com un *eix de posicions* i no com una llista ordenada per índex.

Aquesta decisió permet **dues coses**. La primera és que l'ordre dels nivells es dedueix de comparar posicions, i la validesa és simplement que siguin estrictament creixents, en comptes de mantenir un índex a banda que es pugui desincronitzar. La segona és que els **dos costats d'un nivell Harvard** tenen posicions completament independents, de manera que poden tenir mides diferents sense cap restricció artificial que els lligui.

El que aporta *pedagògicament* és que veure els nivells col·locats sobre un eix de mides fa evident la relació entre posició a la jerarquia i capacitat, que és una de les coses que l'abast (apartat 3.3) vol que l'usuari interioritzi.

6.6.2 El control de l'eix

Visualment és un **eix horitzontal** amb una marca per cada posició possible i un marcador per cada cadena de cau. Un nivell *Von Neumann* aporta un únic marcador,

un de *Harvard* n'aporta dos, distingits per etiqueta i separats verticalment.

La col·locació dels marcadors és proporcional al valor de la posició i es resol amb càlcul **CSS**, no amb geometria calculada en JavaScript. És el que fa que redimensionar la finestra **no exigeixi recalcular res**.

La manipulació combina diversos mecanismes: **arrossegament** amb un llindar mínim per no confondre un clic amb un desplaçament, conversió de píxels a posicions per divisió i arrodoniment, i emissió del moviment *en viu* mentre s'arrossega. Les fletxes de teclat ofereixen una alternativa accessible, i un menú contextual permet convertir un nivell a Harvard o Von Neumann, o eliminar-lo.

El component pare és qui rep el moviment i el trasllada al **magatzem**, que és qui aplica l'acotament final. El control de l'eix **no muta res per si mateix**: només informa del gest de l'usuari.

6.6.3 Els formularis i els controls

El panell de configuració s'organitza en una jerarquia breu de components: un contenidor general, un formulari per nivell, i, dins de cada formulari, els controls específics de cada paràmetre.

El control de **potències de dos** és una entrada de només lectura amb una etiqueta formatada, acompanyada de dos botons que multipliquen o divideixen per dos amb acotament. La justificació és que fa **impossible per construcció** introduir un valor que no sigui potència de dos, una de les restriccions que l'abast identifica com a contingut pedagògic en si mateix.

Aquest control té una **limitació**, i cal dir-ho clar: quan s'arriba al límit, el botó deixa de fer efecte però no es desactiva visualment, de manera que l'usuari no sap que ha topat amb una frontera. És una millora d'usabilitat *pendent*, però ja identificada.

Els nivells **Harvard** es tracten amb dues pestanyes separades i un formulari muntat de nou cada vegada, en comptes de duplicar controls dins de la mateixa vista. Cada costat s'edita com una cau completament independent, i el formulari **no sap si l'altre costat existeix**.

6.6.4 Validació de configuracions

S'observen **quatre estratègies** davant d'una combinació invàlida: bloqueig silenciós en inserir sobre una posició ocupada, correcció per acotament en moure fora de rang, avís amb text en convertir a Von Neumann amb col·lisió, i confirmació prèvia en canviar d'ISA forçant solapament.

Cal declarar que aquest tractament **no és uniforme**, i que hauria de ser-ho: l'abast estableix que quan l'eina impedeix o ajusta una configuració ha de fer visible **per què** (apartat 3.3), perquè aquesta explicació és exactament l'objectiu pedagògic. Els dos casos silenciosos no ho compleixen.

La solució que s'apunta com a *treball futur* és un únic canal de retroalimentació de validació al panell, amb els textos centralitzats i traduïts, en comptes de cadenes literals disperses dins de cada component.

6.6.5 Persistència

Només es persisteix la **configuració de la jerarquia**, en emmagatzematge local del navegador. L'escriptura és **síncrona a cada mutació**, i la restauració en arrencar l'aplicació és completament silenciosa.

La justificació de persistir només això és que és el que costa més de construir i el que l'usuari vol conservar entre sessions, mentre que el codi font ja té el seu propi mecanisme de fitxer i l'**estat de simulació** no té sentit fora d'una execució concreta. Enllaça directament amb el requisit de *tornar a provar sense fricció* de l'apartat 3.5.1.

6.7 Visualització de l'estat en execució

Aquesta secció descriu com es tradueix tot el que passa al motor en alguna cosa que l'usuari pugui veure a la pantalla. El fil conductor és un **únic diagrama** que, com es veurà en aquest apartat, fa molt més que representar estàticament la jerarquia.

6.7.1 El diagrama com a peça central

La decisió que articula tot aquest apartat és que el **mateix diagrama** que serveix per configurar la jerarquia, descrit a l'apartat anterior, és també el que indica l'accés en curs i el que permet navegar entre vistes durant l'execució. No hi ha cap segon component ni cap capa superposada.

La justificació és que l'usuari ja construeix una **imatge mental** de la seva jerarquia mentre la configura, i reutilitzar exactament aquesta imatge durant l'execució és el que fa que el recorregut d'un accés es llegeixi sobre un dibuix que ja li és familiar. Un diagrama diferent l'obligaria a fer la correspondència *mentalment*.

Hi ha una **separació entre càlcul i pintat**: un mòdul calcula la disposició (caixes, connectors i camins SVG com a cadenes de text), i el component només pinta el resultat. És el que permet provar la geometria amb proves unitàries sense muntar

cap component, i encaixa amb el mateix repartiment de responsabilitats descrit a l'apartat 5.2.

6.7.2 Indicació de l'accés en curs

Hi ha **dos senyals simultanis**. Un missatge textual indica si s'està buscant una instrucció, llegint o escrivint, amb l'adreça en hexadecimal. I una marca de color sobre la caixa del nivell on s'ha mirat, diferent segons si el resultat ha estat un **encert** o una **fallada**.

Tota aquesta informació surt directament del *payload* de l'esdeveniment: ja porta el tipus d'accés, l'adreça, el resultat i, per a les caus, l'índex de nivell. No cal cap càlcul addicional a la interfície per obtenir-la.

La decisió que val la pena defensar és que la marca **no és un efecte temporitzat** que s'esvaeix sol: es queda a la pantalla fins que alguna cosa la substitueix. La justificació ve directament de l'abast: si l'usuari avança pas a pas i s'atura a mirar, un ressaltat que desapareix sol el deixaria sense la informació just quan la vol.

La neteja es fa en dos temps: en completar-se una instrucció es marca la **intenció d'esborrar**, i l'esborrat efectiu no passa fins que arriba l'esdeveniment següent, just abans d'aplicar-hi el nou estat. Així, la marca de l'últim accés d'una instrucció sobreviu fins que en comença una altra.

Amb esdeveniments molt ràpids no hi ha cua ni cap limitació: la transició CSS en curs simplement es **reapunta al nou valor**. La justificació és honesta: en mode manual és irrellevant perquè l'usuari mana el ritme, i en mode continu ràpid l'usuari no llegeix marques individuals sinó que observa el *patró* general.

Cal declarar una nota d'implementació: aquesta invocació **no passa pel magatzem**, sinó que el component arrel crida directament mètodes exposats del diagrama mitjançant una referència. És una excepció al patró de subscripció independent descrit a l'apartat 5.6, justificada perquè el diagrama és únic i la seva actualització no és una consulta de dades sinó una **ordre puntual**.

6.7.3 El diagrama com a navegació

Clicar una caixa del diagrama obre la vista de contingut corresponent, i és l'**única via** per canviar de vista. En arrencar i en aturar l'execució, la interfície torna sempre a la vista d'estadístiques.

La justificació és que reforça la idea de fons que les vistes són **finestres** sobre parts d'una mateixa jerarquia, i no pestanyes independents entre elles. També elimina la

necessitat d'una barra de navegació que hauria de reconstruir la mateixa informació que el diagrama ja mostra.

L'efecte pràctic a la interfície és que es munta una **única vista de contingut** alhora, cosa que té la conseqüència que només hi ha un subscriptor de contingut actiu en cada moment.

6.7.4 Finestres virtualitzades

Hi ha **tres peces** que treballen juntes: el càlcul pur del rang de files visibles, la poda del cau de files aprofitant l'ordre d'inserció d'un mapa natiu, i el *composable* que manté una finestra ancorada i la recentra ajustant el desplaçament quan s'acosta a un extrem, sense que l'usuari ho noti.

El recorregut complet és: desplaçament → recàlcul del rang → invocació amb posició i quantitat → resultat al mapa reactiu amb poda → pintat de només les files visibles. És la contrapartida al *front-end* del límit d'entrades per petició que ja s'imposava al controlador, descrit a l'apartat 5.5.

Davant d'un esdeveniment, el refresc torna a demanar només la **finestra visible**, però el cau de files no es buida mai. El raonament, ja present al propi codi, és que la configuració del nivell no canvia durant una execució.

6.7.5 Els límits del mecanisme compartit

Hi ha **dues vistes** que no fan servir el *composable* i que en reimplementen la lògica. No és duplicació per descuit: totes dues necessiten coses que la signatura del composable no admet.

La vista de cau necessita que la mida de finestra i el marge siguin **valors derivats**, que depenen de l'associativitat i de les paraules per línia i que només arriben de manera asíncrona, combinats amb un pressupost fix de píxels. El composable, en canvi, els vol com a nombres fixos des del principi.

La vista de memòria principal necessita una **posició inicial** abans del muntatge, calculada a partir de l'adreça de la secció de text del programa compilat, perquè l'usuari vegi el seu codi i no l'adreça zero. A més, opera sobre un espai d'adreces sense cap total finit natural, mentre que el composable sempre arrenca a zero i necessita un total significatiu.

La conclusió és que el composable resol bé el **cas comú**, i les dues excepcions estan justificades. Generalitzar-lo per admetre valors derivats i una posició inicial és, tot i això, la *línia de treball futura* més clara que deixa aquest apartat.

6.7.6 Navegació per la taula de pàgines

L'estat al *front-end* és una **pila de parells** de nivell i clau de pare: baixar hi afegeix un element utilitzant la clau de la fila triada, pujar en treu un, i anar a l'arrel la buida completament.

El *front-end* **mai reconstrueix l'arbre**: només recorda el camí que s'ha visitat. El filtratge de quines entrades pertanyen a la taula actual el fa el motor sobre el seu mapa pla, descrit a l'apartat 5.3.

Aquesta és la conseqüència directa de la decisió d'arquitectura de **no tenir nodes enllaçats** a la taula de pàgines. Demostra que aquella decisió no impedeix complir el requisit de **navegabilitat per nivells** que fixava l'abast.

6.8 Cicle de vida i coordinació a Vue

Aquesta secció tanca el capítol d'implementació descrivint com **Vue coordina** tot allò que s'ha explicat fins ara: quan es subscriu cada component, quan s'hi desubscriu, i com s'encadenen la compilació i l'execució en iniciar una sessió.

6.8.1 Subscripcions i cancel·lació

El patró és **uniforme** a tota la interfície: assignació de la subscripció en muntar-se i cancel·lació en desmuntar-se. El segueixen totes les vistes de contingut i també el component arrel.

El **magatzem** és un cas diferent: com que no és un component, no té cicle de vida propi de muntatge i desmuntatge. Per això retorna la funció de cancel·lació a qui l'invoca, perquè sigui aquell component qui se'n faci responsable.

Això importa especialment amb aquesta arquitectura: com que les vistes es munten i es desmunten cada vegada que es clica una caixa del diagrama, tal com es descrivia a l'apartat anterior, una subscripció **no cancel·lada** acumularia oients que continuarien demanant dades d'una vista que ja no és a la pantalla.

6.8.2 Arrencada i aturada d'una sessió

La seqüència en iniciar és fixa: es marca l'estat de compilació, es compila, i només si no hi ha errors i hi ha codi generat, es desen les mides compilades i s'arrenca la sessió amb la base, el codi generat, la configuració de la jerarquia i els punts de ruptura.

Que la compilació i la simulació siguin **dues operacions encadenades** i no una de sola és deliberat: permet que un error de compilació no deixi cap sessió a mig

crear, i fa que el resultat de la compilació sigui *reutilitzable* sense haver de tornar a compilar.

L'aturada aprofita el mateix mecanisme de cicle de vida: el panell sencer es **desmunta per condició**, cosa que dispara la cancel·lació de la vista activa sense cap codi de neteja explícit. En reiniciar es torna sempre a estadístiques, sense cap estat residual.

Cal fer una nota honesta i menor: un dels paràmetres que rep el magatzem en arrencar **no s'envia al motor**, sinó que només es guarda per a ús de la interfície.

7 Validació

Aquest capítol valida el simulador contra **dues xifres globals**: 776 casos de prova automàtics i set exercicis verificats manualment. Cap de les dues vies esgota la validació per si sola. El criteri que segueix tot el capítol és contrastar el comportament de l'eina contra **dos referents interns**: el *marc teòric*, que estableix què hauria de passar, i l'*abast*, que estableix quins requisits calia cobrir.

7.1 Estratègia de validació

7.1.1 Contra què es valida

Aquesta eina **no té cap referent extern** contra el qual contrastar xifres. L'abast estableix que els resultats no són predictius en termes absoluts i que les latències configurades no corresponen a cap maquinari real. Contrastar l'AMAT que dona l'eina amb el d'un simulador de recerca no demostraria res, perquè les dues xifres mesurarien sistemes diferents per construcció.

Hi ha **dos referents que sí que valen**. El primer és el *marc teòric*: el comportament observat ha de coincidir amb el que la teoria prediu per a cada mecanisme. El segon és l'*abast*: cada requisit ha de tenir una evidència que el cobreixi.

La conseqüència metodològica és que la validació d'aquest treball és de **comportament i no de xifres absolutes**. Això justifica el pes que tenen la verificació manual i els casos demostratius, per davant d'una comparació numèrica amb cap altre sistema.

7.1.2 Les quatre vies

Les vies de validació són **quatre**. Les *proves automàtiques del motor* comproven, mitjançant compilació condicional, que la lògica de simulació es comporta com s'espera sense cap component d'interfície. Les *proves automàtiques de la interfície* comproven, amb Vitest, el comportament dels components i de les funcions pures que en depenen. La *verificació manual* compara l'execució de l'eina amb un càlcul fet a mà a partir del marc teòric. I la *comprovació directa* constata que un requisit no funcional es compleix executant l'eina en les condicions que descriu.

Les quatre vies calen perquè **no se solapen**. Una prova unitària de la política de reemplaçament no detecta un accés de més al recorregut de la taula de pàgines. Cap prova automàtica demostra que es pugui canviar una configuració sense reiniciar l'aplicació. I cap prova demostra que el que es veu a la pantalla sigui comprensible per a l'usuari.

La taula 9 recull la correspondència entre els requisits i les vies que els cobreixen. Fa explícit que **cap requisit queda sense cap evidència** que el validi.

Requisit	**PM**	**PI**	**VM**	**CD**
Configuració de la jerarquia	●	●	●	
Granularitat d'execució	●	●	●	
Inspecció de l'estat		●	●	
Mètriques de rendiment	●		●	
Rendiment i interactivitat				●
Volum transferit acotat	●	●		
Usabilitat de la configuració		●	●	
Portabilitat				○
Mantenibilitat	●	●		

Taula 9: Correspondència entre els requisits i les vies de validació. Font: elaboració pròpia.

PM correspon a les proves automàtiques del motor, PI a les de la interfície, VM a la verificació manual, i CD a la comprovació directa. El símbol ○ a la fila de Portabilitat indica una verificació parcial: la comprovació cobreix dos dels tres sistemes d'escriptori, ja que no s'ha pogut provar macOS per manca de maquinari.

7.2 Proves automàtiques del motor

7.2.1 Volum i repartiment

El motor té **474 casos de prova** repartits en 32 fitxers, tots dins del mateix mòdul que proven mitjançant compilació condicional (apèndix C). Aquesta organització és la que fa efectiva la propietat que el model és **verificable sense passar pels controladors**, requisit que estableix l'apartat 3.5.4.

La taula 10 mostra el repartiment per subsistema. La jerarquia de memòria i la CPU concentren **més de la meitat dels casos**, i és exactament on viu la lògica que no es pot verificar a ull.

Subsistema	Unitats principals	Casos
Jerarquia de memòria	`hierarchy`, `cache`, `page_table`, `tlb`, `backing_store`, `frames`	168
CPU i sessió	`cpu`, `stats`, `session`	122
Compilador	`expand`, `encode`, `checks`, `grammar`, `tokenizer`, `layout`	144
Controladors	`simulation_session`, `memory_contents`	40
Total		**474**

Taula 10: Repartiment dels casos de prova del motor. Font: elaboració pròpia.

El detall per fitxer es troba a l'apèndix C. Els controladors mostren una **asimetria clara**. Els de sessió i de contingut tenen desenes de casos, mentre que els d'edició i d'estadístiques en tenen un de sol cadascun. La raó és que aquests dos són gairebé *delegació pura*, i el que fan de debò ja està provat al model.

7.2.2 Categories de prova

Es distingeixen **quatre categories** de prova. Les *unitàries* aïllen un mecanisme concret, com `lru_picks_the_candidate_with_the_smallest_last_used`, que construeix tres candidats a mà i comprova quin tria la política. Les de *recorregut complet* segueixen una seqüència sencera, com `a_multi_level_page_table_walks_once_per_level_on_a_full_miss`. Les de *cas límit* comproven configuracions extremes, com `stats_report_memory_accesses_even_with_no_cache_levels_and_vm_off`, amb una jerarquia buida. I les de *regressió* fixen un error que va existir de veritat, com les dues proves sobre reprendre l'execució des d'un punt de ruptura.

La categoria de recorregut complet és la que **valida directament el marc teòric**, perquè comprova seqüències senceres d'estats i no funcions soltes. Una funció aïllada pot fer bé el seu pas i, tot i així, encadenar-se malament amb la resta.

Els casos límit demostren la **robustesa de la configuració**. Que hi hagi una prova amb una jerarquia sense cap nivell de cau és la contrapartida directa del requisit que fixa que el nombre de nivells no estigui acotat.

7.2.3 Les proves com a especificació

Els noms de les proves descriuen el **comportament esperat en llenguatge natural**, i les assercions porten un missatge explicatiu. Una fallada no diu només que

un valor no coincideix, diu *què s'ha trencat conceptualment*. Els noms `evicting_a_dirty_frame_writes_it_back_before_swapping_in_the_new_page` i `a_harvard_levels_two_sides_land_in_two_separate_buckets` són exemples d'aquest estil.

`a_multi_level_page_table_walks_once_per_level_on_a_full_miss` comprova que, amb **tres nivells de taula**, un accés en fred arriba a RAM **quatre vegades**: una per cada nivell de la taula més una per la dada. Aquest recompte és exactament el que prediu el marc teòric per a una traducció multinivell sense res a la TLB ni a cap cau.

`a_fetch_walks_the_same_way_a_load_does_under_vm` comprova que la seqüència de nivells d'una cerca d'instrucció és **idèntica** a la d'una càrrega. El missatge de l'asserció diu que el *fetch* no necessita cap tractament especial perquè comparteix tots els passos amb la càrrega. El valor d'aquesta prova és que comprova una **equivalència** entre dues seqüències, no un valor aïllat, i demostra que la decisió de disseny de no tractar la cerca d'instrucció com un cas especial es compleix.

7.2.4 Proves de regressió

Les proves de regressió d'aquest projecte documenten **errors que van existir de veritat**, no escenaris hipotètics afegits per precaució. Cadascuna fixa un comportament concret que en algun moment va fallar.

Les dues proves sobre el punt de ruptura fixen un error concret: en reprendre l'execució des d'una línia que és, ella mateixa, un punt de ruptura, l'execució es tornava a aturar **immediatament sense avançar**. Sense aquestes proves, l'error reapareixeria fàcilment perquè la comprovació del punt de ruptura i la lògica de represa viuen en llocs diferents del codi, i és fàcil tocar l'una sense recordar l'altra.

La prova sobre el farciment entre taules de pàgines fixa que, sense aquesta entrada, dues taules consecutives cauen **sistemàticament al mateix conjunt** de la cau. Els conflictes que en resultarien serien un *artefacte de la implementació*, no el comportament que l'eina vol mostrar a l'usuari.

7.2.5 Cobertura

La cobertura de línies del motor és del **80,0 %** (2492 de 3116), mesurada amb *cargo-tarpaulin* [33]. La taula 11 en desglossa el repartiment per subsistema.

Subsistema	Línies	Cobertura	Observació
Jerarquia de memòria	700/712	98,3 %	—
CPU i sessió	606/695	87,2 %	Instruccions poc habituals
Compilador	1029/1376	74,8 %	Camins d'error per instrucció
Controladors	157/299	52,5 %	Requereixen entorn d'aplicació
Arrencada	0/34	0,0 %	Cablejat, sense lògica
Total	**2492/3116**	**80,0 %**	

Taula 11: Cobertura de línies del motor per subsistema. Font: elaboració pròpia.

Nou unitats arriben al 100 % de cobertura: `expand`, `preprocessor`, `token`, `riscv/mod`, `error`, `symbol_table`, `backing_store`, `frames` i `seeded_random`. Inclouen precisament les que implementen les **decisions més delicades** del projecte: l'expansió de pseudoinstruccions, la generació determinista, i la gestió de marcs i d'emmagatzematge.

Les zones amb cobertura més baixa segueixen un **patró clar**. El compilador baixa al 74,8 % perquè hi ha camins d'error per a cada tipus d'instrucció que no val la pena cobrir un per un. La CPU i la sessió es queden al 87,2 % per instruccions poc habituals. Els controladors baixen al 52,5 % perquè requereixen un entorn d'aplicació que les proves unitàries del motor no reprodueixen. I l'arrencada es queda al 0 % perquè és *cablejat* sense cap lògica que provar.

La conclusió és senzilla: la cobertura és **alta on hi ha lògica** i **baixa on hi ha enumeració de casos** o dependència de l'entorn. Aquest segon grup queda cobert per les altres vies de validació d'aquest capítol.

7.3 Proves automàtiques de la interfície

7.3.1 Volum i repartiment

La interfície té **302 casos de prova** en 42 fitxers, escrits amb Vitest [29], amb una cobertura de línies del **85,5 %** (apèndix C).

Els tres fitxers amb més casos, `diagram` amb 21, `derive` amb 21 i `operations` amb 20, són **funcions pures i no components**. Això és desitjable: és la conseqüència directa de la separació entre càlcul i pintat descrita al capítol d'implementació, i la prova que aquella decisió tenia el benefici que anunciava.

7.3.2 El que es prova

Hi ha **tres nivells** de prova. Les funcions pures es proven sense muntar res. Els components es munten amb l'API del sistema simulada. I les integracions lleugeres combinen l'editor, el magatzem i les vistes per comprovar-ne la coordinació.

Les proves de validació de configuració cobreixen el requisit d'**usabilitat**. Hi ha proves d'acotament per sobre i per sota al control de potències de dos, de l'associativitat acotada al nombre d'entrades, de la mida de bloc que no pot baixar de la mida de paraula de la ISA, i **cinc proves** d'acotament de marcadors amb detecció de col·lisió. Totes juntes són l'evidència que les relacions entre paràmetres que l'abast identifica com a contingut pedagògic estan realment fixades al codi.

El límit d'entrades per petició, `MAX_COUNT` igual a **512**, l'apliquen sis funcions del controlador de contingut, però **només tres** tenen una prova explícita que en comprovi l'excés: `cache_sets`, `tlb_sets` i `page_table_entries`. L'asimetria és real i queda declarada aquí.

7.3.3 Limitacions de la infraestructura

`npm run test` dona **302 passed** i 0 failed, amb **deu rebutjos no gestionats**. La causa és única: una simulació incompleta de l'API de finestra de Tauri als fitxers que munten l'editor. Cap prova falla per aquest motiu.

Es tracta de *soroll d'infraestructura* amb una solució coneguda: completar la simulació de l'API de Tauri en aquests fitxers eliminaria els deu avisos sense canviar el resultat de cap prova.

7.4 Verificació manual

Amb 776 proves automàtiques ja executades, aquesta via segueix sent necessària perquè les proves comproven que el codi fa el que el programador va decidir que fes, **no que aquella decisió coincideixi amb la teoria**. Un error de concepte passa totes les proves igual de bé que un encert.

7.4.1 Procediment

El protocol té **quatre passos**. Es parteix d'un exercici amb un resultat predictible segons el marc teòric. Es tradueix a una configuració concreta i a un programa. S'executa amb la granularitat adequada per poder-ne seguir cada pas. I es contrasta el resultat amb la predicció.

El contrast és **doble**. Cada exercici es compara amb el càlcul manual i també amb l'arnès de proves. L'arnès crida el motor directament, mentre que la interfície hi

arriba a través dels controladors i del pont de Tauri. Que els dos coincideixin valida també aquesta capa intermèdia.

Hi ha **tres condicions prèvies**. La llavor es fixa a 42 perquè l'execució sigui reproduïble. El percentatge de brutícia es fixa a zero, tant a la cau com a la RAM, perquè no apareguin línies fantasma que farien impossible el càlcul manual. I els nivells són Harvard perquè les cerques d'instrucció no contaminin el recompte de dades.

L'avanç per **acció atòmica** i per **instrucció** és el que fa possible aquesta verificació. Permet comparar estat contra estat i no només el resultat final, de manera que una discrepància es localitza en el pas exacte on apareix. Aquí es tanca el cercle amb el requisit de granularitat d'execució de l'apartat 3.2.2.

7.4.2 Els exercicis de jerarquia de memòria cau

La taula 12 resumeix els set exercicis: el mecanisme que verifica cadascun i la configuració que fa servir. La taula 13 en presenta els resultats: accessos, encerts i fallades del compartiment de dades, i els cicles totals obtinguts des de la interfície i des de l'arnès.

Ex.	**Mecanisme verificat**	**Configuració**
E1	Indexació i localitat espacial	1 nivell Harvard, 2^6, correspondència directa
E2	Fallades de conflicte	1 nivell Harvard, 2^4, 1 i 2 vies
E3	Polítiques de reemplaçament	1 nivell Harvard, 2^1, 2 vies, un sol conjunt
E4	Polítiques d'escriptura	1 nivell Harvard, 2^2, correspondència directa
E5	Derivació de mètriques	2 nivells Harvard, 2^6 i 2^8
E6	Traducció d'adreces	MV activa, TLB de 2 entrades, 2 nivells de taula
E7	*Thrashing* per pressió de marcs	Igual que E6 amb 2 marcs

Taula 12: Resum dels exercicis de verificació manual. Font: elaboració pròpia.

Els programes complets es troben a l'apèndix B.

Ex.	Variant	Dades (L1d)			Cicles		
		Acc.	Enc.	Fall.	Interfície	Arnès	Δ
E1	Blocs de 16	64	48	16	3 674	2 705	969
E1	Blocs de 32 *	64	48	16	3 574	1 805	—
E2	Corresp. directa	40	0	40	4 904	4 532	372
E2	2 vies, LRU	40	38	2	1 104	732	372
E3	LRU	8	4	4	1 712	1 632	80
E3	FIFO	8	5	3	1 612	1 532	80
E3	LFU	8	3	5	1 812	1 732	80
E3	Aleatòria	8	3	5	1 812	—	—
E4	Immediata	20	18	2	3 270	3 010	260
E4	Diferida + res.	20	18	2	1 470	1 210	260
E4	Diferida sense res.	20	0	20	3 270	3 010	260
E5	Dos nivells	64	48	16	2 562	2 562	0

Taula 13: Resultats dels exercicis amb jerarquia de memòria cau. Font: elaboració pròpia.

La diferència de cicles entre interfície i arnès correspon en tots els casos a la variació de la latència d'encert multiplicada pel nombre total d'accessos al nivell.

E1 recorre seqüencialment un vector de 64 paraules. El patró és **una fallada per cada bloc nou** i encerts la resta d'accessos dins d'aquell bloc, que és exactament el comportament que demostra la *localitat espacial*.

E2 fa quaranta accessos alternant entre dues adreces separades exactament per la mida de la cau. Amb correspondència directa **fallen els quaranta accessos**. Amb dues vies **només fallen els dos primers**. Aquest és el cas de conflicte que descriu el marc teòric, i la vista de cau mostra els dos blocs ocupant el mateix conjunt mentre la resta de conjunts resten buits.

E3 aplica la seqüència A, B, A, A, C, B, C, C sobre tres adreces contigües. La taula 14 recull el comportament de les quatre polítiques de reemplaçament sobre aquesta mateixa seqüència. Les quatre donen **tres recomptes diferents** de fallades.

Política	Enc.	Fall.	Cicles	Expulsa	Comportament observat
LRU	4	4	1 712	B	Rotació entre les tres adreces
FIFO	5	3	1 612	A	Conserva l'última inserida
LFU	3	5	1 812	B	Reté A de forma permanent
Aleatòria	3	5	1 812	B	Reté A amb la llavor 42

Taula 14: Comportament de les polítiques de reemplaçament. Font: elaboració pròpia.

La columna *Expulsa* indica quin bloc surt del conjunt en accedir a C. El que aporta la inspecció de l'estat és veure **què expulsa** cada política i no només quantes fallades produeix. LFU reté A de manera permanent perquè arriba a una freqüència de 3, mentre que LRU i FIFO el fan rotar amb la resta d'adreces. Aquesta diferència, entre veure quin bloc surt i només comptar quants en surten, és la diferència entre una *vista de dades* i una *explicació* que estableix l'abast.

E4 escriu deu vegades sobre dues posicions mai llegides abans, amb tres variants de política d'escriptura. L'escriptura immediata i l'escriptura diferida sense reserva d'escriptura donen **exactament els mateixos cicles**, però una composició oposada del compartiment de dades: divuit encerts en un cas i cap en l'altre. Amb reserva d'escriptura, la línia s'insereix a la primera escriptura i els encerts posteriors baixen igualment a RAM per l'escriptura immediata. Sense reserva, no s'insereix mai res i totes les escriptures baixen a RAM directament. El **cost final és el mateix**, però el camí que hi porta és diferent.

E5 repeteix el programa d'E1 sobre dos nivells Harvard. És l'**únic exercici** on no es va canviar cap latència respecte de l'arnès, i per això coincideix amb ell fins i tot en l'AMAT, el CPI i l'MPKI. És l'exercici que contrasta directament les fórmules de 2.5 amb les xifres que produeix l'eina.

7.4.3 La coherència de la comptabilitat de cicles

Les latències d'encert es van canviar respecte de l'arnès per fer visibles les diferències entre variants a la interfície. Els comptadors d'accessos, encerts i fallades **coincideixen exactament** en tots els casos comparables.

La diferència de cicles entre interfície i arnès s'explica sempre com la **variació de latència multiplicada pel nombre total d'accessos** al nivell, sense cap residu inexplicat. Aquesta és una verificació de la comptabilitat de cicles que no estava planificada i que va sorgir del mateix procés de comparar els dos entorns.

7.4.4 Traducció d'adreces

E6 fa sis lectures sobre tres pàgines diferents amb repetició, amb una TLB de dues entrades totalment associativa amb FIFO. S'observen divuit accessos a la TLB amb **nou encerts i nou fallades**, que coincideix amb la predicció manual.

El motiu del patró és el resultat pedagògic de l'exercici. Amb només dues entrades a la TLB, els accessos a les dades **expulsen la traducció de la pàgina de codi**, i la cerca de la instrucció següent la torna a fallar. Dimensionar la TLB sense comptar el codi és un error.

La navegació per la taula de pàgines de dos nivells mostra les entrades creant-se **incrementalment** a mesura que avança l'exercici, cosa que verifica el model d'arbre implícit descrit al capítol d'arquitectura.

Hi ha **activitat de disc encara que no hi hagi cap expulsió**, perquè el primer accés a una pàgina és una fallada de pàgina i la pàgina es llegeix del disc. Val la pena distingir amb claredat una *fallada de pàgina* d'una *expulsió*: la primera passa sempre que la pàgina no és present, la segona només quan cal fer lloc per a una altra.

7.4.5 Thrashing per pressió de marcs

E7 pren la configuració d'E6 i hi redueix els marcs a dos, sobre el mateix programa d'E4. Reutilitzar un programa ja verificat sota una configuració nova és **deliberat**: se'n coneix exactament el comportament de partida, i qualsevol diferència es pot atribuir a la pressió de marcs.

El marc 0 conté la pàgina de codi i **no es mou mai**. El marc 1 alterna entre la pàgina de dades, amb el bit de brutícia actiu, i la segona pàgina de codi, neta. En total es produeixen **vint expulsions**.

La taula 15 recull tres verificacions creuades. Els passos a RAM es calculen com vint-i-un accessos multiplicats per dos més seixanta-cinc, i donen 107, coincidint amb l'eina. Els accessos a disc es calculen com vint-i-un més deu, i donen 31, coincidint també. I els cicles totals, calculats a partir de les latències i els comptadors de cada compartiment, donen 315.720, la mateixa xifra que reporta l'eina.

Magnitud	Composició	Càlcul	Eina
Passos a RAM	$21 \times 2 + 65$	107	107
Accessos a disc	$21 + 10$	31	31
Cicles totals	$65{\cdot}5 + 107{\cdot}50 + 31{\cdot}10^4 + 45$	315 720	315 720

Taula 15: Verificacions creuades de l'exercici E7. Font: elaboració pròpia.

Els accessos a disc corresponen a 21 lectures per fallada de pàgina més 10 escriptures d'expulsió bruta; les nou expulsions restants eren de pàgines netes.

La segona verificació és la que **importa més**. Si totes les expulsions haguessin estat brutes, els accessos a disc haurien estat 42: 21 lectures per fallada de pàgina més 21 escriptures d'expulsió. La xifra real és 31, perquè només deu de les vint expulsions eren brutes; les altres nou eren de pàgines netes que no calia escriure. Aquesta diferència és exactament el **mecanisme del bit de brutícia**, i el número el verifica.

El resultat pedagògic és clar: la **taxa de fallada de pàgina és del 100 %**, el 98,2 % dels cicles es consumeixen al disc, i quaranta-cinc instruccions costen més de tres-cents mil cicles. La pressió real prové de la **competència entre el codi i la seva pròpia dada** per ocupar l'únic marc lliure, un fenomen que un estudiant no espera trobar-se.

7.4.6 Límits del mètode

El mètode és **manual**, no es pot repetir automàticament, i només cobreix configuracions prou petites per calcular-les a mà, que són precisament les que menys tensionen el motor.

La conclusió és que les dues vies, automàtica i manual, són **complementàries**, i cap de les dues sola seria suficient per validar el simulador.

7.5 Verificació dels requisits no funcionals

7.5.1 Portabilitat

L'execució s'ha comprovat a **Windows i a Linux**. No s'ha pogut comprovar a **macOS** per manca de maquinari, de manera que el requisit de portabilitat queda **parcialment verificat**.

És raonable esperar que l'eina també funcioni a macOS. El marc d'aplicació té aquest sistema entre els objectius de compilació configurats, i no hi ha cap dependència ni

cap fragment de codi específic de plataforma al projecte. Cal deixar clar que això és un **argument i no una comprovació**.

L'eina depèn del motor de renderització web del sistema en comptes d'empaquetar-ne un de propi, decisió ja justificada al capítol d'eines i tecnologies. És precisament aquesta dependència la que fa que la comprovació a macOS **no sigui prescindible**: el motor de renderització d'aquella plataforma és diferent del de Windows i Linux, i el comportament de la interfície en depèn.

7.5.2 Interactivitat

L'argument de compliment és de **construcció**, amb tres motius. El volum per petició està acotat a 512 entrades i comprovat amb proves. El refresc davant d'un esdeveniment demana només la finestra visible en cada moment. I el cost d'un pas de simulació depèn del nombre de passos que calgui fer, no de la mida de les estructures que l'usuari hagi configurat.

Un argument de construcció **no substitueix una mesura real**. La instrumentació de temps per pas queda com a treball futur, amb una forma concreta: enregistrar el temps que triga cada pas de simulació i comprovar-ne la distribució sota les configuracions més grans que admet l'eina.

7.5.3 Volum transferit

El límit d'entrades per petició, `MAX_COUNT` igual a 512, s'aplica a **sis funcions** del controlador de contingut. Només **tres**, `cache_sets`, `tlb_sets` i `page_table_entries`, tenen una prova explícita que en comprovi l'excés, la mateixa asimetria ja declarada a l'apartat 7.3.2.

7.6 Casos d'ús demostratius

Els casos que segueixen **no són proves en sentit estricte**. Demostren que l'eina fa visible allò per a què va ser construïda. Cadascun aïlla un fenomen concret del marc teòric, i tots quatre reutilitzen configuracions ja verificades a l'apartat 7.4.

7.6.1 Fallades de conflicte (E2)

Aquest cas mostra una **fallada de conflicte**. Aquest tipus de fallada depèn de com es reparteixen els blocs entre els conjunts de la cau, no de la mida de les dades ni de la capacitat total. Importa perquè és exactament el tipus de fallada que la classificació clàssica en tres categories distingeix de les altres dues, classificació que aquesta eina no calcula directament.

La configuració és E2: correspondència directa, amb dues adreces separades exactament per la mida de la cau. Els quaranta accessos fallen tots. En passar a dues vies, **només fallen els dos primers**. La vista de cau mostra els dos blocs ocupant el mateix conjunt mentre la resta de conjunts resten completament buits.

L'abast estableix que l'usuari pot arribar a la causa d'una fallada observant l'estat de la jerarquia, sense que l'eina la classifiqui en les tres categories habituals. Aquest cas és la **demostració directa** d'aquella afirmació: veure els dos blocs compartint un únic conjunt, envoltats de conjunts buits, és identificar una fallada de conflicte sense que cap etiqueta ho digui.

7.6.2 Divergència entre polítiques de reemplaçament (E3)

Aquest cas mostra que **polítiques de reemplaçament diferents**, aplicades a la mateixa seqüència d'accessos, produeixen resultats diferents. Importa perquè la tria de política sol presentar-se com un detall d'implementació, quan en realitat determina directament quantes fallades produeix un programa.

La configuració és E3, amb la seqüència A, B, A, A, C, B, C, C sobre tres adreces contigües i un sol conjunt de dues vies. LRU produeix quatre fallades, FIFO tres, i LFU i l'aleatòria cinc cadascuna.

El valor d'aquest cas és veure **què expulsa cada política** en cada pas, no només el recompte final de fallades. LFU reté l'adreça A de manera permanent perquè hi arriba una freqüència de tres, mentre que LRU i FIFO la fan rotar amb la resta. Aquesta diferència de comportament, visible pas a pas, és la que dona sentit a triar una política per sobre d'una altra.

7.6.3 Polítiques d'escriptura (E4)

Aquest cas mostra que dues polítiques d'escriptura poden arribar al **mateix cost en cicles** per camins completament diferents. Importa perquè una xifra agregada, com el total de cicles, pot amagar diferències de comportament que només es veuen mirant els comptadors interns.

La configuració és E4, amb deu escriptures sobre dues posicions mai llegides abans. L'escriptura immediata i l'escriptura diferida sense reserva d'escriptura donen exactament els mateixos cicles. Els comptadors de dades, però, són **oposats**: divuit encerts en un cas i cap en l'altre.

Aquest és l'exemple clar que una xifra agregada pot amagar una diferència real de comportament. Amb reserva d'escriptura, la línia s'insereix a la primera escriptura i els encerts posteriors baixen igualment a RAM per l'efecte de l'escriptura immediata. Sense reserva, no s'insereix mai res i totes les escriptures baixen a RAM directament.

El cost final coincideix, però el **camí que hi porta és diferent**, i només es veu inspeccionant l'estat de la cau accés per accés.

7.6.4 Thrashing per pressió de marcs (E7)

Aquest cas mostra el fenomen de *thrashing*: un programa que, per manca de marcs físics, passa la major part del temps intercanviant pàgines en comptes d'executar instruccions. Importa perquè és el cas on la memòria virtual **deixa de ser transparent** i el seu cost es fa dominant.

La configuració és E7, amb només dos marcs disponibles. El marc 0 conté la pàgina de codi i no es mou mai. El marc 1 alterna entre la pàgina de dades i la segona pàgina de codi, amb vint expulsions en total. La taxa de fallada de pàgina arriba al **100 %**, i el **98,2 %** dels cicles es consumeixen al disc.

La lliçó és sobre **dimensionament de memòria física**. Un estudiant sol pensar la pressió de marcs només en termes de les dades d'un programa, però el codi també ocupa marcs i també competeix per ells. Quan els marcs disponibles no basten ni per al codi ni per a les dades alhora, el resultat és exactament aquest: quaranta-cinc instruccions que costen més de tres-cents mil cicles.

7.7 Limitacions de la validació

La limitació **més gran** és que no hi ha proves amb usuaris. El valor pedagògic de l'eina està argumentat des dels requisits i demostrat amb els casos d'ús de l'apartat 7.6, però no s'ha mesurat amb l'usuari final. La forma concreta que queda com a treball futur és una sessió amb estudiants de l'assignatura corresponent, amb un qüestionari abans i després de fer-la servir.

No hi ha comparació amb altres simuladors. Com ja s'argumenta a l'apartat 7.1.1, això **no és una mancança**: és una conseqüència directa del disseny, perquè contrastar xifres amb un altre simulador no aportaria cap evidència vàlida donades les limitacions ja declarades a l'abast.

La **portabilitat** només s'ha comprovat a dos dels tres sistemes d'escriptori.

La verificació manual cobreix la jerarquia de cau i el camí de traducció d'adreces. L'**expulsió de marcs** queda coberta per l'exercici E7 i per les proves automàtiques del motor.

La cobertura de proves automàtiques és **desigual**, amb les zones baixes ja identificades a l'apartat 7.2.5.

8 Ús d'eines d'Intel·ligència Artificial

8.1 Eines utilitzades

L'única eina d'intel·ligència artificial emprada durant el projecte ha estat **Claude**, d'Anthropic.

8.2 Finalitat i abast de l'ús

L'ús s'ha concentrat en cinc àmbits.

El primer és el **suport durant el desenvolupament de la interfície**. Quan em trobava amb alguna cosa que no sabia com resoldre, demanava orientació sobre l'enfocament i, a partir d'aquella indicació, continuava la implementació pel meu compte. La consulta servia per desencallar una decisió, i el codi resultant l'escrivia i l'integrava jo.

El segon és la **consulta sobre el llenguatge**. Al principi del projecte no tenia prou pràctica amb Rust, i preguntava dubtes concrets de sintaxi o d'ús de la biblioteca estàndard. Aquestes consultes van funcionar com un substitut de cercar dins la documentació oficial quan es tractava de detalls puntuals que no recordava. La documentació oficial ha estat igualment una font constant al llarg de tot el desenvolupament.

El tercer, i el de més pes, és la **redacció de les proves unitàries**. El procediment ha estat sempre el mateix. En acabar un apartat de la interfície o del motor, explicava què acabava de construir i indicava quines proves volia. L'eina proposava una llista de casos, i jo la revisava abans d'acceptar-la: hi afegia proves quan veia parts sense cobrir i en retirava quan no aportaven res, de manera que la decisió de què s'havia de provar ha estat meva en tots els casos. Aquesta divisió m'ha permès *paral·lelitzar* la feina. Escriure proves unitàries és una tasca mecànica que no m'ensenyava res de nou sobre el projecte, i delegar-ne la redacció em deixava continuar desenvolupant mentre es preparaven.

El quart són els **programes d'exemple en assemblador RISC-V** que apareixen a la validació i als casos d'ús. Escriure'ls a mà no aportava res al treball, perquè el que importa de cada programa és el patró d'accés que provoca, i aquest patró el decidia jo abans de demanar-ne la redacció. La comprovació posterior sí que va caldre, perquè l'eina no admet el llenguatge sencer. No hi ha macros ni inclusió de fitxers, i el conjunt d'instruccions i directives admeses és el descrit al capítol d'implementació, de manera que diversos programes es van haver de simplificar o reescriure perquè el compilador els acceptés.

Els set exercicis de verificació manual segueixen aquest mateix repartiment. El disseny de cada exercici és meu: quin mecanisme havia de quedar aïllat, quina configuració ho permetia i quin patró d'accés calia per provocar-lo. L'assistència va cobrir la redacció dels programes i la construcció de l'arnès que els executa i en registra els resultats. El contrast d'aquests resultats amb el càlcul manual s'explica a l'apartat següent.

El cinquè és la **revisió del document**. He fet servir l'eina com a corrector ortogràfic i gramatical, funció equivalent a la de qualsevol corrector convencional, i per aplicar la negreta i la cursiva que marquen els conceptes rellevants de cada paràgraf. Aquesta segona part és una decisió estètica destinada a fer la lectura més fluida. El contingut, l'estructura i els arguments de la memòria són propis.

8.3 Verificació dels resultats obtinguts

La verificació del codi és immediata. La implementació la feia jo a partir de l'orientació rebuda, cosa que exigia entendre-la abans d'escriure-la, i un enfocament que no encaixés amb l'arquitectura del projecte quedava descartat en el moment d'intentar aplicar-lo. Pel que fa a les consultes sobre Rust, el compilador actua com a verificador directe, i els punts que afectaven decisions de disseny es van contrastar amb la documentació oficial.

Les proves unitàries s'han verificat en dues passes. La primera és la revisió de la llista proposada abans d'escriure res, on decidia si el conjunt de casos cobria el que calia cobrir. La segona és l'execució: les 474 proves del motor i les 302 de la interfície s'executen sense cap fallada sobre codi que he escrit jo mateix.

Els programes d'exemple es van verificar per execució dins de la pròpia eina. Un programa que faci servir una construcció no admesa falla en compilar, de manera que el compilador actua com a filtre.

Una prova automàtica, però, només comprova que el codi fa el que es va decidir que fes. Per això el capítol de validació incorpora una via que no depèn de cap assistència: els valors esperats dels set exercicis es van obtenir treballant a mà sobre les traces d'execució, i comparar-los amb els que reporta l'eina és el que permet afirmar que el simulador es comporta com prediu el marc teòric.

La correcció del document s'ha revisat llegint el resultat complet. Els canvis afectaven ortografia, gramàtica i format, de manera que qualsevol modificació que hagués alterat el sentit d'una frase era detectable en la relectura.

9 Conclusions

Els quatre blocs de requisits funcionals fixats a l'abast es compleixen, i els casos d'ús de l'apartat 7.6 en són l'evidència. Els quatre **objectius pedagògics** de 3.1.1 queden demostrats i no només afirmats: E1 *relaciona* un patró d'accés amb el nombre de fallades que provoca, E2 permet *predir* l'efecte d'un canvi de paràmetre en reduir les fallades de 40 a 2 en passar de correspondència directa a dues vies, E3 fa visible *què* expulsa cada política i no només quantes fallades produeix, i E6 permet *seguir* una traducció d'adreça sencera amb una taula de pàgines de dos nivells. Dels requisits no funcionals, dos queden amb reserves que cal declarar: la **portabilitat** només s'ha comprovat a Windows i Linux, i la **interactivitat** està argumentada per construcció però no mesurada.

La decisió de la qual depèn tot el disseny és la **representació explícita de l'estat** d'un accés a memòria mitjançant un cursor. Sense ella no hi hauria avanç per acció atòmica, sense això no hi hauria inspecció intermèdia, i sense inspecció l'eina no es distingiria de les que ja existeixen. La segona decisió que s'ha defensat sola és la **separació estricta entre motor i presentació**. La prova va aparèixer durant la validació: l'arnès de verificació construeix la sessió amb la mateixa funció que fa servir l'aplicació real, sense necessitar l'entorn de Tauri, cosa que no hauria estat possible si la separació no s'hagués respectat.

L'aportació més defensable respecte de les eines revisades a 2.6 és que cap d'elles **modela latències**, i per tant cap permet contrastar l'AMAT mesurat amb el que l'usuari calcularia a mà a partir de les taxes i les latències. E5 demostra aquest exercici. És una aportació d'*abast delimitat* i no una superioritat general: les eines de recerca ofereixen una fidelitat amb el maquinari real que aquest projecte no busca.

Les limitacions del model estan declarades i justificades a l'abast, amb una excepció que cal reconèixer: la **política d'inclusió** s'accepta des de la interfície però no té cap efecte al motor. A la implementació queden pendents la navegació des d'un error de compilació fins a la línia de l'editor, la uniformització del tractament de configuracions invàlides i l'exportació de resultats. Les línies de treball futur, per ordre de valor, són les **proves amb usuaris reals**, la instrumentació del temps per pas, i la comprovació a macOS.

Aquest projecte ha estat un repte sobretot per l'**envergadura de la feina**: hi apareixen, dins d'un mateix codi, pràcticament tots els àmbits que s'han anat treballant al llarg del grau, des del disseny d'un compilador fins a l'arquitectura de computadors, l'enginyeria del programari i el desenvolupament d'una interfície, i cap d'aquestes parts es pot entendre del tot sense les altres. Cada decisió presa sobre el projecte ha acabat tenint conseqüències, bones i dolentes, i han estat precisament les *dolentes* les que han fet més evident com una decisió presa al principi

pot fer que les tasques que se'n deriven acabin excedint l'abast esperat.

Dues coses van sortir de la mida esperada. La primera és la quantitat de decisions que exigeix un motor que s'ha de poder aturar en qualsevol punt del seu recorregut: fins i tot una estructura que sembla senzilla acaba necessitant un *estat explícit* i una manera de reprendre's exactament on s'havia quedat. La segona és que la validació no només va confirmar el que ja se sabia: va **destapar coses que el codi no deia enlloc**, com el directori temporal compartit del compilador o la limitació d'usabilitat de les pestanyes Harvard, cap de les dues visible fins que el sistema es va posar sota una condició que ningú havia previst.

10 Línies de futur

La **primera línia**, i la de més valor, és la validació amb usuaris reals. Tot el valor pedagògic de l'eina està argumentat des dels requisits i demostrat amb els casos d'ús, però no s'ha mesurat amb l'usuari final per al qual es va dissenyar. La forma concreta seria una sessió amb estudiants de l'assignatura corresponent, amb un qüestionari abans i després de fer servir l'eina, que permetés contrastar si les quatre capacitats de l'apartat 3.1.1 milloren de veritat.

La **segona** és l'obertura de l'eina a l'ús docent regular. Ara mateix el que passa dins d'una sessió no surt de l'aplicació: no hi ha exportació de traces, d'estadístiques ni de configuracions. Poder desar una configuració i compartir-la permetria que un docent preparés un exercici i el distribuís ja muntat, i poder exportar els resultats obriria l'ús en pràctiques avaluables. Cap de les dues exigeix tocar el motor, i totes dues canvien substancialment com es pot fer servir l'eina.

La **tercera** és l'ampliació del model. Els mecanismes que l'abast va deixar fora deliberadament, el *prefetch* i el *buffer* d'escriptura en primer lloc, són els que més distorsionen la interpretació dels resultats, perquè fan semblar l'escriptura immediata pitjor del que és i els patrons regulars més costosos del que serien. Afegir-los com a mecanismes opcionals i desactivables per defecte mantindria l'atribuïbilitat que justifica l'omissió actual, i alhora permetria veure precisament què aporten.

La **quarta** agrupa les millores identificades durant la validació i la implementació: la navegació des d'un error de compilació fins a la línia de l'editor, la uniformització del tractament de configuracions invàlides amb un únic canal de retroalimentació, la generació automàtica dels tipus que creuen el pont entre les dues capes, la instrumentació del temps per pas, i la comprovació pendent a macOS.

11 Planificació temporal

El treball s'ha organitzat en quatre fases. La primera va ser de conceptualització, dedicada a delimitar què havia de fer l'eina i a plantejar les alternatives de disseny abans d'escriure codi. La segona, la implementació, es va estructurar sobre el repositori: cada funcionalitat gran correspon a una **història** (branques amb prefix `S`) i cada història es desglossa en **tasques tècniques** (prefix `TT`), numerades amb l'identificador de la història a la qual pertanyen. La branca `TT4.2b` és l'única excepció a aquest esquema: el seu contingut entrava dins de l'abast de `TT4.2`, però tenia prou entitat per separar-la. La tercera fase és la validació, feta en dos moments: una validació de cada història en tancar-la, i una de global un cop integrades totes. La quarta és la redacció de la memòria, que va començar uns dies abans de tancar el desenvolupament i va avançar en paral·lel amb les últimes tasques. Les històries es van abordar en l'ordre `S1`, `S3`, `S2`, `S4` i `S5`.

El repartiment de temps entre històries respon a les dependències que arrossega cadascuna. `S1` i `S3` van ser les més ràpides, perquè no depenien de res i es podien construir de manera aïllada. `S2` va requerir més temps en dependre de `S1`, i `S4` és, amb diferència, la història més llarga: depèn de totes les anteriors, i bona part de les hores es van dedicar a resoldre problemes que apareixien a les branques prèvies un cop integrades. `S5` depèn principalment de `S4` i va tenir un cost intermedi. Dins de cada història, les tasques tècniques es van desenvolupar de manera seqüencial. El període de treball va de l'1 de juny al 15 de setembre, data límit de lliurament de la memòria.

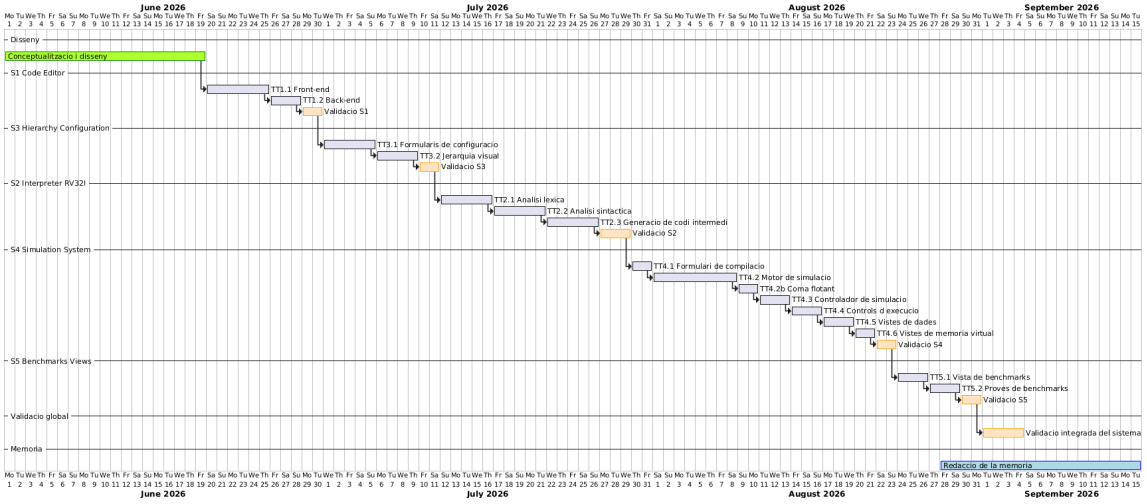


Figura 21: Planificació temporal en diagrama de Gantt. Font: elaboració pròpia [16].

La figura 22 mostra com es reparteix l'esforç entre els diferents tipus de tasca. La implementació concentra més de la meitat del temps, repartida entre el motor de simulació i la interfície, amb un pes més gran del primer per la complexitat de la jerarquia i del compilador. La conceptualització ocupa una sisena part del total, que correspon a les tres primeres setmanes dedicades a delimitar l'abast i plantejar el disseny abans d'escriure codi. La validació apareix amb un pes relativament baix perquè la redacció de les proves unitàries es va fer en paral·lel al desenvolupament, tal com s'explica a l'apartat 8, el percentatge recull les hores de disseny dels casos de prova, la verificació manual dels exercicis i la validació integrada del sistema.

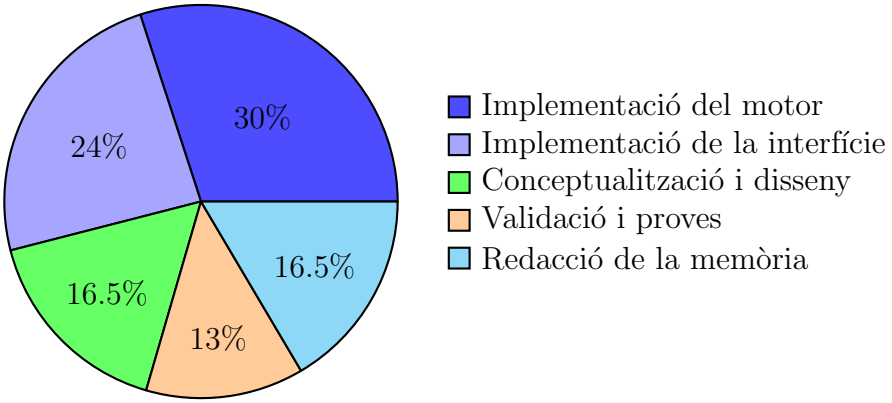


Figura 22: Distribució de l'esforç per tipus de tasca. Font: elaboració pròpia.

El ritme de treball no ha estat constant. Al juny i la primera setmana de juliol encara quedaven assignatures del grau per enllestir, de manera que la dedicació es va limitar a unes 3 hores diàries entre setmana i 2 hores els dissabtes. De la segona setmana de juliol fins a l'última d'agost la dedicació va pujar a 8 hores diàries entre setmana, amb unes 2 hores diàries més els caps de setmana per acabar el que havia quedat pendent. Del 1 al 10 de setembre, ja en el sprint final, es feien 10 hores diàries. Els últims cinc dies, de l'11 al 15, es van dedicar únicament a corregir la memòria amb els comentaris del tutor: unes 8 hores en total. En total, el projecte ha suposat unes **545 hores**.

Referències

[1] C. Carvalho. The gap between processor and memory speeds. pages 27–34, 01 2002.

[2] P. J. Denning. The locality principle. *Commun. ACM*, 48(7):19–24, July 2005.

[3] J. Edler and M. D. Hill. Dinero IV: Trace-driven uniprocessor cache simulator. `https://pages.cs.wisc.edu/~markhill/DineroIV/`, 2023. Consultat el 10 de setembre de 2026.

[4] N. Gober, G. Chacon, L. Wang, P. V. Gratz, D. A. Jimenez, E. Teran, S. Pugsley, and J. Kim. The championship simulator: Architectural simulation for education and competition. *arXiv preprint arXiv:2210.14324*, 2022.

[5] Google. Angular documentation. `https://angular.dev/`, 2026. Consultat el 10 de setembre de 2026.

[6] J. L. Hennessy and D. A. Patterson. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, Cambridge, MA, 6 edition, 2019.

[7] M. D. Hill and A. J. Smith. Evaluating associativity in CPU caches. volume 38, pages 1612–1630, 1989.

[8] J. Lowe-Power, A. M. Ahmad, A. Akram, M. Alian, R. Amslinger, M. Andreozzi, A. Armejach, N. Asmussen, B. Beckmann, S. Bharadwaj, et al. The gem5 simulator: Version 20.0+. *arXiv preprint arXiv:2007.03152*, 2020.

[9] Meta Platforms, Inc. React documentation. `https://react.dev/`, 2026. Consultat el 10 de setembre de 2026.

[10] G. E. Moore. Cramming more components onto integrated circuits, reprinted from electronics, volume 38, number 8, april 19, 1965, pp.114 ff. *IEEE Solid-State Circuits Society Newsletter*, 11(3):33–35, 2006.

[11] Neutralinojs Contributors. Neutralinojs documentation. `https://neutralino.js.org/docs/`, 2026. Consultat el 10 de setembre de 2026.

[12] npm, Inc. npm documentation. `https://docs.npmjs.com/`, 2026. Consultat el 10 de setembre de 2026.

[13] OpenJS Foundation. Electron documentation. `https://www.electronjs.org/docs/latest/`, 2026. Consultat el 10 de setembre de 2026.

[14] D. A. Patterson. *Computer architecture*. Morgan Kaufmann, 1996.

[15] D. A. Patterson. *The morgan kaufmann series in Computer Architecture and Design: Computer Organization and Design, Revised Printing, third edition: The hardware*. Morgan Kaufmann, 2007.

[16] PlantUML Contributors. Plantuml language reference guide. `https://plantuml.com/`, 2026. Consultat el 10 de setembre de 2026.

[17] Ripes Contributors. Ripes: A graphical processor simulator and assembly editor for the RISC-V ISA. `https://ripes.me/Ripes/`, 2026. Consultat el 10 de setembre de 2026.

[18] Rust Project. The cargo book. `https://doc.rust-lang.org/cargo/`, 2026. Consultat el 10 de setembre de 2026.

[19] Rust Project. The rust programming language. `https://www.rust-lang.org/`, 2026. Consultat el 10 de setembre de 2026.

[20] P. Sanderson and K. Vollmar. MARS: MIPS assembler and runtime simulator. `https://github.com/dpetersanderson/MARS`, 2026. Consultat el 10 de setembre de 2026.

[21] Serde Contributors. Serde: A serialization framework for rust. `https://serde.rs/`, 2026. Consultat el 10 de setembre de 2026.

[22] R. E. Smith. A historical overview of computer architecture. *IEEE Annals of the History of Computing*, 10(4):277–303, Oct 1988.

[23] SolidJS Contributors. Solidjs documentation. `https://docs.solidjs.com/`, 2026. Consultat el 10 de setembre de 2026.

[24] Svelte Contributors. Svelte documentation. `https://svelte.dev/docs`, 2026. Consultat el 10 de setembre de 2026.

[25] Tauri Contributors. Tauri documentation. `https://tauri.app/`, 2026. Consultat el 10 de setembre de 2026.

[26] F. Tiosso, S. M. Bruschi, P. S. Lopes de Souza, and C. E. de Andrade Cacho. Amnesia: Cache simulator. `https://www.merlot.org/merlot/viewMaterial.htm?id=929863`, 2015. Simulador de jerarquia de memòria amb finalitats educatives. Consultat el 10 de setembre de 2026.

[27] Valgrind Developers. Cachegrind: a high-precision tracing profiler. `https://valgrind.org/docs/manual/cg-manual.html`, 2026. Consultat el 10 de setembre de 2026.

[28] Vite Contributors. Vite documentation. `https://vite.dev/`, 2026. Consultat el 10 de setembre de 2026.

[29] Vitest Contributors. Vitest documentation. `https://vitest.dev/`, 2026. Consultat el 10 de setembre de 2026.

[30] Vue.js Contributors. Vue.js documentation. `https://vuejs.org/`, 2026. Consultat el 10 de setembre de 2026.

[31] Wails Contributors. Wails documentation. `https://wails.io/docs/introduction/`, 2026. Consultat el 10 de setembre de 2026.

[32] A. Waterman and K. Asanović. The RISC-V instruction set manual, volume I: Unprivileged ISA. Technical report, RISC-V International, 2019. Document Version 20191213.

[33] xd009642. Tarpaulin: A code coverage tool for rust. `https://github.com/xd009642/tarpaulin`, 2026. Consultat el 10 de setembre de 2026.

A Subconjunt de RISC-V admès

Aquest apèndix recull tot el que el compilador de l'eina accepta sense error. Les instruccions que no hi apareixen no estan implementades: no s'admet cap instrucció de les extensions A ni C.

A.1 RV32I

Mnemònic	Format	Operands	Descripció
add	R	rd, rs1, rs2	Suma
sub	R	rd, rs1, rs2	Resta
sll	R	rd, rs1, rs2	Desplaçament lògic a l'esquerra
slt	R	rd, rs1, rs2	Comparació amb signe
sltu	R	rd, rs1, rs2	Comparació sense signe
xor	R	rd, rs1, rs2	XOR bit a bit
srl	R	rd, rs1, rs2	Desplaçament lògic a la dreta
sra	R	rd, rs1, rs2	Desplaçament aritmètic a la dreta
or	R	rd, rs1, rs2	OR bit a bit
and	R	rd, rs1, rs2	AND bit a bit

Taula 16: RV32I: operacions aritmètiques i lògiques entre registres. Font: elaboració pròpia a partir de [32].

Mnemònic	Format	Operands	Descripció
addi	I	rd, rs1, imm	Suma amb immediat de 12 bits amb signe
slti	I	rd, rs1, imm	Comparació amb signe
sltiu	I	rd, rs1, imm	Comparació sense signe
xori	I	rd, rs1, imm	XOR amb immediat
ori	I	rd, rs1, imm	OR amb immediat
andi	I	rd, rs1, imm	AND amb immediat
slli	I	rd, rs1, shamt	Desplaçament lògic a l'esquerra
srli	I	rd, rs1, shamt	Desplaçament lògic a la dreta
srai	I	rd, rs1, shamt	Desplaçament aritmètic a la dreta

Taula 17: RV32I: operacions amb immediat i desplaçaments. Font: elaboració pròpia a partir de [32].

Mnemònic	Format	Operands	Descripció
lb	I	rd, imm(rs1)	Carrega un byte amb extensió de signe
lh	I	rd, imm(rs1)	Carrega una mitja paraula amb extensió de signe
lw	I	rd, imm(rs1)	Carrega una paraula
lbu	I	rd, imm(rs1)	Carrega un byte sense signe
lhu	I	rd, imm(rs1)	Carrega una mitja paraula sense signe
sb	S	rs2, imm(rs1)	Emmagatzema un byte
sh	S	rs2, imm(rs1)	Emmagatzema una mitja paraula
sw	S	rs2, imm(rs1)	Emmagatzema una paraula

Taula 18: RV32I: accés a memòria. Font: elaboració pròpia a partir de [32].

Mnemònic	Format	Operands	Descripció
beq	B	rs1, rs2, imm	Salta si són iguals
bne	B	rs1, rs2, imm	Salta si són diferents
blt	B	rs1, rs2, imm	Salta si rs1 és menor, amb signe
bge	B	rs1, rs2, imm	Salta si rs1 és major o igual, amb signe
bltu	B	rs1, rs2, imm	Salta si rs1 és menor, sense signe
bgeu	B	rs1, rs2, imm	Salta si rs1 és major o igual, sense signe
jal	J	rd, imm	Salta i desa l'adreça de retorn a rd
jalr	I	rd, imm(rs1)	Salta a una adreça calculada i desa el retorn

Taula 19: RV32I: salts. Font: elaboració pròpia a partir de [32].

Mnemònic	Format	Operands	Descripció
lui	U	rd, imm20	Carrega l'immediat desplaçat 12 bits
auipc	U	rd, imm20	Suma l'immediat desplaçat 12 bits al PC
fence	I	cap o dos identificadors	Sincronització de memòria
ecall	I	cap	Crida a l'entorn
ebreak	I	cap	Punt de ruptura

Taula 20: RV32I: immediats llargs i instruccions de sistema. Font: elaboració pròpia a partir de [32].

A.2 RV64I

Mnemònic	Format	Operands	Descripció
lwu	I	rd, imm(rs1)	Carrega una paraula sense extensió de signe
ld	I	rd, imm(rs1)	Carrega una doble paraula
sd	S	rs2, imm(rs1)	Emmagatzema una doble paraula
addiw	I	rd, rs1, imm	Suma amb immediat, resultat de 32 bits
slliw	I	rd, rs1, shamt	Desplaçament lògic a l'esquerra de 32 bits
srliw	I	rd, rs1, shamt	Desplaçament lògic a la dreta de 32 bits
sraiw	I	rd, rs1, shamt	Desplaçament aritmètic a la dreta de 32 bits
addw	R	rd, rs1, rs2	Suma de 32 bits
subw	R	rd, rs1, rs2	Resta de 32 bits
sllw	R	rd, rs1, rs2	Desplaçament lògic a l'esquerra de 32 bits
srlw	R	rd, rs1, rs2	Desplaçament lògic a la dreta de 32 bits
sraw	R	rd, rs1, rs2	Desplaçament aritmètic a la dreta de 32 bits

Taula 21: RV64I: instruccions afegides sobre RV32I.
Font: elaboració pròpia a partir de [32].

A.3 Extensió M

Mnemònic	Format	Operands	Descripció
mul	R	rd, rs1, rs2	Producte, bits baixos
mulh	R	rd, rs1, rs2	Producte amb signe, bits alts
mulhsu	R	rd, rs1, rs2	Producte de rs1 amb signe per rs2 sense signe
mulhu	R	rd, rs1, rs2	Producte sense signe, bits alts
div	R	rd, rs1, rs2	Divisió amb signe
divu	R	rd, rs1, rs2	Divisió sense signe
rem	R	rd, rs1, rs2	Residu amb signe
remu	R	rd, rs1, rs2	Residu sense signe

Taula 22: Extensió M: multiplicació i divisió enteres.
Font: elaboració pròpia a partir de [32].

A.4 Extensions de coma flotant

Les extensions F, D i Q comparteixen el mateix conjunt d'operacions i es distingeixen pel sufix del mnemònic: `.s` per a precisió simple, `.d` per a doble i `.q` per a quàdruple. La taula 23 recull les operacions comunes a les tres, i la taula 24 les conversions, que sí que difereixen entre extensions.

Mnemònic	Format	Operands	Descripció
flw, fld, flq	I	rd, imm(rs1)	Carrega des de memòria
fsw, fsd, fsq	S	rs2, imm(rs1)	Emmagatzema a memòria
fadd.x	R	rd, rs1, rs2	Suma
fsub.x	R	rd, rs1, rs2	Resta
fmul.x	R	rd, rs1, rs2	Multiplicació
fdiv.x	R	rd, rs1, rs2	Divisió
fsqrt.x	R	rd, rs1	Arrel quadrada
fmin.x	R	rd, rs1, rs2	Mínim
fmax.x	R	rd, rs1, rs2	Màxim
fsgnj.x	R	rd, rs1, rs2	Copia el signe de rs2
fsgnjn.x	R	rd, rs1, rs2	Copia el signe negat de rs2
fsgnjx.x	R	rd, rs1, rs2	Copia el XOR dels dos signes

Mnemònic	Format	Operands	Descripció
feq.x	R	rd, rs1, rs2	Igualtat
flt.x	R	rd, rs1, rs2	Menor que
fle.x	R	rd, rs1, rs2	Menor o igual
fclass.x	R	rd, rs1	Classifica el valor
fmadd.x	R4	rd, rs1, rs2, rs3	Multiplicació i suma combinades
fmsub.x	R4	rd, rs1, rs2, rs3	Multiplicació i resta combinades
fnmadd.x	R4	rd, rs1, rs2, rs3	Multiplicació i suma amb signe negat
fnmsub.x	R4	rd, rs1, rs2, rs3	Multiplicació i resta amb signe negat

Taula 23: Operacions disponibles a les extensions F, D i Q. Substituïu `.x` pel sufix de l'extensió. Font: elaboració pròpia a partir de [32].

Mnemònic	Extensió	Descripció
fcvt.w.s	F	De simple precisió a enter amb signe
fcvt.wu.s	F	De simple precisió a enter sense signe
fcvt.s.w	F	D'enter amb signe a simple precisió
fcvt.s.wu	F	D'enter sense signe a simple precisió
fmv.x.w	F	Mou el patró de bits a un registre enter
fmv.w.x	F	Mou el patró de bits des d'un registre enter
fcvt.w.d	D	De doble precisió a enter amb signe
fcvt.wu.d	D	De doble precisió a enter sense signe
fcvt.d.w	D	D'enter amb signe a doble precisió
fcvt.d.wu	D	D'enter sense signe a doble precisió
fcvt.s.d	D	De doble a simple precisió
fcvt.d.s	D	De simple a doble precisió
fcvt.w.q	Q	De quàdruple precisió a enter amb signe
fcvt.wu.q	Q	De quàdruple precisió a enter sense signe
fcvt.q.w	Q	D'enter amb signe a quàdruple precisió
fcvt.q.wu	Q	D'enter sense signe a quàdruple precisió
fcvt.s.q	Q	De quàdruple a simple precisió
fcvt.q.s	Q	De simple a quàdruple precisió
fcvt.d.q	Q	De quàdruple a doble precisió
fcvt.q.d	Q	De doble a quàdruple precisió

Taula 24: Conversions i moviments entre formats. Font: elaboració pròpia a partir de [32].

A.5 Pseudoinstruccions

L'eina admet vint-i-quatre pseudoinstruccions, que s'expandeixen a instruccions reals durant la generació de codi. Totes produeixen una única instrucció llevat de `la` i, en determinats casos, `li`. La línia i la columna de la pseudoinstrucció original es copien a totes les instruccions que en resulten.

Pseudoinstrucció	Expansió	Observacions
nop	addi x0, x0, 0	
li rd, imm	addi rd, x0, imm	Si l'immediat cap en 12 bits amb signe
	lui rd, hi	Altrament, amb la part baixa ajustada al seu valor amb signe
	addi rd, rd, lo	
mv rd, rs	addi rd, rs, 0	
not rd, rs	xori rd, rs, -1	
neg rd, rs	sub rd, x0, rs	
seqz rd, rs	sltiu rd, rs, 1	
snez rd, rs	sltu rd, x0, rs	
sltz rd, rs	slt rd, rs, x0	
sgtz rd, rs	slt rd, x0, rs	
beqz rs, etiqueta	beq rs, x0, etiqueta	
bnez rs, etiqueta	bne rs, x0, etiqueta	
blez rs, etiqueta	bge x0, rs, etiqueta	
bgez rs, etiqueta	bge rs, x0, etiqueta	
bltz rs, etiqueta	blt rs, x0, etiqueta	
bgtz rs, etiqueta	blt x0, rs, etiqueta	
bgt rs1, rs2, etiq.	blt rs2, rs1, etiq.	Operands intercanviats
ble rs1, rs2, etiq.	bge rs2, rs1, etiq.	Operands intercanviats
bgtu rs1, rs2, etiq.	bltu rs2, rs1, etiq.	Operands intercanviats
bleu rs1, rs2, etiq.	bgeu rs2, rs1, etiq.	Operands intercanviats
j etiqueta	jal x0, etiqueta	

Pseudoinstrucció	Expansió	Observacions
jr rs	jalr x0, 0(rs)	
ret	jalr x0, 0(x1)	Registre de retorn fix a x1
call etiqueta	jal x1, etiqueta	Desa l'adreça de retorn a x1
tail etiqueta	jal x0, etiqueta	Expansió idèntica a `j`
la rd, etiqueta	auipc rd, etiqueta addi rd, rd, etiqueta	La divisió en part alta i baixa es resol més endavant

Taula 25: Pseudoinstruccions i la seva expansió. Font: elaboració pròpia a partir de [32].

A.6 Directives

Directiva	Arguments	Bytes	Efecte
.text	cap	0	Canvia la secció activa a text
.data	cap	0	Canvia la secció activa a dades
.bss	cap	0	Es comporta igual que `.data` i no reserva zeros
.global, .globl	una etiqueta	0	Es reconeixen sense cap efecte
.word	immediats o etiquetes	4	Les etiquetes es resolen en acabar el disseny de memòria
.half	immediats o etiquetes	2	Mateix mecanisme que `.word`
.byte	immediats	1	Un argument que no sigui immediat es descarta sense error
.string, .asciz	una cadena	n+1	Bytes de la cadena amb terminador nul
.ascii	una cadena	n	Bytes de la cadena sense terminador

Taula 26: Directives reconegudes. Font: elaboració pròpia.

Les directives `.space` i `.align` no estan implementades i el seu ús es rebutja amb un error de directiva desconeguda.

A.7 Noms de registre

A més de la notació `xN` i `fN`, s'admeten els noms de l'ABI.

Àlies	Registre	Àlies	Registre	Àlies	Registre
zero	x0	a1	x11	s7	x23
ra	x1	a2	x12	s8	x24
sp	x2	a3	x13	s9	x25
gp	x3	a4	x14	s10	x26
tp	x4	a5	x15	s11	x27
t0	x5	a6	x16	t3	x28
t1	x6	a7	x17	t4	x29
t2	x7	s2	x18	t5	x30
s0, fp	x8	s3	x19	t6	x31
s1	x9	s4	x20		
a0	x10	s5	x21		
		s6	x22		

Taula 27: Àlies dels registres enters. Font: elaboració pròpia a partir de [32].

Àlies	Registres
ft0–ft7	f0–f7
fs0–fs1	f8–f9
fa0–fa7	f10–f17
fs2–fs11	f18–f27
ft8–ft11	f28–f31

Taula 28: Àlies dels registres de coma flotant. Font: elaboració pròpia a partir de [32].

A.8 Sintaxi admesa

Format	Prefix	Exemple
Decimal	cap	42
Hexadecimal	`0x` o `0X`	0x2A
Binari	`0b` o `0B`	0b101010

Taula 29: Formats d'immediat admesos. Font: elaboració pròpia.

Els tres formats admeten valors negatius, perquè el signe es reconeix com un element independent que el parser aplica abans de llegir el literal. No hi ha cap prefix per a notació octal: un literal com `010` es llegeix com el decimal 10.

Regla	Valor
Primer caràcter	Lletra ASCII o guió baix. No pot ser un dígit
Caràcters següents	Alfanumèrics ASCII o guió baix
Majúscules	Es distingeixen: la comparació és literal
Per línia	Una sola etiqueta, que pot compartir línia amb una instrucció

Taula 30: Regles de formació d'etiquetes. Font: elaboració pròpia.

Forma	Sintaxi	Observacions
De línia	`#`	Arriba fins al final de la línia. No es reconeix dins d'una cadena entre cometes
De bloc	`/* */`	Pot ocupar diverses línies. Si no es tanca, es genera un error de comentari sense terminar

Taula 31: Formes de comentari admeses. Font: elaboració pròpia.

Els literals de cadena no admeten cap seqüència d'escapament. La cadena es tanca sempre a la primera cometa doble que es troba, de manera que construccions com `\n` o `\"` es tracten com a caràcters literals.

A la sintaxi d'accés a memòria `imm(base)`, el desplaçament admet un signe opcional seguit d'un únic valor numèric en qualsevol dels tres formats de la taula 29, sense expressions ni operacions. La base ha de ser un registre; qualsevol altra cosa entre parèntesis genera un error.

A.9 Límits i comprovacions

Instrucció	**Format**	**Rang en bytes**
beq, bne, blt, bge, bltu, bgeu	B	-4096 a 4094
jal	J	$-1\,048\,576$ a $1\,048\,574$

Taula 32: Rangs de salt comprovats durant la generació de codi. Font: elaboració pròpia.

Excedir aquests rangs genera un error de destinació de salt fora de rang. El parell d'instruccions que genera `la`, en canvi, no té cap comprovació de rang, de manera que un desplaçament que no cabés al camp d'immediat es codificaria sense avís.

Condició	**Error**
Instrucció de RV64I amb base RV32I	La instrucció requereix la base RV64I
Instrucció d'una extensió no activada	La instrucció requereix l'extensió corresponent
Nombre d'operands incorrecte	S'indiquen els operands esperats i els trobats
Registre enter on se n'espera un de coma flotant, o a la inversa	S'indica el tipus de registre esperat
Immediat de 12 bits amb signe fora de rang	L'immediat no cap als bits disponibles
Quantitat de desplaçament fora de rang	L'immediat no cap als bits sense signe disponibles
`lui` o `auipc` amb immediat de més de 20 bits	Mateix error que l'anterior
Segon operand d'una càrrega o emmagatzematge que no té la forma `imm(base)`	S'espera un operand de desplaçament
Salt sense etiqueta com a operand final	S'espera un operand d'etiqueta
`fence` amb un nombre d'operands diferent de zero o dos	S'esperen zero o dos operands
`fence` amb un operand que no és un identificador	Els operands han de ser identificadors
`.text`, `.data` o `.bss` amb arguments	La directiva no accepta arguments
`.global` sense exactament una etiqueta	S'espera un únic argument d'etiqueta
`.word`, `.half` o `.byte` sense valors	S'espera com a mínim un valor
`.word`, `.half` o `.byte` amb un valor que no és immediat ni etiqueta	S'esperen immediats o etiquetes
`.string`, `.asciz` o `.ascii` sense exactament una cadena	S'espera un únic argument de cadena
Directiva desconeguda	Directiva no reconeguda
Etiqueta duplicada	L'etiqueta ja està definida
Referència a una etiqueta inexistent	Etiqueta no definida

Taula 33: Comprovacions de l'anàlisi semàntica. Font: elaboració pròpia.

Finalment, hi ha construccions del llenguatge estàndard que l'eina rebutja de manera deliberada. La directiva `.include` genera un error que indica que només s'admet un únic fitxer, decisió coherent amb l'absència de macros i amb la dimensió dels programes que l'eina espera. Les directives `.space` i `.align` no estan implementades i cauen a l'error de directiva desconeguda.

B Programes dels exercicis de verificació

Aquest apèndix recull els programes en assemblador RISC-V dels set exercicis de verificació manual del capítol 7. Tots s'executen amb la llavor fixada a 42 i amb el percentatge de línies brutes a zero, de manera que els resultats són reproduïbles.

B.1 E1. Localitat espacial

Recorregut seqüencial d'un vector de 64 paraules sobre un únic nivell Harvard de 2^6 entrades amb correspondència directa, executat amb blocs de 16 i de 32 bytes.

```
.data
array:
.word 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
.word 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
.word 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
.word 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
.text
start:
  la x1, array
  addi x2, x0, 64
loop:
  lw x3, 0(x1)
  addi x1, x1, 4
  addi x2, x2, -1
  bne x2, x0, loop
```

B.2 E2. Fallades de conflicte

Bucle de 20 iteracions que llegeix alternadament dues adreces separades exactament per la mida de la memòria cau, sobre un nivell Harvard de 2^4 entrades amb blocs de 16 bytes. S'executa amb correspondència directa i amb dues vies i política LRU.

```
.data
bufA: .word 0
.text
start:
  la x1, bufA
  addi x2, x1, 256
  addi x3, x0, 20
loop:
  lw x4, 0(x1)
```

```
  lw x5, 0(x2)
  addi x3, x3, -1
  bne x3, x0, loop
```

B.3 E3. Divergència entre polítiques de reemplaçament

Seqüència d'accessos sobre tres adreces contigües, dissenyada perquè les polítiques prenguin decisions d'expulsió diferents. La configuració és un nivell Harvard de 2^1 entrades amb dues vies, de manera que hi ha un únic conjunt útil, i blocs de 4 bytes. S'executa amb LRU, FIFO, LFU i política aleatòria.

```
.data
vals: .word 0, 0, 0
.text
start:
  la x1, vals
  addi x2, x1, 4
  addi x3, x1, 8
  lw x4, 0(x1)
  lw x5, 0(x2)
  lw x4, 0(x1)
  lw x4, 0(x1)
  lw x6, 0(x3)
  lw x5, 0(x2)
  lw x6, 0(x3)
  lw x6, 0(x3)
```

Després dels tres primers accessos, l'adreça A acumula una freqüència d'ús més alta que B però va ser inserida abans. En arribar C amb el conjunt ple, cada política mira un criteri diferent i expulsa un bloc diferent. La cua d'accessos repeteix un únic candidat perquè aquesta decisió es reflecteixi al recompte final.

B.4 E4. Polítiques d'escriptura

Bucle de 10 iteracions que escriu sobre dues posicions que no s'han llegit abans, de manera que la primera escriptura a cadascuna és una fallada d'escriptura real. La configuració és un nivell Harvard de 2^2 entrades amb correspondència directa i blocs de 4 bytes, amb una latència de memòria principal molt superior a la d'encert. S'executa amb escriptura immediata, amb escriptura diferida i reserva, i amb escriptura diferida sense reserva.

```
.data
vals: .word 0, 0
.text
start:
  la x1, vals
  addi x2, x1, 4
  addi x3, x0, 7
  addi x4, x0, 10
loop:
  sw x3, 0(x1)
  sw x3, 0(x2)
  addi x4, x4, -1
  bne x4, x0, loop
```

B.5 E5. Mètriques amb dos nivells

Fa servir el mateix programa que E1, executat sobre una jerarquia de dos nivells Harvard, de 2^6 i 2^8 entrades, amb latències d'encert diferents a cada nivell.

B.6 E6. Traducció d'adreces

Sis lectures sobre tres pàgines diferents amb repetició, amb memòria virtual activa i sense cap nivell de memòria cau, per aïllar el camí de traducció. La TLB té dues entrades i la taula de pàgines dos nivells, amb marcs suficients perquè no hi hagi cap expulsió.

```
.text
start:
  li x1, 0x10000000
  li x2, 0x10001000
  li x3, 0x10002000
  lw x4, 0(x1)
  lw x4, 0(x2)
  lw x4, 0(x1)
  lw x4, 0(x3)
  lw x4, 0(x1)
  lw x4, 0(x2)
```

B.7 E7. Thrashing per pressió de marcs

Fa servir el mateix programa que E4, executat sota la configuració d'E6 reduïda a dos marcs. Reutilitzar un programa ja verificat sota una configuració nova permet atribuir el canvi de comportament únicament a la configuració. El bucle recorre dues pàgines de codi i una de dades, de manera que amb dos marcs disponibles la pàgina que cada iteració necessita acaba de ser expulsada per l'anterior.

C Detall de les proves per fitxer

Aquest apèndix recull el repartiment complet dels casos de prova. La versió agrupada per subsistema, amb la interpretació de les xifres, és a l'apartat 7.2.

C.1 Proves del motor

Fitxer	Casos
models/simulation/cpu.rs	100
models/simulation/memory/hierarchy.rs	45
models/simulation/memory/cache.rs	28
models/interpreter/riscv/codegen/expand.rs	28
models/interpreter/riscv/codegen/encode.rs	23
controllers/simulation_session_controller.rs	23
models/simulation/memory/page_table.rs	22
models/simulation/memory/tlb.rs	18
models/simulation/memory/backing_store.rs	16
models/interpreter/riscv/semantic/checks.rs	16
models/simulation/memory/frames.rs	15
controllers/memory_contents_controller.rs	15
models/interpreter/riscv/parser/grammar.rs	12
models/simulation/stats.rs	11
models/simulation/session.rs	11
models/simulation/memory/replacement.rs	11
models/interpreter/riscv/lexer/tokenizer.rs	11
models/interpreter/riscv/mod.rs	8
models/interpreter/riscv/lexer/token.rs	8
models/interpreter/riscv/codegen/layout.rs	8
models/simulation/memory/config.rs	7
models/interpreter/riscv/lexer/preprocessor.rs	7
models/simulation/memory/seeded_random.rs	6
models/interpreter/error.rs	6
models/interpreter/riscv/semantic/mod.rs	5
models/interpreter/riscv/codegen/mod.rs	5
models/interpreter/riscv/semantic/symbol_table.rs	3
models/interpreter/riscv/parser/mod.rs	2
models/interpreter/riscv/parser/ast.rs	1
models/interpreter/riscv/codegen/ir.rs	1
controllers/stats_controller.rs	1
controllers/editor_controller.rs	1

Fitxer	Casos
Total	**474**

Taula 34: Repartiment dels casos de prova del motor per fitxer. Font: elaboració pròpia.

C.2 Proves de la interfície

Fitxer	Casos
lib/hierarchy/diagram.test.ts	21
lib/hierarchy/derive.test.ts	21
lib/hierarchy/operations.test.ts	20
stores/simulation.test.ts	14
components/simulation/RamView.test.ts	14
stores/hierarchyConfig.test.ts	13
components/simulation/CacheLevelView.test.ts	13
components/hierarchy-config/HierarchyDiagram.test.ts	13
components/compile-config/RiscvCompileConfigForm.test.ts	13
components/simulation/SimulationToolbar.test.ts	9
App.simulation.test.ts	9
components/simulation/DiskView.test.ts	8
lib/lruCache.test.ts	7
components/ui/HexField.test.ts	7
components/simulation/RegistersView.test.ts	7
components/simulation/PageTableView.test.ts	7
components/hierarchy-config/GlobalConfigBar.test.ts	7
lib/virtualizedWindow.test.ts	6
components/simulation/TlbView.test.ts	6
components/simulation/FramesView.test.ts	6
components/hierarchy-config/HierarchySlider.menu.test.ts	6
components/hierarchy-config/CacheConfigForm.test.ts	6
lib/virtualScroll.test.ts	5
components/simulation/StatsView.test.ts	5
components/simulation/stats-cells/IntegerStat.test.ts	5
components/simulation/SimulationInfoPanel.test.ts	5
components/hierarchy-config/PowerOfTwoStepper.test.ts	5
components/hierarchy-config/HierarchySlider.render.test.ts	5
components/simulation/stats-cells/StackedPercentStat.test.ts	4
components/hierarchy-config/TlbConfigForm.test.ts	4
components/hierarchy-config/LevelTabs.test.ts	4

Fitxer	**Casos**
`components/hierarchy-config/HierarchySlider.interaction.test.ts`	4
`components/Editor.simulation.test.ts`	4
`stores/compileConfig.test.ts`	3
`components/simulation/stats-cells/PercentStat.test.ts`	3
`components/Editor.compileConfig.test.ts`	3
`components/simulation/stats-cells/DecimalStat.test.ts`	2
`components/hierarchy-config/HierarchyConfigPanel.test.ts`	2
`components/Editor.isa.test.ts`	2
`components/compile-config/CompileConfigForm.test.ts`	2
`components/simulation/StatsButton.test.ts`	1
`bootstrap.test.ts`	1
Total	**302**

Taula 35: Repartiment dels casos de prova de la interfície per fitxer. Font: elaboració pròpia.